

Поставновка задачи

Это домашнее задание посвящено полноценному решению задачи машинного обучения.

Есть две части этого домашнего задания:

Отправить ваши предсказания в Stepik.

За прохождение определенных порогов будут начисляться баллы. Эта часть оценивается из 5 баллов.

1) $1.00 \geq score \geq 0.84$ --- 5 баллов

2) $0.84 > score \geq 0.77$ --- 4 балла

3) $0.77 > score \geq 0.70$ --- 3 балла

4) $0.70 > score \geq 0.65$ --- 2 балла

5) $0.65 > score \geq 0.6$ --- 1 балл

6) $0.60 > score$ --- 0 баллов

Для этого мы предварительно разделили данные в задании на две части.

- `train.csv`. На этом наборе данных вам необходимо создать и обучить модель. Подробное описание файла есть в ноутбуке.
- `test.csv`. В каждой строчке файла `test.csv` указаны признаки тестовых объектов. Предсказания для этого набора необходимо записать в файл `submission.csv` и сдать в соответствующий шаг на Stepik. Количество попыток ограничено 100 штук.

Отправлять в Stepik вы будете файл с ответами `my_submission.csv`. В этом файле вам необходимо для каждого объекта из датасета предсказать вероятность класса 1. Мы подготовили для вас файл с примером отправки `submission.csv`. Ваш файл должен быть оформлен таким же образом. В ноутбуке есть пример оформления файла отправки. Для отправки файла с предсказаниями на Stepik есть отдельный шаг.

Сделать полноценный отчет о вашей работе.

Опишите, как вы обработали данные, какие модели попробовали и какие результаты получились (максимум 10 баллов). За каждую выполненную часть будет начислено определенное количество баллов. В этом пункте вам необходимо отправить файл в формате `.ipynb` на Stepik --- для этого в домашнем задании есть отдельный шаг. Этот пункт оценивается из 10 баллов.

Вторая часть будет проверяться в формате peer-review. Т.е. вашу отсылку на Stepik будут проверять 3 других студента, и медианное значение их оценок будет выставлено. Чтобы получить баллы, вам также нужно будет проверить трех других учеников. Это станет доступно после того, как вы сдадите задание сами.

Контекст на Kaggle

Для вашего удобства мы подготовили [контекст на Kaggle](#). Тестирующая система на Kaggle выдает те же баллы, что и Stepik. Мы рекомендуем в качестве основной тестирующей системы использовать именно Kaggle, а затем сдать лучшую отсылку на Stepik. Так вы привыкните работать с Kaggle.

Проблема для российских пользователей Kaggle заключается в том, что подтверждение аккаунта по номеру телефона работает с перебоями или не работает вообще. Без подтвержденного номера телефона вы не сможете совершать отсылки в kaggle. Если у вас возникла такая проблема, мы советуем следующие шаги:

- Попробовать зарегистрировать аккаунт с аутентификацией через gmail. В таком случае, аккаунт должен работать без подтверждения номера телефона.
- Если у вас есть не российский номер, используйте его для верификации аккаунта.
- Если на номер телефона не приходит смс для верификации аккаунта, обратитесь в службу поддержки kaggle. Они с переменным успехом умеют подтверждать аккаунты вручную.
- Если все же подтвердить аккаунт и получить доступ к сдаче заданий не удалось, используйте только сдачу в Stepik.

Несколько замечаний по выполнению работы

- Во всех пунктах указания это минимальный набор вещей, которые стоит сделать. Если вы можете сделать какой-то шаг лучше или добавить что-то свое --- дерзайте!
- Пожалуйста, перед сдачей ноутбука убедитесь, что работа чистая и понятная. Это значительно облегчит проверку и повысит ваши ожидаемые баллы.
- Если у вас будут проблемы с решением или хочется совета, то пишите в наш чат в телеграме.

Данные: [train.csv](#), [test.csv](#), [submission.csv](#).

Если ссылки на данные не работают, их можно скачать [на Kaggle](#).

Как проверять?

Ставьте полный балл, если выполнены все рекомендации или сделано что-то более интересное и сложное. За каждый отсутствующий пункт из рекомендации снижайте 1 балл.

Если решение верное, но не удовлетворяет вашим эстетическим предпочтениям, за это баллы снижать не нужно.

Метрика

Перед решением любой задачи важно понимать, как будет оцениваться ваше решение. В данном случае мы используем стандартную для задачи классификации метрику ROC-AUC. Ее можно вычислить, используя только предсказанные вероятности и истинные классы без конкретного порога классификации + она работает даже если классы в данных сильно несбалансированы (примеров одного класса в десятки раз больше примеров другого). Именно поэтому она очень удобна для соревнований.

Посчитать ее легко:

```
from sklearn.metrics import roc_auc_score

y_true = [
    0,
    1,
    1,
    0,
    1
]

y_predictions = [
    0.1,
    0.9,
    0.4,
    0.6,
    0.61
]

roc_auc_score(y_true, y_predictions)

0.8333333333333333
```

Импорт библиотек, константы

```
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns
import phik

from sklearn.preprocessing import MinMaxScaler, StandardScaler,
PolynomialFeatures
from category_encoders.ordinal import OrdinalEncoder
from category_encoders.one_hot import OneHotEncoder
from category_encoders.target_encoder import TargetEncoder
from category_encoders.leave_one_out import LeaveOneOutEncoder

from sklearn.linear_model import LogisticRegression,
LogisticRegressionCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from catboost import CatBoostClassifier, Pool, cv
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import StackingClassifier

import optuna

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import roc_auc_score
from scipy import stats
import association_metrics as am
from sklearn.metrics import roc_curve, auc

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

import warnings
warnings.filterwarnings('ignore')
# sns.set_theme(palette='muted')
pd.set_option('display.max_columns', None)

RANDOM_STATE = 42

```

Исследование данных

Загрузка данных (2 балла)

1) Посмотрите на случайные строчки.

2) Посмотрите, есть ли в датасете незаполненные значения (nan'ы) с помощью data.isna() или data.info() и, если нужно, замените их на что-то. Будет хорошо, если вы построите табличку с количеством nan в каждой колонке.

```

# !gdown 1ERwQ5odiK1ZvilLtjpkzCMUswYsAX8_K # train.csv
# !gdown 1fGw_-RFwvn_LEdt91Jq-7A-wzG6mmH8r # test.csv
# !gdown 199Mt40YZNaelT83U-HGDsEYs2YcUGQ6y # submission.csv

# !mkdir content

# !gdown 1ERwQ5odiK1ZvilLtjpkzCMUswYsAX8_K -O content/train.csv
# !gdown 1fGw_-RFwvn_LEdt91Jq-7A-wzG6mmH8r -O content/test.csv
# !gdown 199Mt40YZNaelT83U-HGDsEYs2YcUGQ6y -O content/submission.csv

# root = './'
root = 'content/'

train = pd.read_csv(root + 'train.csv')
test = pd.read_csv(root + 'test.csv')
submission = pd.read_csv(root + 'submission.csv')

```

Обзор датасета

Размер датасета

train.shape

(5282, 20)

Случайные строки

train.sample(5)

	ClientPeriod	MonthlySpending	TotalSpent	Sex
IsSeniorCitizen \				
913	18	40.20	711.95	Female
0				
2763	72	79.35	5753.25	Male
0				
4461	72	25.10	1789.9	Male
1				
2522	61	101.15	6383.9	Female
0				
2545	48	108.10	5067.45	Male
0				

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
913	No	No	No	No phone service	
2763	Yes	No	Yes	No	
4461	No	No	Yes	Yes	
2522	Yes	No	Yes	Yes	
2545	No	No	Yes	Yes	

	HasInternetService	HasOnlineSecurityService	HasOnlineBackup
\			

913	DSL	Yes	Yes	
2763	DSL	Yes	Yes	
4461	No	No internet service	No internet service	
2522	Fiber optic	No	No	
2545	Fiber optic	No	Yes	
	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV \	
913	No	Yes	No	
2763	No	Yes	Yes	
4461	No internet service	No internet service	No internet service	
2522	No	Yes	Yes	
2545	Yes	No	Yes	
	HasMovieSubscription	HasContractPhone	IsBillingPaperless \	
913	No	One year	Yes	
2763	Yes	Two year	Yes	
4461	No internet service	Two year	Yes	
2522	Yes	One year	No	
2545	Yes	Two year	Yes	
	PaymentMethod	Churn		
913	Mailed check	0		
2763	Bank transfer (automatic)	0		
4461	Credit card (automatic)	0		
2522	Bank transfer (automatic)	0		
2545	Bank transfer (automatic)	0		
test.sample(5)				
	ClientPeriod	MonthlySpending	TotalSpent	Sex
IsSeniorCitizen \				
998	1	79.25	79.25	Male
0				
285	33	19.80	641.35	Female
0				
1115	21	96.80	2030.3	Male
1				
884	72	49.20	3580.95	Female
0				
4	71	20.10	1389.6	Female
0				
	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers \
998	No	No	Yes	No
285	Yes	Yes	Yes	No
1115	No	No	Yes	No

884	Yes	No	No	No phone service
4	Yes	Yes	Yes	No
HasInternetService HasOnlineSecurityService HasOnlineBackup				
998	Fiber optic		No	No
285	No	No internet service	No internet service	
1115	Fiber optic		No	No
884	DSL		Yes	Yes
4	No	No internet service	No internet service	
HasDeviceProtection HasTechSupportAccess HasOnlineTV \				
998	No	No	No	No
285	No internet service	No internet service	No internet service	
1115	Yes	No	Yes	
884	Yes	No	No	
4	No internet service	No internet service	No internet service	
HasMovieSubscription HasContractPhone IsBillingPaperless \				
998	Yes	Month-to-month	Yes	
285	No internet service	Two year	No	
1115	Yes	Month-to-month	Yes	
884	Yes	Two year	Yes	
4	No internet service	Two year	No	
PaymentMethod				
998	Electronic check			
285	Credit card (automatic)			
1115	Bank transfer (automatic)			
884	Electronic check			
4	Bank transfer (automatic)			

Описание колонок датасета

- **ClientPeriod:** количество периодов, которые клиент является клиентом компании.
- **MonthlySpending:** ежемесячные расходы клиента на услуги.
- **TotalSpent:** общая сумма, потраченная клиентом за всё время пользования услугами.
- **Sex:** пол клиента.
- **IsSeniorCitizen:** признак того, является ли клиент пожилым человеком.
- **HasPartner:** наличие у клиента супруга.
- **HasChild:** наличие у клиента детей.
- **HasPhoneService:** наличие у клиента телефонной услуги.
- **HasMultiplePhoneNumbers:** наличие нескольких телефонных номеров у клиента.

- **HasInternetService**: наличие интернет-услуги у клиента.
- **HasOnlineSecurityService**: наличие у клиента услуги онлайн-безопасности.
- **HasOnlineBackup**: наличие услуги облачного резервного копирования данных.
- **HasDeviceProtection**: наличие защиты устройств.
- **HasTechSupportAccess**: наличие доступа к технической поддержке.
- **HasOnlineTV**: наличие услуги онлайн-ТВ.
- **HasMovieSubscription**: наличие подписки на киносервисы/библиотеки фильмов.
- **HasContractPhone**: тип договора на телефонную связь.
- **IsBillingPaperless**: признак безбумажного биллинга (электронных счетов).
- **PaymentMethod**: способ оплаты услуг клиентом.

Целевой признак

- **Churn**: ушел ли клиент.

Проверка на наличие дубликатов

Сначала проверим датасеты на наличие дубликатов

```
print('Дубликаты в train:', train.duplicated().sum())
print('Дубликаты в test:', test.duplicated().sum())
```

```
train[train.duplicated(keep=False)].sort_values(by=[*train.columns]).head(6)
```

Дубликаты в train: 14

Дубликаты в test: 3

	ClientPeriod	MonthlySpending	TotalSpent	Sex
IsSeniorCitizen \				
838	1	19.65	19.65	Female
0				
1636	1	19.65	19.65	Female
0				
594	1	20.05	20.05	Male
0				
1474	1	20.05	20.05	Male
0				
3735	1	20.20	20.2	Male
0				
3749	1	20.20	20.2	Male
0				

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
838	No	No	Yes		No
1636	No	No	Yes		No
594	No	No	Yes		No
1474	No	No	Yes		No
3735	No	No	Yes		No

3749	No	No	Yes	No
	HasInternetService	HasOnlineSecurityService		HasOnlineBackup
838	No	No internet service	No internet service	No internet service
1636	No	No internet service	No internet service	No internet service
594	No	No internet service	No internet service	No internet service
1474	No	No internet service	No internet service	No internet service
3735	No	No internet service	No internet service	No internet service
3749	No	No internet service	No internet service	No internet service

	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV	\
838	No internet service	No internet service	No internet service	No internet service
1636	No internet service	No internet service	No internet service	No internet service
594	No internet service	No internet service	No internet service	No internet service
1474	No internet service	No internet service	No internet service	No internet service
3735	No internet service	No internet service	No internet service	No internet service
3749	No internet service	No internet service	No internet service	No internet service

	HasMovieSubscription	HasContractPhone	IsBillingPaperless	
PaymentMethod	\			
838	No internet service	Month-to-month	No	Mailed
1636	No internet service	Month-to-month	No	Mailed
594	No internet service	Month-to-month	No	Mailed
1474	No internet service	Month-to-month	No	Mailed
3735	No internet service	Month-to-month	No	Mailed
3749	No internet service	Month-to-month	No	Mailed

	Churn
838	0
1636	0
594	0
1474	0
3735	0
3749	0

Дубликаты в train без Churn

```
duplicates_without_target = train.drop('Churn',
```

```
axis=1).duplicated().sum()

print('Дубликаты в train без Churn:', duplicates_without_target)

train[train.drop('Churn',
axis=1).duplicated(keep=False)].sort_values(by=[*train.columns]).head(
6)
```

Дубликаты в train без Churn: 22

	ClientPeriod	MonthlySpending	TotalSpent	Sex
IsSeniorCitizen \				
838	1	19.65	19.65	Female
0				
1636	1	19.65	19.65	Female
0				
4811	1	19.65	19.65	Female
0				
3463	1	19.75	19.75	Male
0				
4294	1	19.75	19.75	Male
0				
3831	1	20.00	20	Male
0				

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
838	No	No	Yes		No
1636	No	No	Yes		No
4811	No	No	Yes		No
3463	No	No	Yes		No
4294	No	No	Yes		No
3831	No	No	Yes		No

	HasInternetService	HasOnlineSecurityService	HasOnlineBackup
\			
838	No	No internet service	No internet service
1636	No	No internet service	No internet service
4811	No	No internet service	No internet service
3463	No	No internet service	No internet service
4294	No	No internet service	No internet service
3831	No	No internet service	No internet service

	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV	\
838	No internet service	No internet service	No internet service	
1636	No internet service	No internet service	No internet service	

4811	No internet service	No internet service	No internet service
3463	No internet service	No internet service	No internet service
4294	No internet service	No internet service	No internet service
3831	No internet service	No internet service	No internet service

	HasMovieSubscription	HasContractPhone	IsBillingPaperless		
PaymentMethod \					
838	No internet service	Month-to-month	No	Mailed	check
1636	No internet service	Month-to-month	No	Mailed	check
4811	No internet service	Month-to-month	No	Mailed	check
3463	No internet service	Month-to-month	No	Mailed	check
4294	No internet service	Month-to-month	No	Mailed	check
3831	No internet service	Month-to-month	No	Mailed	check

	Churn
838	0
1636	0
4811	1
3463	0
4294	1
3831	0

```
# Удалим дубликаты
# Кроме теста, иначе сабмишн не сойдется по количеству строк
```

```
train = train.drop_duplicates(keep='last')
# test = test.drop_duplicates(keep='last')

print('Дубликаты в train:', train.duplicated().sum())
# print('Дубликаты в test:', test.duplicated().sum())
```

Дубликаты в train: 0

```
# Удалим дубликаты, найденные без учёта колонки Churn
```

```
# Оставляем последнюю запись для каждого дубликата
tmp_mask = train.drop('Churn', axis=1).duplicated(keep='last')
duplicate_idx = train[tmp_mask].index # индексы
```

```
# Удаляем дубликаты
train = train.drop(index=duplicate_idx).reset_index(drop=True)
```

```
duplicates_without_target = train.drop('Churn',
```

```
axis=1).duplicated().sum()  
print('Дубликаты в train без Churn:', duplicates_without_target)
```

Дубликаты в train без Churn: 0

Выявление и обработка пропусков

Проверим на наличие пропусков

```
train.isna().sum()
```

ClientPeriod	0
MonthlySpending	0
TotalSpent	0
Sex	0
IsSeniorCitizen	0
HasPartner	0
HasChild	0
HasPhoneService	0
HasMultiplePhoneNumbers	0
HasInternetService	0
HasOnlineSecurityService	0
HasOnlineBackup	0
HasDeviceProtection	0
HasTechSupportAccess	0
HasOnlineTV	0
HasMovieSubscription	0
HasContractPhone	0
IsBillingPaperless	0
PaymentMethod	0
Churn	0
dtype:	int64

*# Проверим на наличие символов и значений, означающих пропуски,
а также на наличие нетипичных для соответствующих признаков значений*

```
print(train.nunique())
```

ClientPeriod	73
MonthlySpending	1468
TotalSpent	4978
Sex	2
IsSeniorCitizen	2
HasPartner	2
HasChild	2
HasPhoneService	2
HasMultiplePhoneNumbers	3
HasInternetService	3
HasOnlineSecurityService	3
HasOnlineBackup	3

```

HasDeviceProtection      3
HasTechSupportAccess     3
HasOnlineTV              3
HasMovieSubscription     3
HasContractPhone         3
IsBillingPaperless       2
PaymentMethod            4
Churn                    2
dtype: int64

```

```

def check_col_values(df):
    for col in df.columns.drop(['MonthlySpending', 'TotalSpent']):
        print(col.ljust(24), df[col].unique())

    print()

    try:
        print(df['MonthlySpending'].mean())
    except: print('Строковые значения в MonthlySpending')

    try:
        print(df['TotalSpent'].mean())
    except: print('Строковые значения в TotalSpent')

```

```
check_col_values(train)
```

```

ClientPeriod      [55 72  1 32 60 25 27 50 16 70 69 26 38 30 52
71 59 61  7 20 48 22  2 21
 44 66 58 46 45  5  4  3 62  6 39 68 51 40 10 43 12 35 31 67  8 11 29
37
 33 14 15 47 23 65 13 41 53  9 64 42 34 17 18 63 56 54 24 28 49 19 57
36
  0]
Sex               <ArrowStringArray>
['Male', 'Female']
Length: 2, dtype: str
IsSeniorCitizen   [0 1]
HasPartner        <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasChild          <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasPhoneService   <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasMultiplePhoneNumbers <ArrowStringArray>
['No', 'Yes', 'No phone service']
Length: 3, dtype: str
HasInternetService <ArrowStringArray>

```

```

['No', 'Fiber optic', 'DSL']
Length: 3, dtype: str
HasOnlineSecurityService <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasOnlineBackup <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasDeviceProtection <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasTechSupportAccess <ArrowStringArray>
['No internet service', 'Yes', 'No']
Length: 3, dtype: str
HasOnlineTV <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasMovieSubscription <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasContractPhone <ArrowStringArray>
['One year', 'Two year', 'Month-to-month']
Length: 3, dtype: str
IsBillingPaperless <ArrowStringArray>
['No', 'Yes']
Length: 2, dtype: str
PaymentMethod <ArrowStringArray>
[ 'Mailed check', 'Credit card (automatic)',
  'Electronic check', 'Bank transfer (automatic)']
Length: 4, dtype: str
Churn [0 1]

```

65.04087452471482

Строковые значения в TotalSpent

check_col_values(test)

```

ClientPeriod [42 29 9 67 71 20 41 72 23 7 66 12 35 14 69
10 68 40 1 25 36 31 5 17
24 6 2 43 51 13 44 55 64 26 27 53 61 33 39 57 38 48 52 8 4 50 63
45
46 18 49 37 30 70 19 11 62 3 0 32 56 58 34 47 28 16 22 59 65 60 15
54
21]
Sex <ArrowStringArray>
['Male', 'Female']
Length: 2, dtype: str
IsSeniorCitizen [0 1]
HasPartner <ArrowStringArray>
['Yes', 'No']

```

```

Length: 2, dtype: str
HasChild <ArrowStringArray>
['No', 'Yes']
Length: 2, dtype: str
HasPhoneService <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasMultiplePhoneNumbers <ArrowStringArray>
['Yes', 'No', 'No phone service']
Length: 3, dtype: str
HasInternetService <ArrowStringArray>
['DSL', 'Fiber optic', 'No']
Length: 3, dtype: str
HasOnlineSecurityService <ArrowStringArray>
['Yes', 'No', 'No internet service']
Length: 3, dtype: str
HasOnlineBackup <ArrowStringArray>
['No', 'Yes', 'No internet service']
Length: 3, dtype: str
HasDeviceProtection <ArrowStringArray>
['No', 'Yes', 'No internet service']
Length: 3, dtype: str
HasTechSupportAccess <ArrowStringArray>
['No', 'Yes', 'No internet service']
Length: 3, dtype: str
HasOnlineTV <ArrowStringArray>
['No', 'Yes', 'No internet service']
Length: 3, dtype: str
HasMovieSubscription <ArrowStringArray>
['No', 'Yes', 'No internet service']
Length: 3, dtype: str
HasContractPhone <ArrowStringArray>
['One year', 'Month-to-month', 'Two year']
Length: 3, dtype: str
IsBillingPaperless <ArrowStringArray>
['No', 'Yes']
Length: 2, dtype: str
PaymentMethod <ArrowStringArray>
[ 'Credit card (automatic)', 'Electronic check',
  'Bank transfer (automatic)', 'Mailed check']
Length: 4, dtype: str

64.27260079500284
Строковые значения в TotalSpent

```

В обоих датасетах (train и test) в колонке TotalSpent есть строковые значения (скорее всего обозначающие пропуски).

```
# Посмотрим внимательнее на значения в TotalSpent
```

```
print(np.sort(train['TotalSpent'].unique()))  
total_spent_nan = np.sort(train['TotalSpent'].unique())[0]
```

```
[' ' '100.2' '100.25' ... '997.65' '998.1' '999.9']
```

```
train[train['TotalSpent'] == total_spent_nan]
```

	ClientPeriod	MonthlySpending	TotalSpent	Sex
IsSeniorCitizen \				
1040	0	25.75		Male
0				
1693	0	73.35		Female
0				
2527	0	19.70		Male
0				
3060	0	80.85		Female
0				
3678	0	20.00		Female
0				
3981	0	61.90		Male
0				
4305	0	25.35		Male
0				
4529	0	52.55		Female
0				
4576	0	56.05		Female
0				

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
1040	Yes	Yes	Yes		Yes
1693	Yes	Yes	Yes		Yes
2527	Yes	Yes	Yes		No
3060	Yes	Yes	Yes		No
3678	Yes	Yes	Yes		No
3981	No	Yes	Yes		Yes
4305	Yes	Yes	Yes		Yes
4529	Yes	Yes	No	No phone service	
4576	Yes	Yes	No	No phone service	

	HasInternetService	HasOnlineSecurityService	HasOnlineBackup
\			
1040	No	No internet service	No internet service
1693	DSL	No	Yes
2527	No	No internet service	No internet service
3060	DSL	Yes	Yes

3678	No	No internet service	No internet service
3981	DSL	Yes	Yes
4305	No	No internet service	No internet service
4529	DSL	Yes	No
4576	DSL	Yes	Yes

	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV	\
1040	No internet service	No internet service	No internet service	
1693	Yes	Yes	Yes	
2527	No internet service	No internet service	No internet service	
3060	Yes	No	Yes	
3678	No internet service	No internet service	No internet service	
3981	No	Yes	No	
4305	No internet service	No internet service	No internet service	
4529	Yes	Yes	Yes	
4576	Yes	Yes	Yes	

	HasMovieSubscription	HasContractPhone	IsBillingPaperless	\
1040	No internet service	Two year	No	
1693	No	Two year	No	
2527	No internet service	One year	Yes	
3060	Yes	Two year	No	
3678	No internet service	Two year	No	
3981	No	Two year	Yes	
4305	No internet service	Two year	No	
4529	No	Two year	Yes	
4576	No	Two year	No	

	PaymentMethod	Churn
1040	Mailed check	0
1693	Mailed check	0
2527	Mailed check	0
3060	Mailed check	0
3678	Mailed check	0
3981	Bank transfer (automatic)	0
4305	Mailed check	0
4529	Bank transfer (automatic)	0
4576	Credit card (automatic)	0

Заменяем отсутствующее значение ' ' в TotalSpent на значение 0.0 типа float

```
def prepare_total_spent(df):
    df.loc[df['TotalSpent'] == ' ', 'TotalSpent'] = '0'
    df['TotalSpent'] = df['TotalSpent'].astype(float)
```

```
prepare_total_spent(train)
prepare_total_spent(test)
```

Проверим заполнение

```
train.loc[[1048, 1707, 4551], :]
```

	ClientPeriod	MonthlySpending	TotalSpent	Sex	IsSeniorCitizen
1048	63	83.5	5435.0	Male	0
1707	13	83.2	1060.6	Male	0
4551	29	84.9	2516.2	Male	0

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	
1048	Yes	Yes	Yes		Yes
1707	No	No	Yes		No
4551	No	No	Yes		Yes

	HasInternetService	HasOnlineSecurityService	HasOnlineBackup	
1048	DSL		Yes	Yes
1707	Fiber optic		No	No
4551	Fiber optic		No	No

	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV	
1048	Yes	No	Yes	
1707	Yes	No	Yes	
4551	No	No	No	

	HasMovieSubscription	HasContractPhone	IsBillingPaperless	
1048	Yes	Two year		No
1707	No	Month-to-month		Yes
4551	Yes	Month-to-month		Yes

	PaymentMethod	Churn
1048	Credit card (automatic)	0
1707	Electronic check	1
4551	Electronic check	0

check_col_values(train)

check_col_values(test)

```
print(train.isna().sum().sum())
```

```
print(test.isna().sum().sum())
```

```
0
```

```
0
```

Для вашего удобства списки с именами разных признаков

```

# Числовые признаки

# train.select_dtypes(include='number').columns.drop('Churn')
num_cols = [
    'ClientPeriod',
    'MonthlySpending',
    'TotalSpent'
]

# Категориальные признаки
# train.select_dtypes(include='object').columns
cat_cols = [
    'Sex',
    'IsSeniorCitizen',
    'HasPartner',
    'HasChild',
    'HasPhoneService',
    'HasMultiplePhoneNumbers',
    'HasInternetService',
    'HasOnlineSecurityService',
    'HasOnlineBackup',
    'HasDeviceProtection',
    'HasTechSupportAccess',
    'HasOnlineTV',
    'HasMovieSubscription',
    'HasContractPhone',
    'IsBillingPaperless',
    'PaymentMethod'
]

features = num_cols + cat_cols
target = 'Churn'

```

Анализ данных (3 балла)

1) Для численных признаков постройте гистограмму (*plt.hist(...)*) или boxplot (*plt.boxplot(...)*). Для категориальных посчитайте количество каждого значения для каждого признака. Для каждой колонки надо сделать *data.value_counts()* и построить bar диаграммы *plt.bar(...)* или круговые диаграммы *plt.pie(...)* (хорошо, если вы сможете это сделать на одном графике с помощью *plt.subplots(...)*).

2) Посмотрите на распределение целевой переменной и скажите, являются ли классы несбалансированными.

3) (Если будет желание) Поиграйте с разными библиотеками для визуализации - *sns*, *pandas_visual_analysis*, etc.

Второй пункт очень важен, потому что существуют задачи классификации с несбалансированными классами. Например, это может значить, что в датасете намного больше примеров 0 класса. В таких случаях нужно 1) не использовать accuracy как метрику

2) использовать методы борьбы с imbalanced dataset (обычно если датасет сильно несбалансирован, т.е. класса 1 в 20 раз меньше класса 0).

```
train.info()
# train.dtypes

<class 'pandas.DataFrame'>
RangeIndex: 5260 entries, 0 to 5259
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ClientPeriod                          5260 non-null   int64
1   MonthlySpending                       5260 non-null   float64
2   TotalSpent                            5260 non-null   float64
3   Sex                                    5260 non-null   str
4   IsSeniorCitizen                       5260 non-null   int64
5   HasPartner                            5260 non-null   str
6   HasChild                              5260 non-null   str
7   HasPhoneService                       5260 non-null   str
8   HasMultiplePhoneNumbers               5260 non-null   str
9   HasInternetService                   5260 non-null   str
10  HasOnlineSecurityService              5260 non-null   str
11  HasOnlineBackup                       5260 non-null   str
12  HasDeviceProtection                   5260 non-null   str
13  HasTechSupportAccess                  5260 non-null   str
14  HasOnlineTV                           5260 non-null   str
15  HasMovieSubscription                  5260 non-null   str
16  HasContractPhone                      5260 non-null   str
17  IsBillingPaperless                    5260 non-null   str
18  PaymentMethod                         5260 non-null   str
19  Churn                                 5260 non-null   int64
dtypes: float64(2), int64(3), str(15)
memory usage: 1.3 MB

# Статистики для числовых признаков

train.describe(include='number').T

# Аномальных значений не обнаружено
```

	count	mean	std	min	25%
50% \					
ClientPeriod	5260.0	32.528327	24.517317	0.00	9.0000
29.0					
MonthlySpending	5260.0	65.040875	30.153470	18.25	35.7375
70.5					
TotalSpent	5260.0	2296.898165	2269.107245	0.00	403.3000
1414.9					
IsSeniorCitizen	5260.0	0.159886	0.366535	0.00	0.0000
0.0					

Churn	5260.0	0.260646	0.439029	0.00	0.0000
0.0					

	75%	max
ClientPeriod	55.0000	72.00
MonthlySpending	90.0500	118.75
TotalSpent	3813.7125	8684.80
IsSeniorCitizen	0.0000	1.00
Churn	1.0000	1.00

Статистики для категориальных признаков

```
train.describe(include='object').T
```

	count	unique	top	freq
Sex	5260	2	Male	2644
HasPartner	5260	2	No	2683
HasChild	5260	2	No	3654
HasPhoneService	5260	2	Yes	4739
HasMultiplePhoneNumbers	5260	3	No	2489
HasInternetService	5260	3	Fiber optic	2335
HasOnlineSecurityService	5260	3	No	2603
HasOnlineBackup	5260	3	No	2323
HasDeviceProtection	5260	3	No	2308
HasTechSupportAccess	5260	3	No	2595
HasOnlineTV	5260	3	No	2090
HasMovieSubscription	5260	3	No	2072
HasContractPhone	5260	3	Month-to-month	2898
IsBillingPaperless	5260	2	Yes	3139
PaymentMethod	5260	4	Electronic check	1780

Визуализация

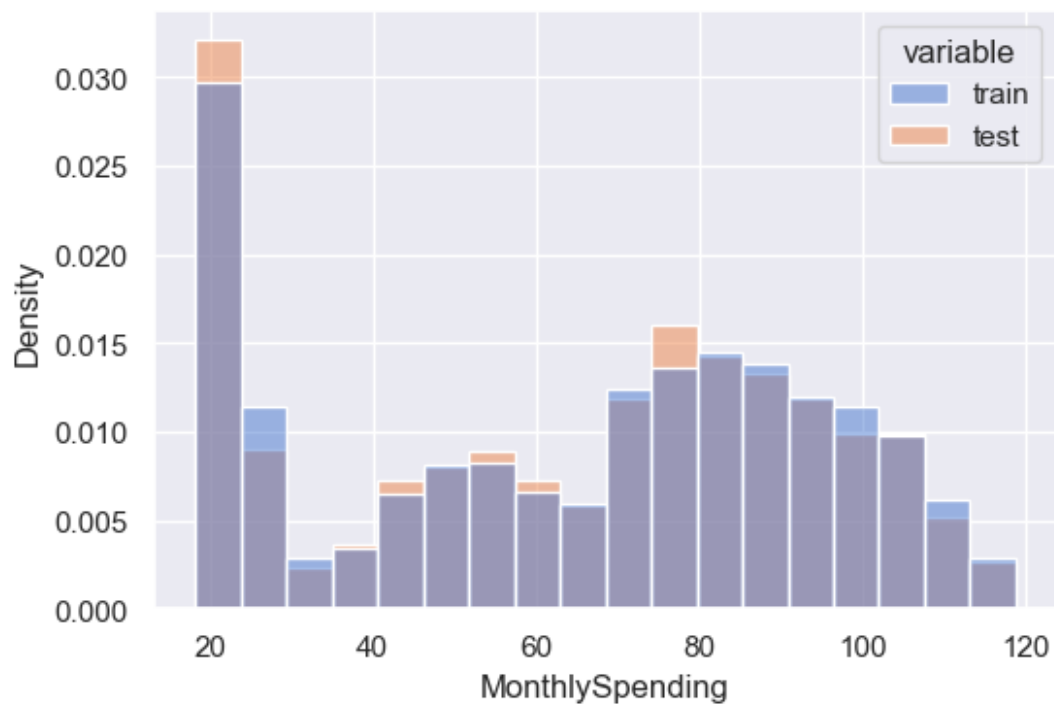
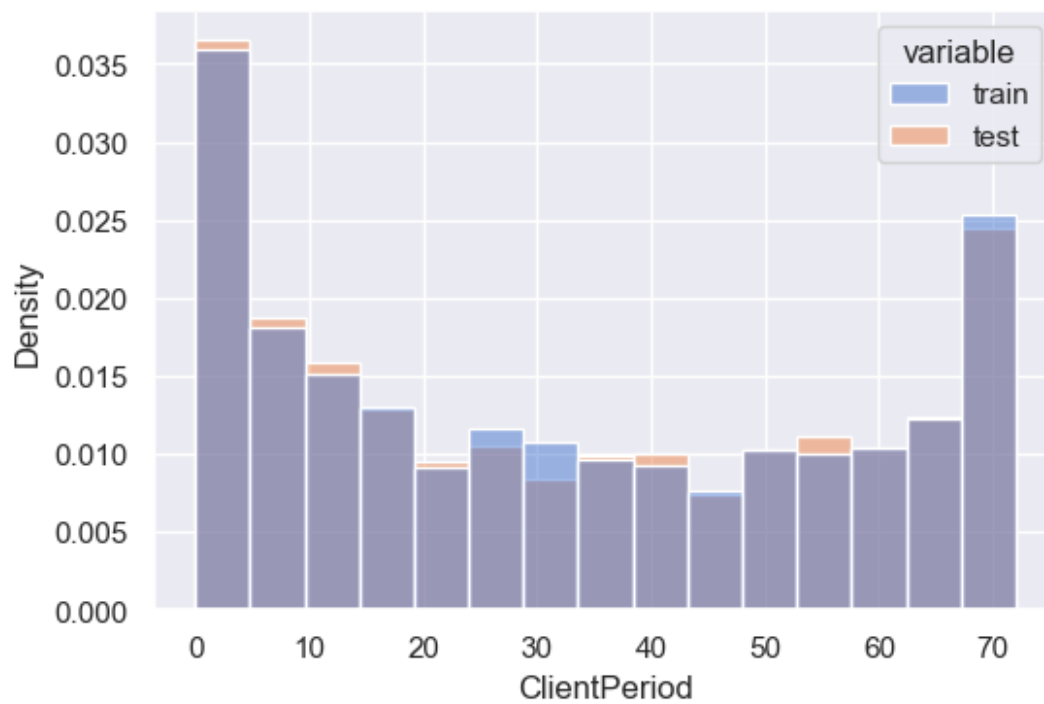
Распределение значений числовых признаков, отношение значений train и test по частоте

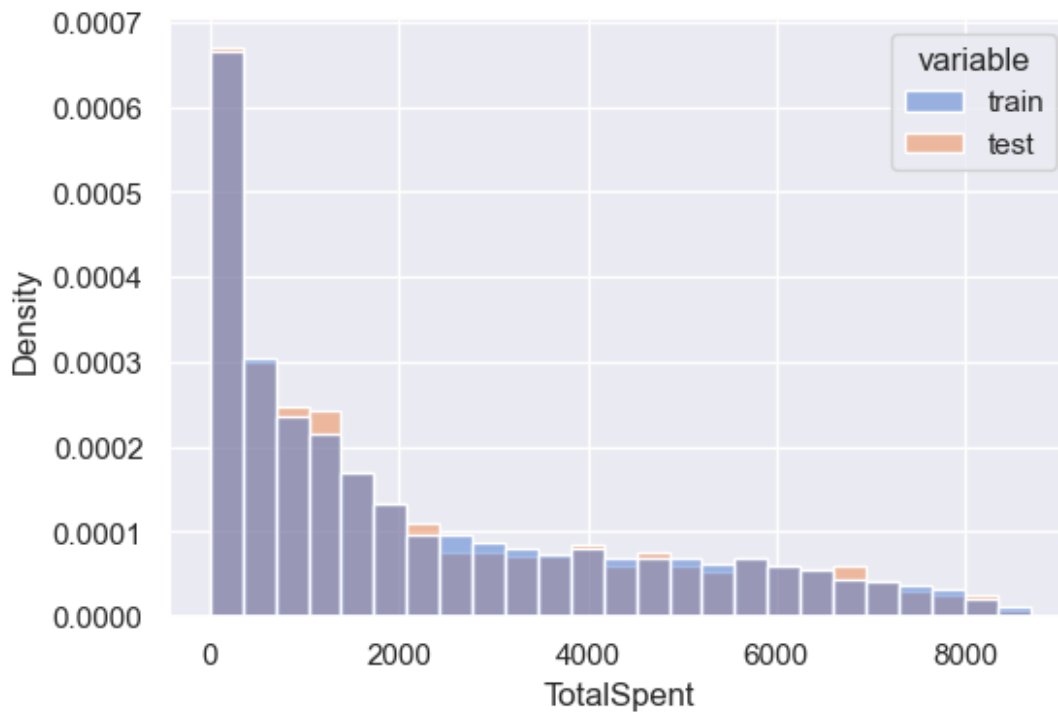
```
sns.set_theme(palette='muted')
```

```
for col in num_cols:
    plt.figure(figsize=(6,4))

    tmp = pd.DataFrame()
    tmp['train'] = train[col]
    tmp['test'] = test[col]
    tmp = tmp.melt().dropna()
    ax = sns.histplot(data=tmp, x='value', hue='variable',
stat='density', common_norm=False)
    ax.set(xlabel=col)

    plt.show();
```





Проверка на наличие выбросов

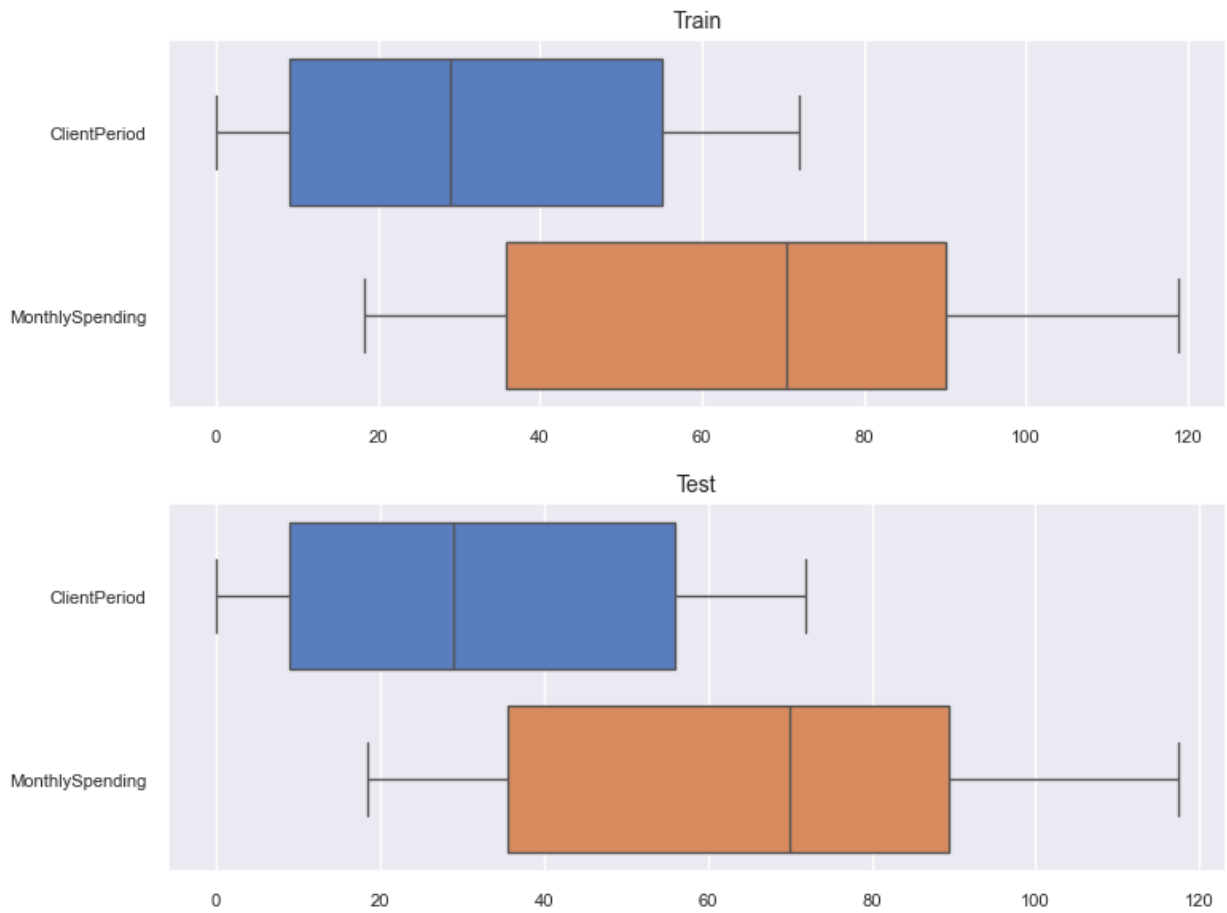
```
fig, (ax1, ax2) = plt.subplots(nrows=2, ncols=1, figsize=(8, 6))

box_cols = [col for col in num_cols if col != 'TotalSpent']
sns.boxplot(data=train[box_cols], orient='h', ax=ax1)
ax1.set_title('Train', fontsize=10)
ax1.tick_params(axis='both', labelsz=8)

sns.boxplot(data=test[box_cols], orient='h', ax=ax2)
ax2.set_title('Test', fontsize=10)
ax2.tick_params(axis='both', labelsz=8)

plt.tight_layout()

plt.show();
```



```
fig, ax = plt.subplots(figsize=(8, 5))

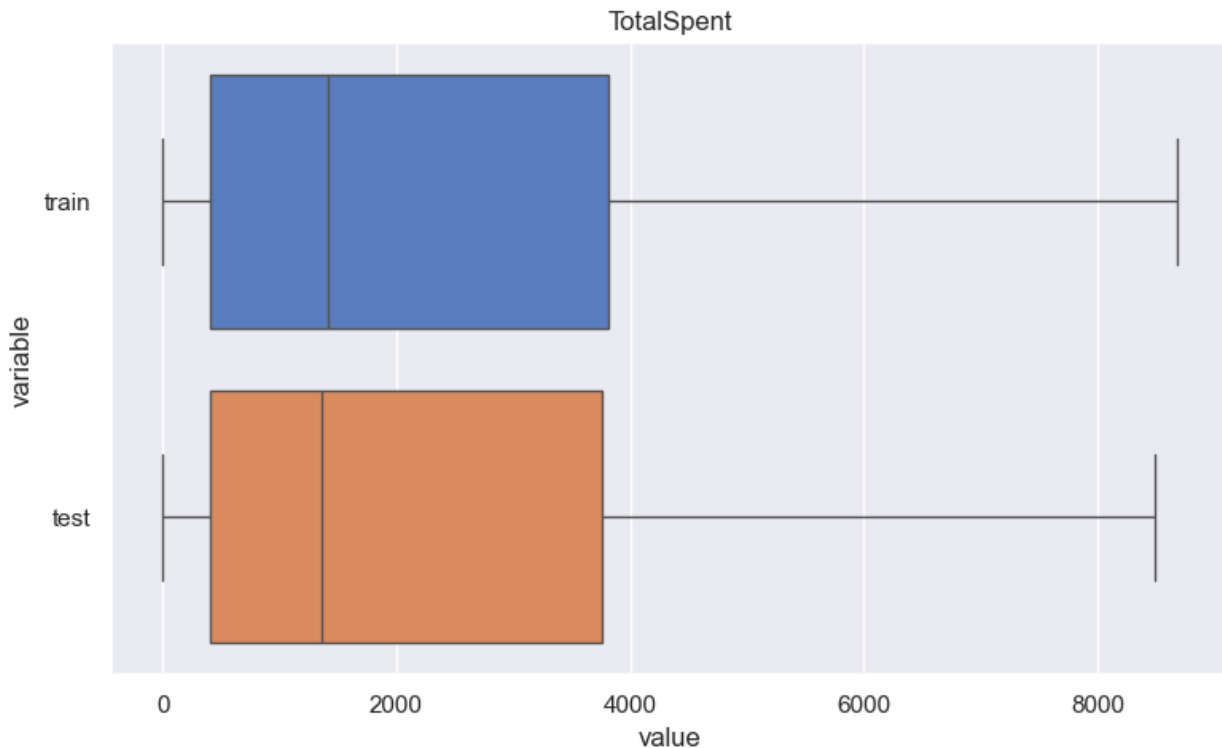
tmp = pd.DataFrame()
tmp['train'] = train['TotalSpent']
tmp['test'] = test['TotalSpent']
tmp = tmp.melt().dropna()

sns.boxplot(data=tmp, y='variable', x='value', hue='variable')

ax.set_title('TotalSpent')
ax.tick_params(axis='both')

plt.tight_layout()

plt.show();
```



Коммент

```
# Распределение значений категориальных признаков в train и test

for i, col in enumerate(cat_cols):
    fig, ax = plt.subplots(nrows=2, ncols=2, figsize=(12, 6))
    fig.suptitle(f'Распределение {col}', fontsize=10, ha='center')

    for j, df in enumerate([train, test]):
        vals = df[col].value_counts()
        colors = sns.color_palette('muted', n_colors=len(vals))

        # Pie
        wedges, texts, autotexts = ax[j, 0].pie(x=vals,
                                                labels=vals.index,
                                                colors=colors,
                                                autopct=lambda n, vals=vals:
f'{n:.2f}% ({n * vals.sum() / 100:.0f})',
                                                textprops={'fontsize': 7},
                                                # labeldistance=1.2,
                                                pctdistance=0.6,
                                                rotatelabels=False)

        for text in texts:
            text.set_fontsize(7)
            # text.set_horizontalalignment('center')
```

```

for autotext in autotexts:
    autotext.set_fontsize(7)
    autotext.set_horizontalalignment('center')
    autotext.set_verticalalignment('center')

    ax[j, 0].set_title(f"{'Train' if j==0 else 'Test'} - {col}",
fontsize=8)

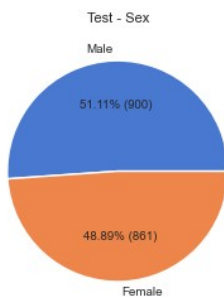
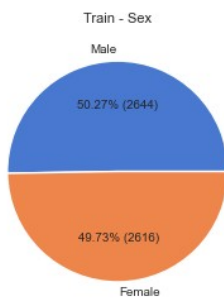
    # Bar
    ax[j, 1].bar(x=vals.index,
                height=vals.values,
                color=colors)

    ax[j, 1].set_title(f"{'Train' if j==0 else 'Test'} - {col}",
fontsize=8)
    ax[j, 1].tick_params(axis='x', rotation=0, labels=7)
    ax[j, 1].tick_params(axis='y', labels=7)

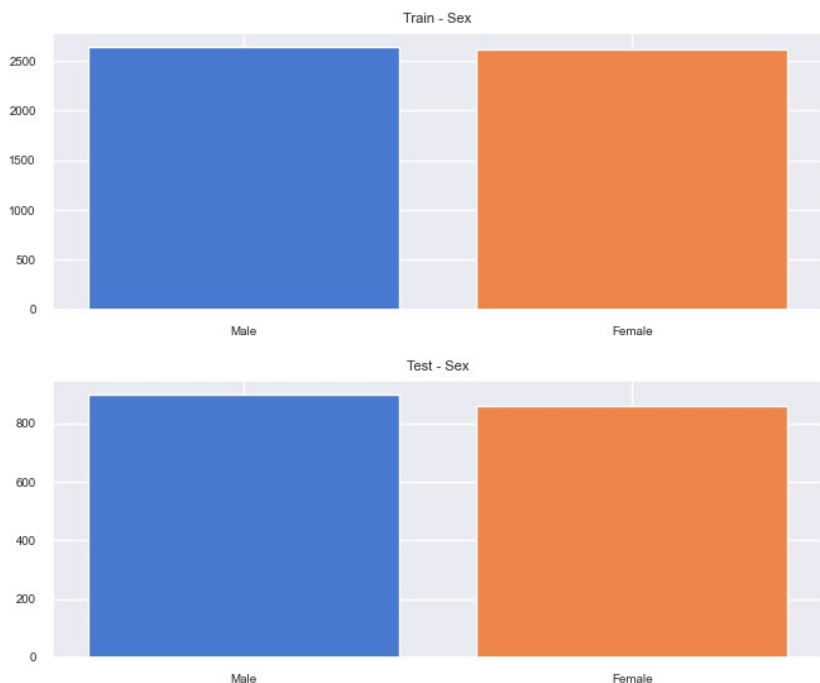
plt.tight_layout()

plt.show();

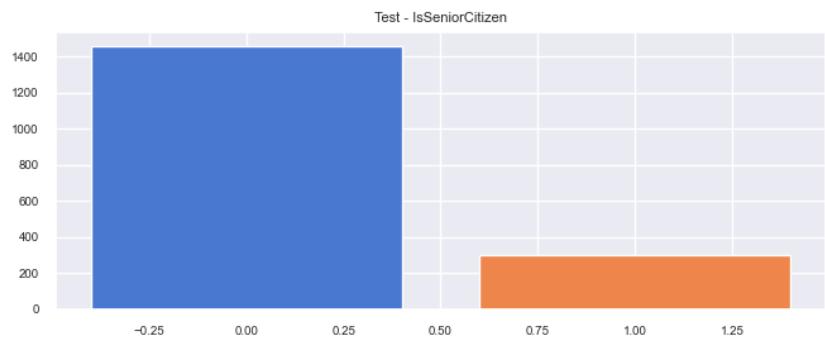
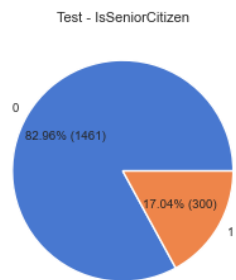
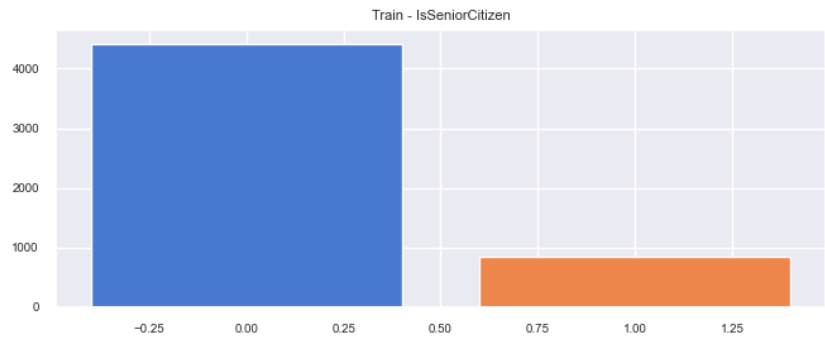
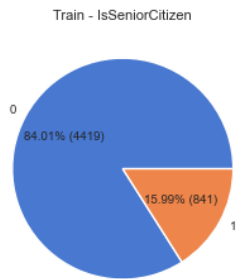
```



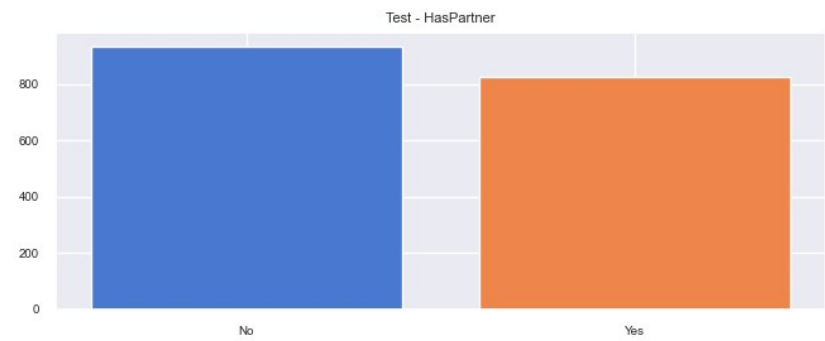
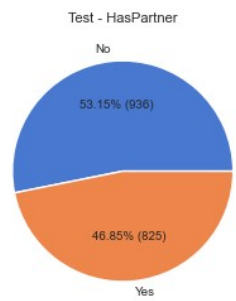
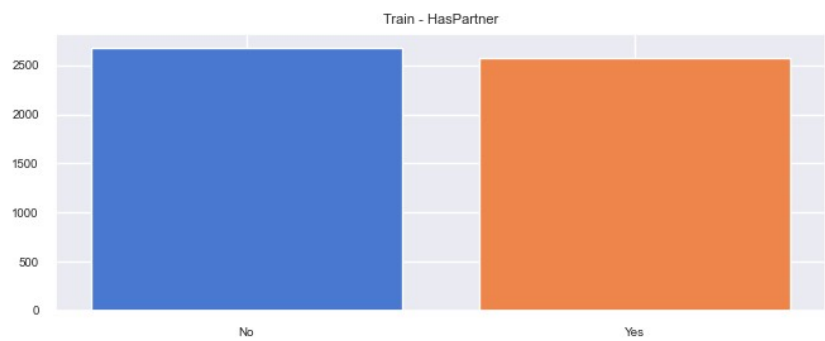
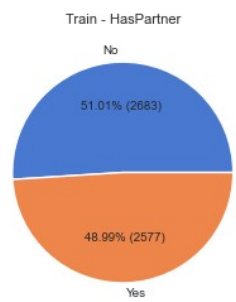
Распределение Sex



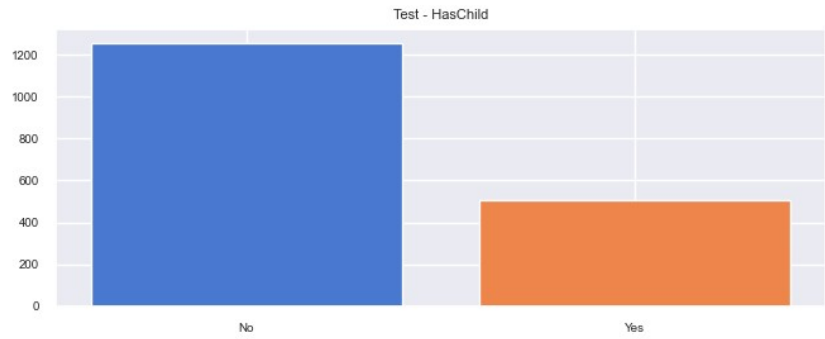
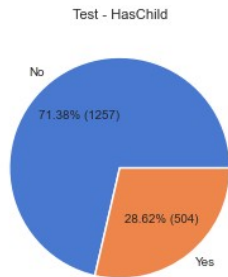
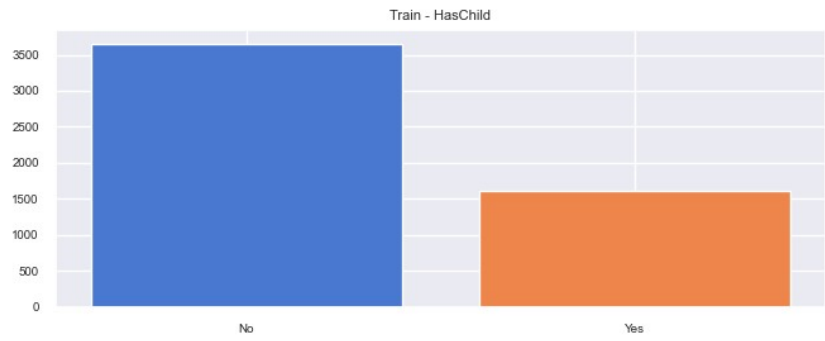
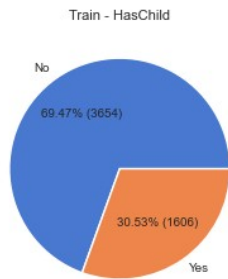
Распределение IsSeniorCitizen



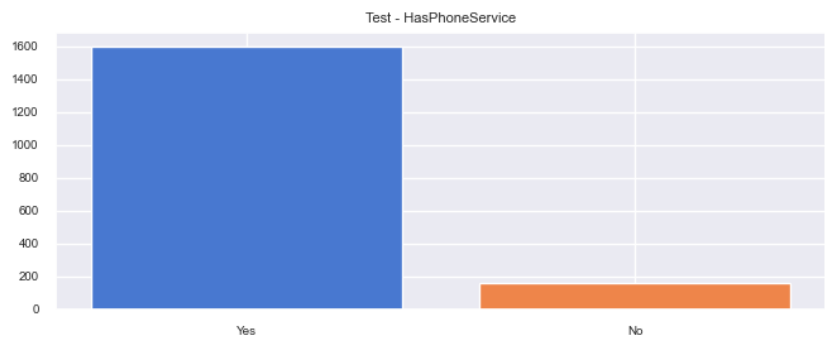
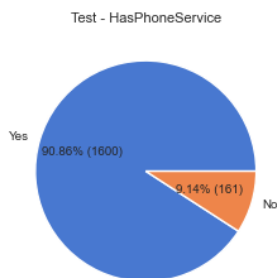
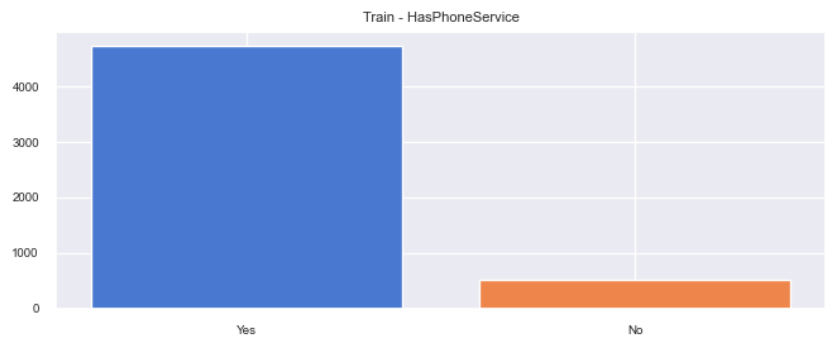
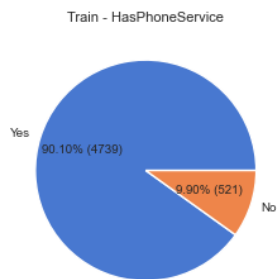
Распределение HasPartner



Распределение HasChild

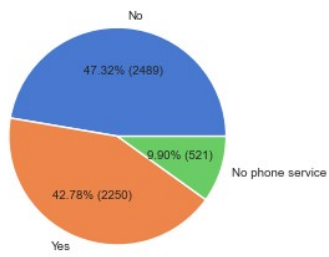


Распределение HasPhoneService

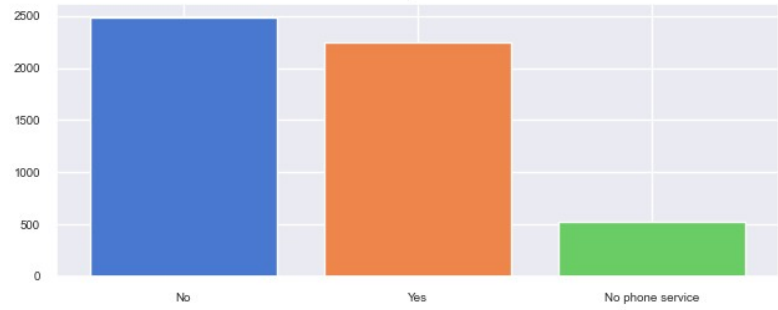


Распределение HasMultiplePhoneNumbers

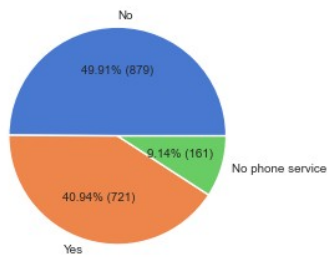
Train - HasMultiplePhoneNumbers



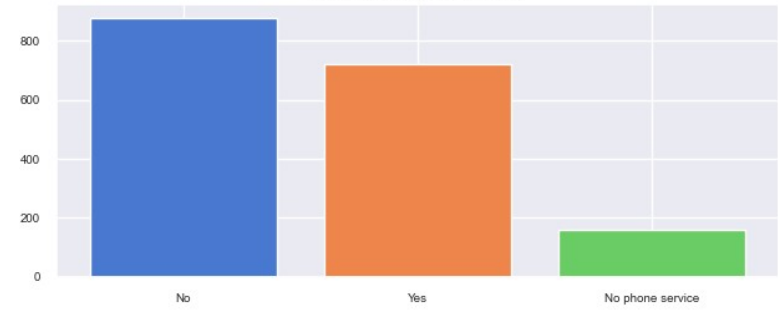
Train - HasMultiplePhoneNumbers



Test - HasMultiplePhoneNumbers

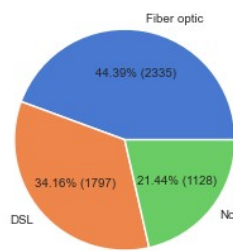


Test - HasMultiplePhoneNumbers

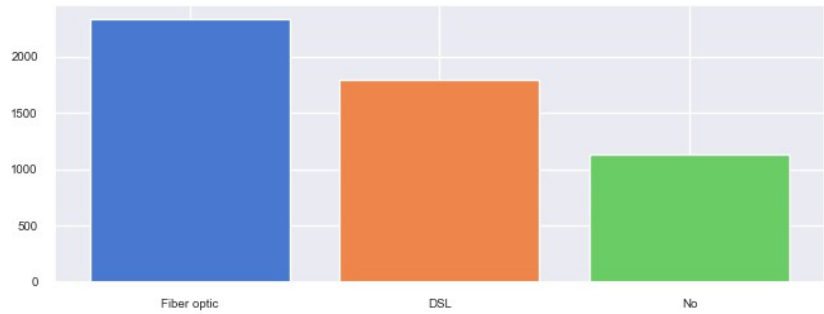


Распределение HasInternetService

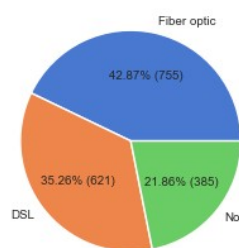
Train - HasInternetService



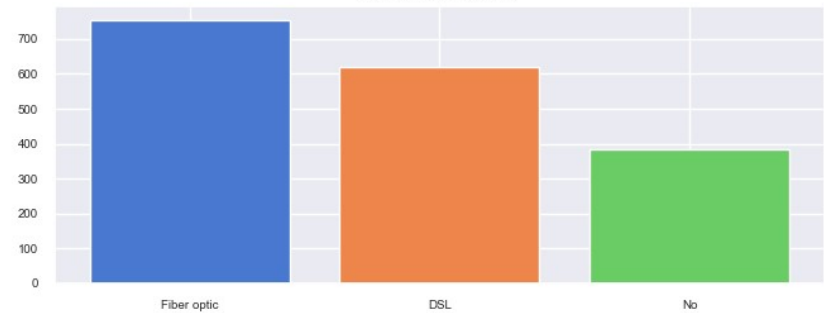
Train - HasInternetService



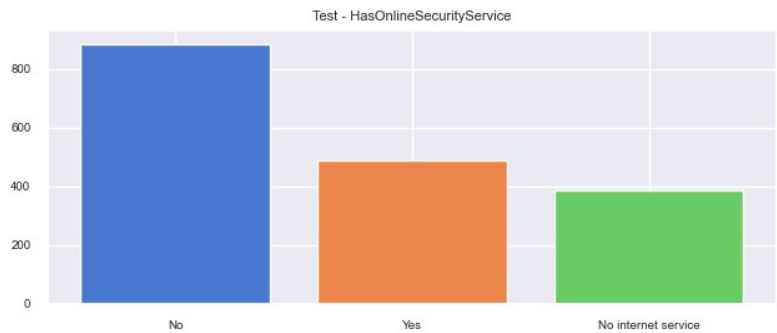
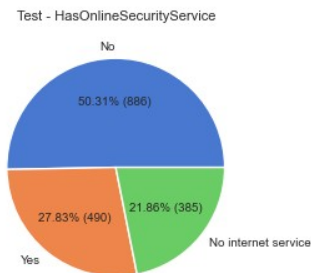
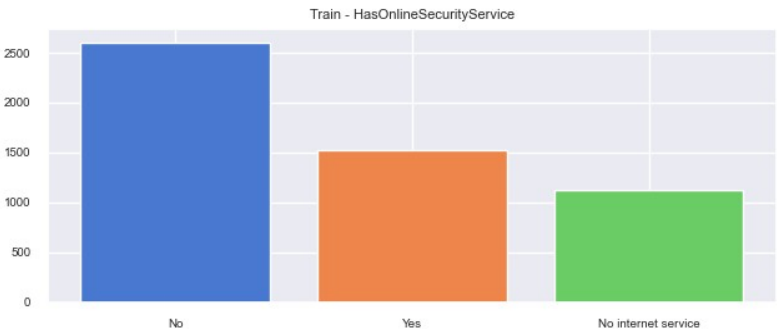
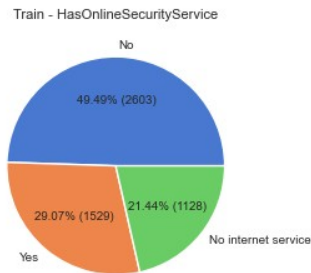
Test - HasInternetService



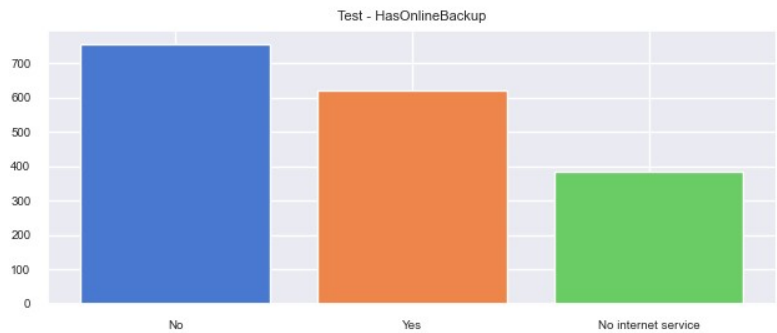
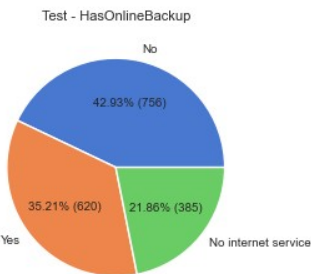
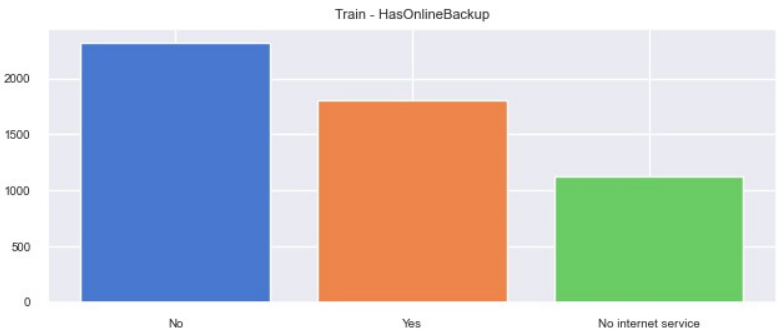
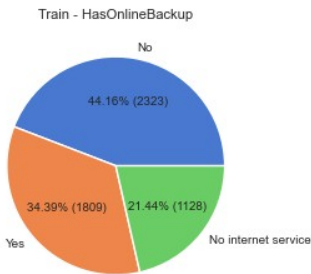
Test - HasInternetService



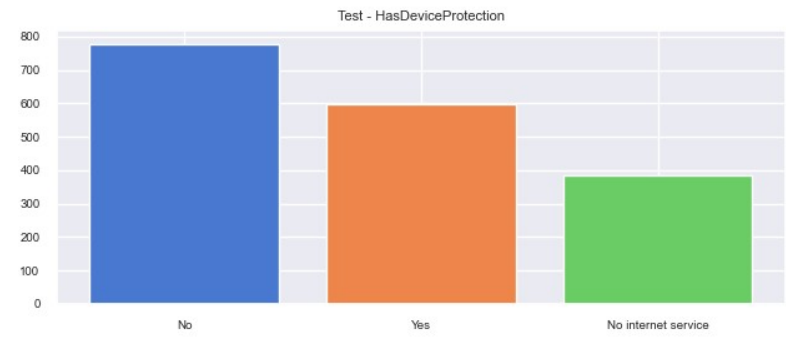
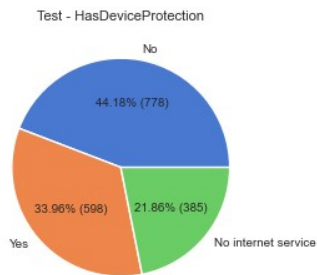
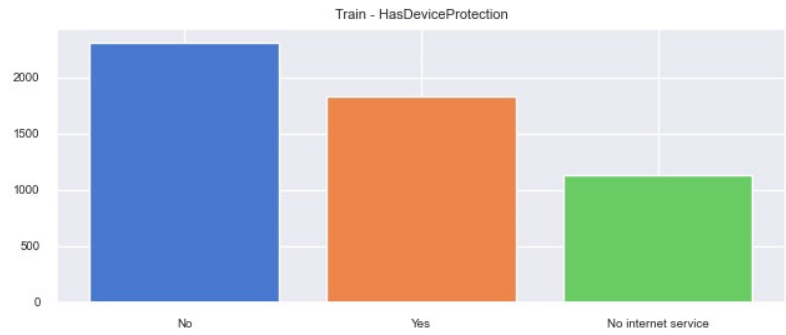
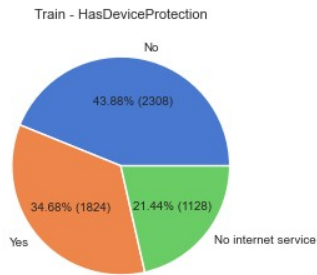
Распределение HasOnlineSecurityService



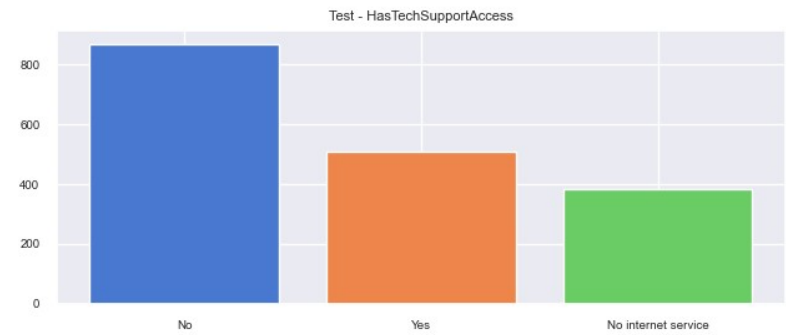
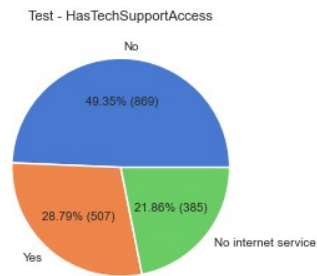
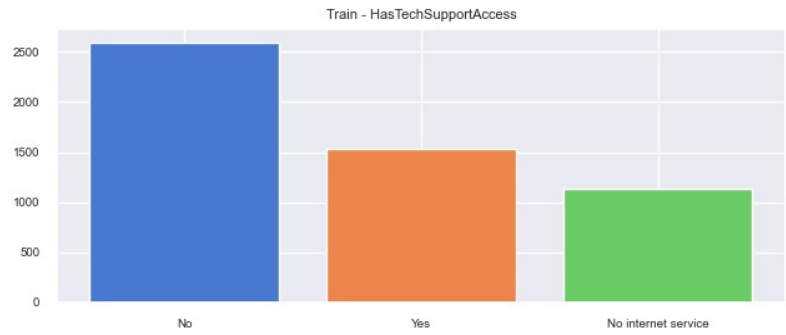
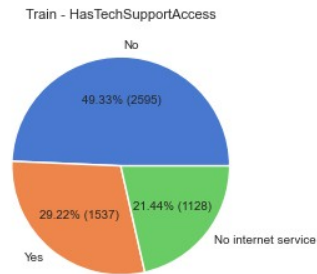
Распределение HasOnlineBackup



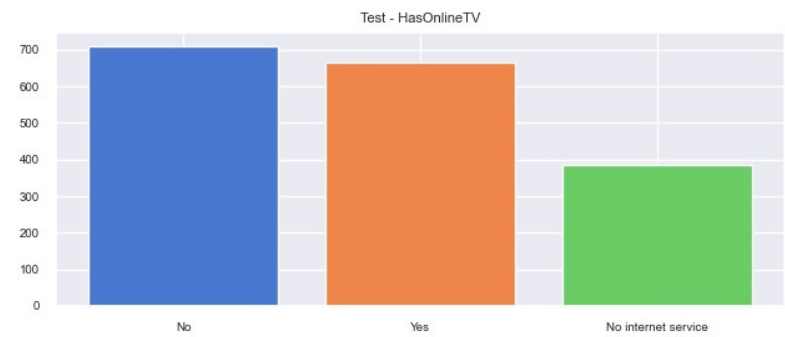
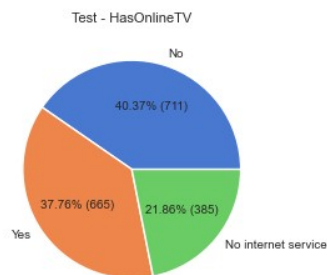
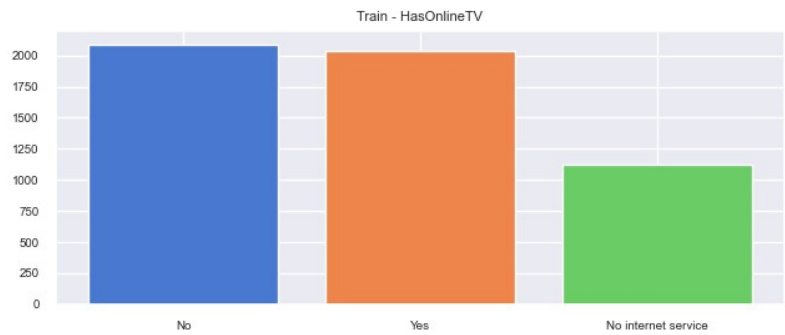
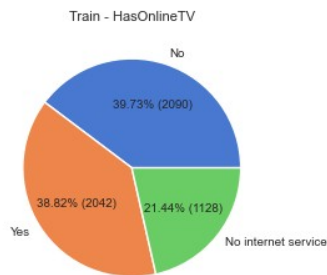
Распределение HasDeviceProtection



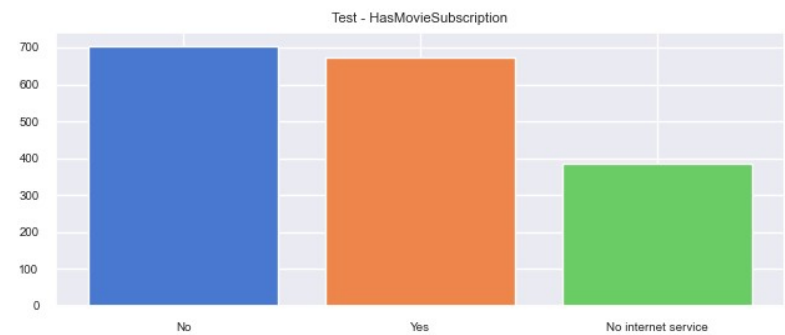
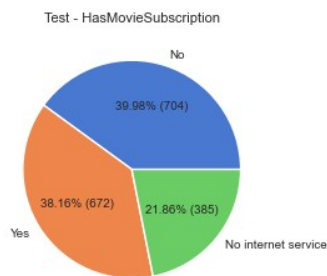
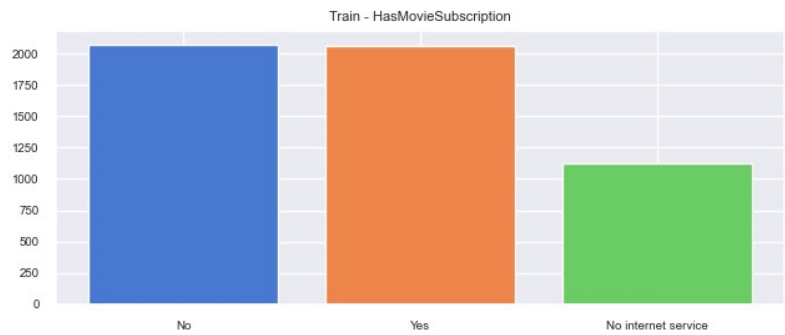
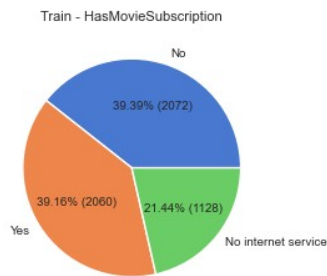
Распределение HasTechSupportAccess



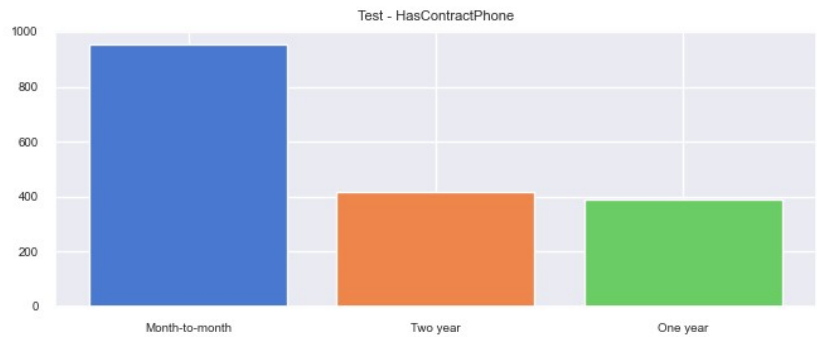
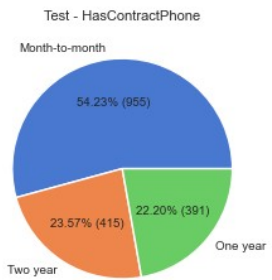
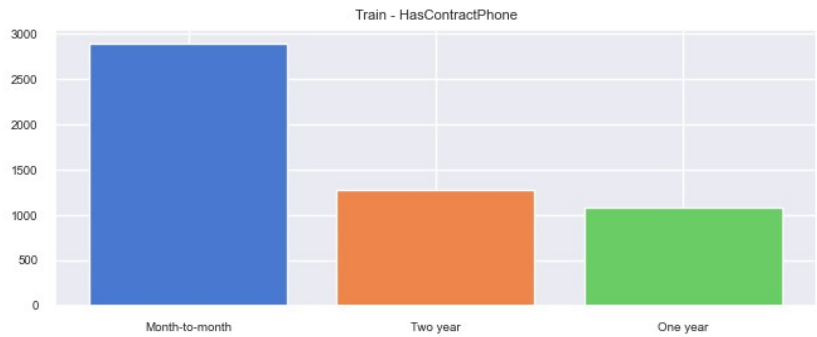
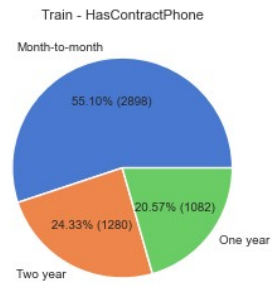
Распределение HasOnlineTV



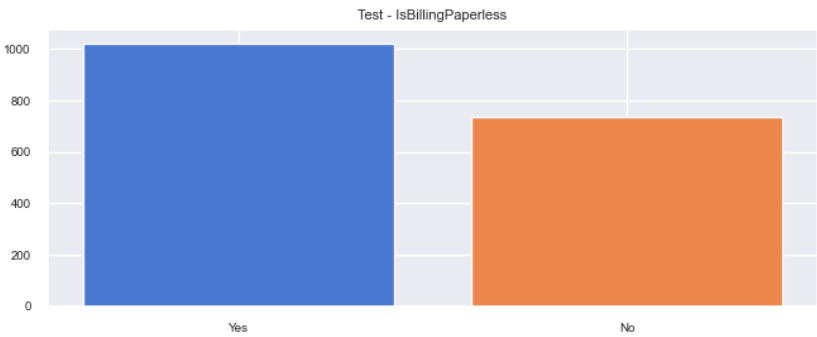
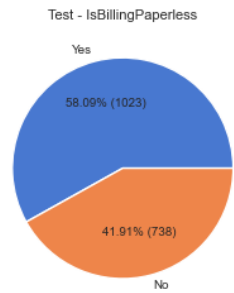
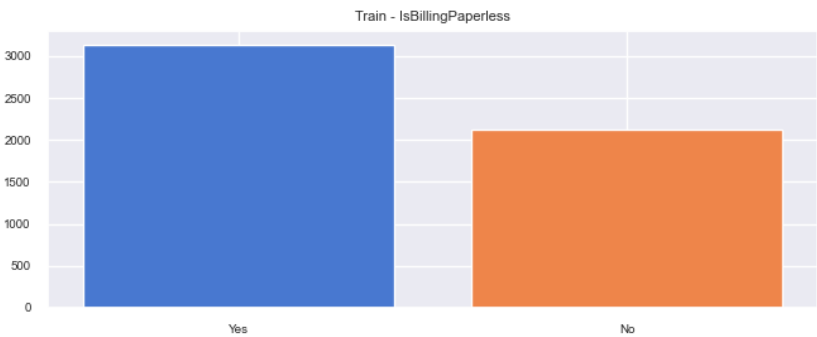
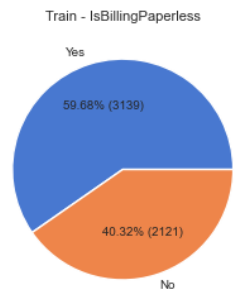
Распределение HasMovieSubscription



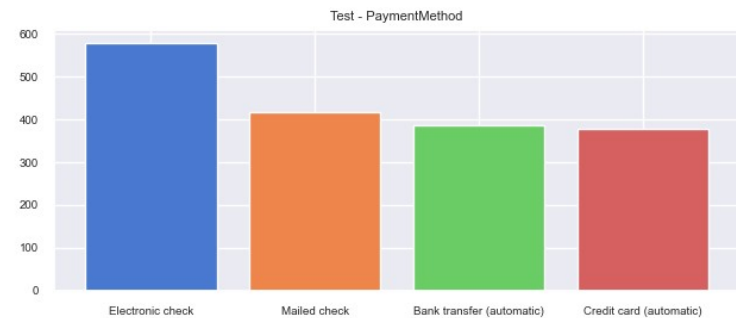
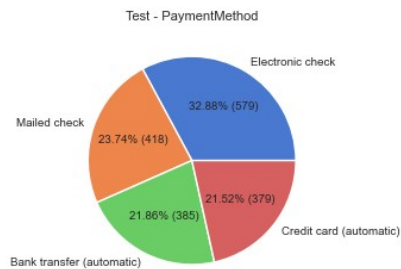
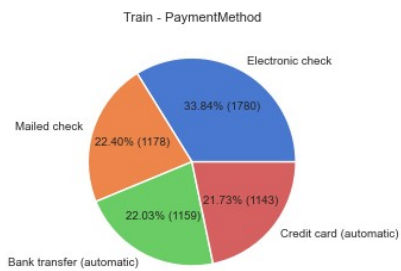
Распределение HasContractPhone



Распределение IsBillingPaperless



Распределение PaymentMethod



Распределения признаков в тренировочной и тестовой выборках практически идентичны.

Распределение значений по каждому категориальному признаку (по требованию в задании)

```
for col in cat_cols:
    val_counts = train[col].value_counts()
    val_proportion = val_counts / val_counts.sum()

    tmp = pd.DataFrame({
        col: val_counts.index,
        'count': val_counts.values,
        'proportion': val_proportion.values
    })

    display(tmp)
```

	Sex	count	proportion
0	Male	2644	0.502662
1	Female	2616	0.497338

	IsSeniorCitizen	count	proportion
0	0	4419	0.840114
1	1	841	0.159886

	HasPartner	count	proportion
0	No	2683	0.510076
1	Yes	2577	0.489924

	HasChild	count	proportion
0	No	3654	0.694677
1	Yes	1606	0.305323

	HasPhoneService	count	proportion
0	Yes	4739	0.900951
1	No	521	0.099049

	HasMultiplePhoneNumbers	count	proportion
0	No	2489	0.473194
1	Yes	2250	0.427757
2	No phone service	521	0.099049

	HasInternetService	count	proportion
0	Fiber optic	2335	0.443916
1	DSL	1797	0.341635
2	No	1128	0.214449

	HasOnlineSecurityService	count	proportion
0	No	2603	0.494867
1	Yes	1529	0.290684
2	No internet service	1128	0.214449

	HasOnlineBackup	count	proportion
0	No	2323	0.441635
1	Yes	1809	0.343916
2	No internet service	1128	0.214449

	HasDeviceProtection	count	proportion
0	No	2308	0.438783
1	Yes	1824	0.346768
2	No internet service	1128	0.214449

	HasTechSupportAccess	count	proportion
0	No	2595	0.493346
1	Yes	1537	0.292205
2	No internet service	1128	0.214449

	HasOnlineTV	count	proportion
0	No	2090	0.397338
1	Yes	2042	0.388213
2	No internet service	1128	0.214449

	HasMovieSubscription	count	proportion
0	No	2072	0.393916
1	Yes	2060	0.391635
2	No internet service	1128	0.214449

	HasContractPhone	count	proportion
0	Month-to-month	2898	0.550951

1	Two year	1280	0.243346
2	One year	1082	0.205703

	IsBillingPaperless	count	proportion
0	Yes	3139	0.596768
1	No	2121	0.403232

	PaymentMethod	count	proportion
0	Electronic check	1780	0.338403
1	Mailed check	1178	0.223954
2	Bank transfer (automatic)	1159	0.220342
3	Credit card (automatic)	1143	0.217300

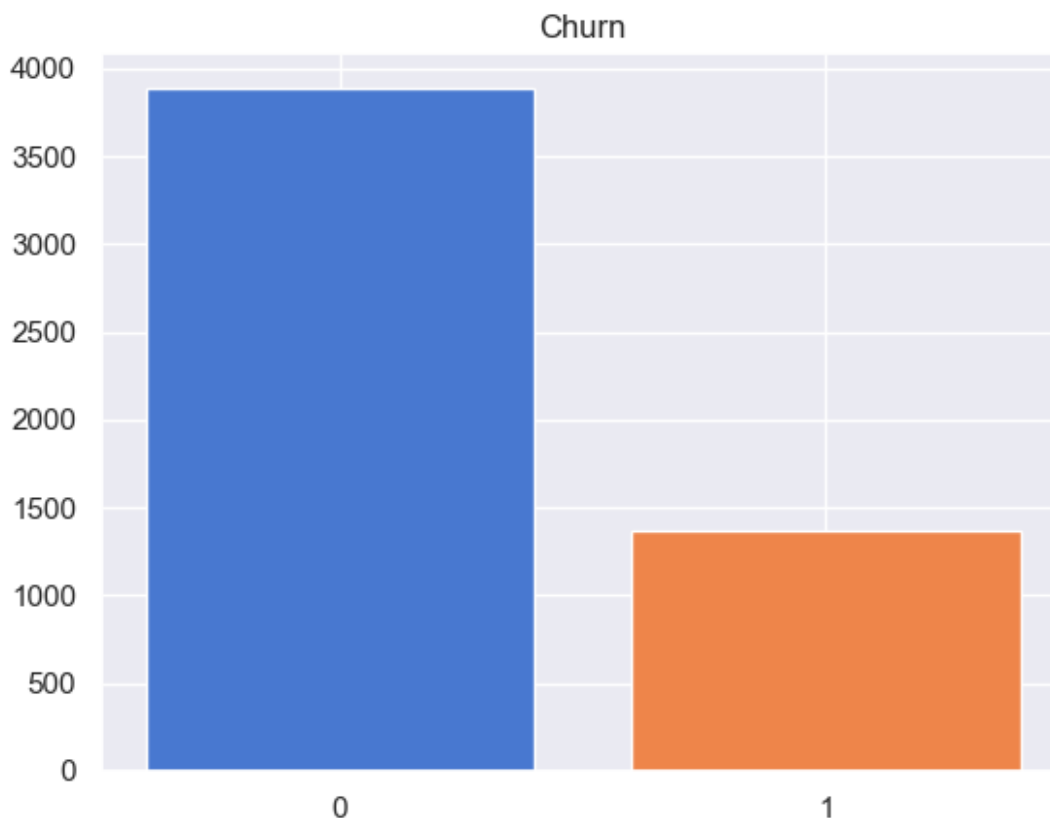
Распределение целевой переменной

```
vals = train[target].value_counts()
colors = sns.color_palette('muted', n_colors=len(vals))

plt.bar(x=vals.index, height=vals.values, color=colors)
plt.title(target)
plt.xticks(vals.index)

plt.show();

print(vals[0] / vals[1])
```



2.836615609044493

Распределение целевой переменной несбалансированно. Класс 0 встречается в ~2.83 чаще класса 1.

Далее поработаем с имеющимися признаками и попробуем создать новые.

train.dtypes

ClientPeriod	int64
MonthlySpending	float64
TotalSpent	float64
Sex	str
IsSeniorCitizen	int64
HasPartner	str
HasChild	str
HasPhoneService	str
HasMultiplePhoneNumbers	str
HasInternetService	str
HasOnlineSecurityService	str
HasOnlineBackup	str
HasDeviceProtection	str
HasTechSupportAccess	str
HasOnlineTV	str
HasMovieSubscription	str
HasContractPhone	str
IsBillingPaperless	str
PaymentMethod	str
Churn	int64
dtype:	object

(Дополнительно) Если вы нашли какие-то ошибки в данных или выбросы, то можете их убрать. Тут можно поэкспериментировать с обработкой данных как угодно, но не за баллы.

Выбросов и аномальных значений не обнаружено.

```
# print(train.columns)
# check_col_values(train)
```

Проверим некоторые признаки на предмет влияния значения признака на то, уйдет ли клиент.

```
cols = ['HasPartner', 'HasChild', 'HasPhoneService',
        'HasMultiplePhoneNumbers',
        'HasInternetService', 'HasOnlineSecurityService',
        'HasOnlineBackup',
        'HasDeviceProtection', 'HasTechSupportAccess', 'HasOnlineTV',
        'HasMovieSubscription', 'IsBillingPaperless']
```

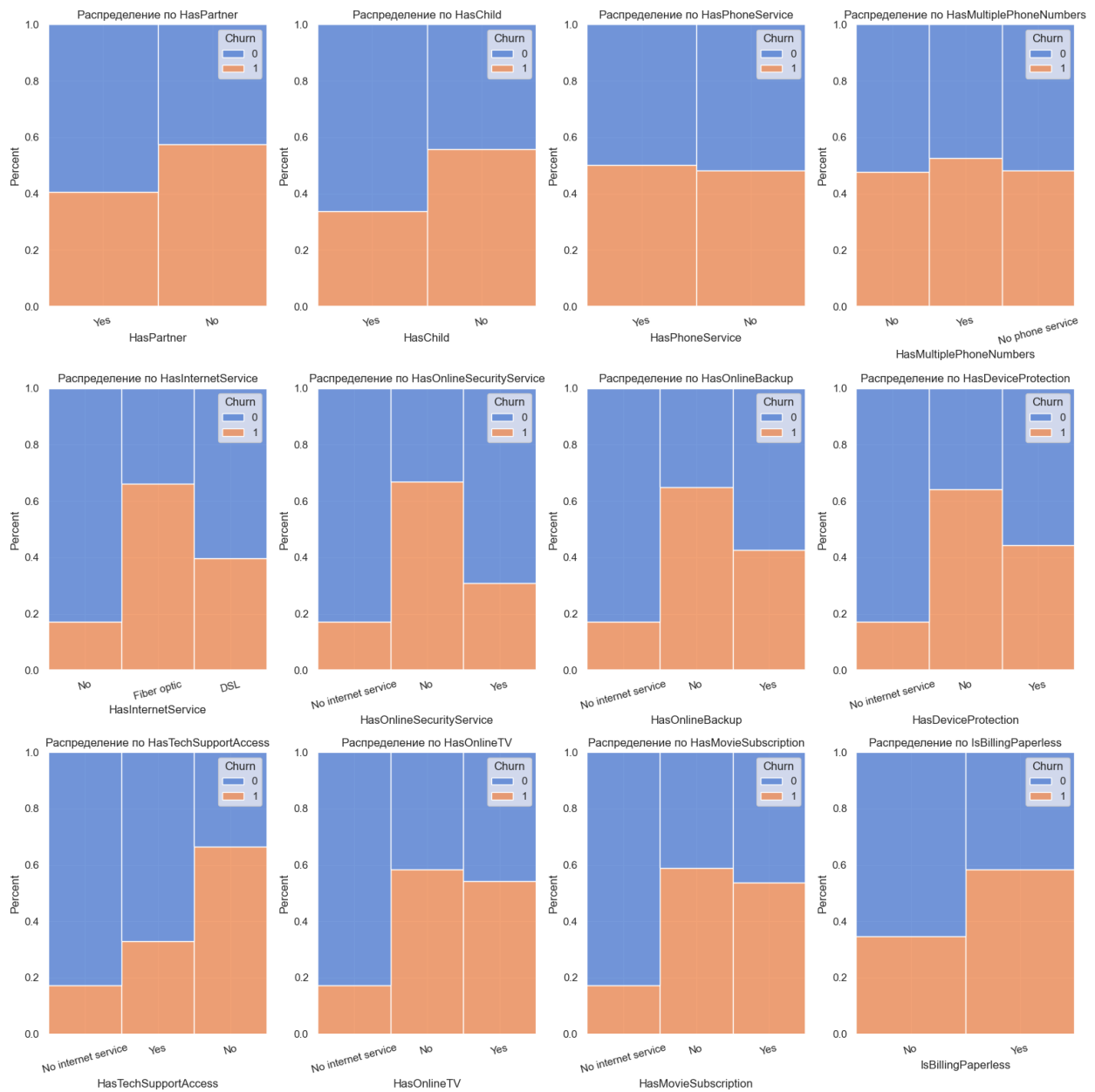
```
fig, axes = plt.subplots(nrows=(len(cols) + 3) // 4, ncols=4,
figsize=(16, 16))
plt.subplots_adjust(wspace=0.5, hspace=0.5)

for i, col in enumerate(cols):
    row_idx = i // 4
    col_idx = i % 4
    ax = axes[row_idx, col_idx]

    sns.histplot(data=train, x=col, hue='Churn',
                  multiple='fill', # заполнение до 100%
                  stat='percent',
                  common_norm=False, # нормировка
                  ax=ax)

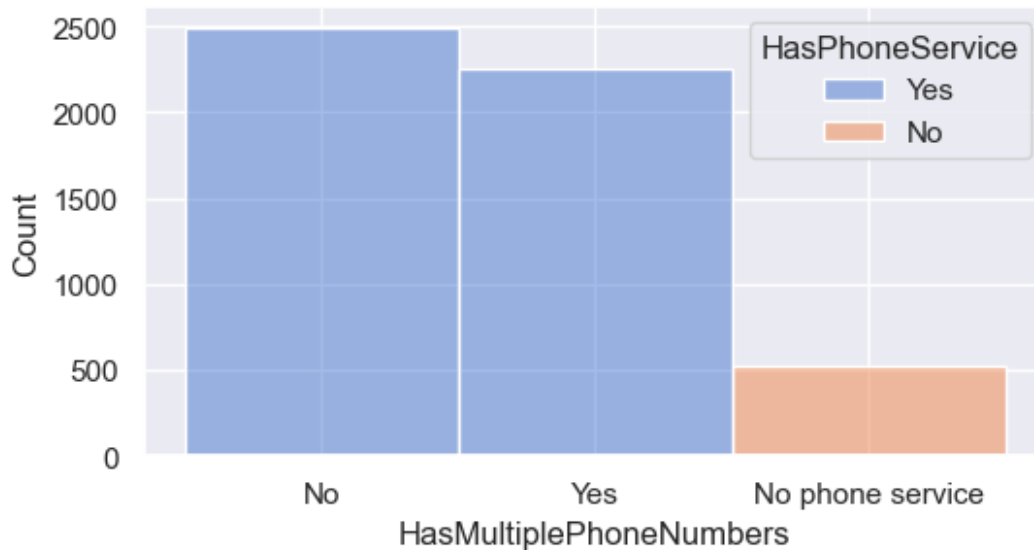
    ax.set_title(f'Распределение по {col}')
    ax.set_xticklabels(ax.get_xticklabels(), rotation=15)

plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(6, 3))
sns.histplot(data=train, x='HasMultiplePhoneNumbers',
hue='HasPhoneService')

plt.show();
```



```
check_col_values(train)
ClientPeriod      [55 72  1 32 60 25 27 50 16 70 69 26 38 30 52
71 59 61  7 20 48 22  2 21
 44 66 58 46 45  5  4  3 62  6 39 68 51 40 10 43 12 35 31 67  8 11 29
37
 33 14 15 47 23 65 13 41 53  9 64 42 34 17 18 63 56 54 24 28 49 19 57
36
 0]
Sex                <ArrowStringArray>
['Male', 'Female']
Length: 2, dtype: str
IsSeniorCitizen    [0 1]
HasPartner         <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasChild           <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasPhoneService    <ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str
HasMultiplePhoneNumbers <ArrowStringArray>
['No', 'Yes', 'No phone service']
Length: 3, dtype: str
HasInternetService  <ArrowStringArray>
['No', 'Fiber optic', 'DSL']
Length: 3, dtype: str
HasOnlineSecurityService <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasOnlineBackup     <ArrowStringArray>
```

```

['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasDeviceProtection      <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasTechSupportAccess      <ArrowStringArray>
['No internet service', 'Yes', 'No']
Length: 3, dtype: str
HasOnlineTV               <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasMovieSubscription      <ArrowStringArray>
['No internet service', 'No', 'Yes']
Length: 3, dtype: str
HasContractPhone          <ArrowStringArray>
['One year', 'Two year', 'Month-to-month']
Length: 3, dtype: str
IsBillingPaperless        <ArrowStringArray>
['No', 'Yes']
Length: 2, dtype: str
PaymentMethod              <ArrowStringArray>
[
    'Mailed check', 'Credit card (automatic)',
    'Electronic check', 'Bank transfer (automatic)']
Length: 4, dtype: str
Churn                      [0 1]

65.04087452471482
2296.8981653992396

```

Предобработка признаков

Преобразование признаков

```

def prep_features(df, bins=None):
    bin_cols = ['Sex', 'HasPartner', 'HasChild', 'IsBillingPaperless']

    tmp = df.copy()

    # Бинаризируем признаки
    for col in bin_cols:
        if col == 'Sex':
            tmp[col] = (tmp[col] == 'Male').astype(int)
        else:
            tmp[col] = (tmp[col] == 'Yes').astype(int)

    # Сгруппируем с разделением на 10 групп периодов
    # tmp['ClientPeriodGroup'] = pd.cut(tmp['ClientPeriod'], bins=10,
labels=False) # Правильно применить к test нужно

```

```

# Сгруппируем с разделением на 10 групп периодов

# Для трейн
if bins is None:
    tmp['ClientPeriodGroup'], bins_edges =
pd.cut(tmp['ClientPeriod'],

                                             bins=10,
                                             labels=False,
                                             retbins=True)

    return tmp, bins_edges

# Для теста разбиение по интервалам разбиения трейна
else:
    # bins = np.array([-0.072, 7.2, 14.4, 21.6, 28.8, 36. , 43.2,
    50.4, 57.6, 64.8, 72. ])

    tmp['ClientPeriodGroup'] = pd.cut(tmp['ClientPeriod'],
                                      bins=bins,
                                      labels=False,
                                      include_lowest=True)

    return tmp

# Создание новых признаков

def create_features(df):
    cols = ['HasPhoneService',
            'HasInternetService', 'HasOnlineSecurityService',
            'HasOnlineBackup', 'HasDeviceProtection',
            'HasTechSupportAccess',
            'HasOnlineTV', 'HasMovieSubscription']

    tmp = df.copy()

    not_test = False

    if 'Churn' in tmp.columns:
        not_test = True
        churn = tmp['Churn'].copy()
        tmp = tmp.drop('Churn', axis=1)

    # Средние траты за месяц
    tmp['AverageMonthlySpend'] = tmp['TotalSpent'] /
(tmp['ClientPeriod'] + 0.0001)

    # Наличие семьи (сервисы на несколько человек)
    tmp['NotAlone'] = ((tmp['HasPartner'] == 1) | (tmp['HasChild'] ==
1)).astype(int)

    # Одинокие пенсионеры
    tmp['IsAloneSenior'] = ((tmp['IsSeniorCitizen'] == 1) &

```

```
(tmp['HasPartner'] == 0)).astype(int)

# Количество сервисов
mask = pd.DataFrame()
for col in cols:
    if col == 'HasInternetService':
        mask[col] = (tmp[col] != 'No')
    else:
        mask[col] = (tmp[col] == 'Yes')
tmp['HasMultipleServices'] = mask.sum(axis=1)

# Возвращаем целевую переменную
if not_test:
    tmp['Churn'] = churn

return tmp
```

```
train_prep, train_bins = prep_features(train)
train_generated = create_features(train_prep)
```

```
train_generated.head()
```

	ClientPeriod	MonthlySpending	TotalSpent	Sex	IsSeniorCitizen	\
0	55	19.50	1026.35	1	0	
1	72	25.85	1872.20	1	0	
2	1	75.90	75.90	1	0	
3	32	79.30	2570.00	0	1	
4	60	115.25	6758.45	0	0	

	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
0	1	1	Yes	No	
1	1	0	Yes	Yes	
2	0	0	Yes	No	
3	1	0	Yes	Yes	
4	1	1	Yes	Yes	

	HasInternetService	HasOnlineSecurityService	HasOnlineBackup	\
0	No	No internet service	No internet service	
1	No	No internet service	No internet service	
2	Fiber optic	No	No	
3	Fiber optic	No	No	
4	Fiber optic	Yes	Yes	

	HasDeviceProtection	HasTechSupportAccess	HasOnlineTV	\
0	No internet service	No internet service	No internet service	
1	No internet service	No internet service	No internet service	
2	No	Yes	No	
3	Yes	No	No	
4	Yes	Yes	Yes	

	HasMovieSubscription	HasContractPhone	IsBillingPaperless	\
0	No internet service	One year	0	
1	No internet service	Two year	0	
2	No	Month-to-month	1	
3	No	Month-to-month	0	
4	Yes	Two year	0	

	PaymentMethod	ClientPeriodGroup	AverageMonthlySpend
NotAlone \			
0	Mailed check	7	18.660875
1			
1	Credit card (automatic)	9	26.002742
1			
2	Electronic check	0	75.892411
0			
3	Mailed check	4	80.312249
1			
4	Credit card (automatic)	8	112.640646
1			

	IsAloneSenior	HasMultipleServices	Churn
0	0	1	0
1	0	1	0
2	0	3	1
3	0	3	0
4	0	8	0

```
# train_bins = np.array([-0.072, 7.2, 14.4, 21.6, 28.8, 36. , 43.2, 50.4, 57.6, 64.8, 72. ])
```

```
test_prep = prep_features(test, bins=train_bins)
test_generated = create_features(test_prep)
```

```
test_generated
```

	ClientPeriod	MonthlySpending	TotalSpent	Sex	IsSeniorCitizen
\					
0	42	56.10	2386.85	1	0
1	29	98.50	3004.15	1	1
2	9	76.25	684.85	1	0
3	67	74.00	4868.40	0	0
4	71	20.10	1389.60	0	0
...	...	...	...	...	...
1756	71	105.15	7555.00	0	0

1757	13	20.85	272.35	0	0
1758	2	91.45	171.45	1	0
1759	19	89.10	1620.80	0	0
1760	72	65.60	4566.50	1	0
	HasPartner	HasChild	HasPhoneService	HasMultiplePhoneNumbers	\
0	1	0	Yes		Yes
1	1	0	Yes		Yes
2	0	0	Yes		Yes
3	1	1	Yes		Yes
4	1	1	Yes		No
...	...	...	...		...
1756	1	0	Yes		Yes
1757	1	1	Yes		No
1758	1	0	Yes		Yes
1759	1	0	Yes		Yes
1760	0	0	No	No phone service	
	HasInternetService	HasOnlineSecurityService		HasOnlineBackup	
\					
0	DSL		Yes		No
1	Fiber optic		No		No
2	Fiber optic		No		No
3	DSL		Yes		Yes
4	No	No internet service	No internet service		
...	...		...		...
1756	Fiber optic		No		Yes
1757	No	No internet service	No internet service		
1758	Fiber optic		No		No
1759	Fiber optic		No		Yes
1760	DSL		Yes		Yes
	HasDeviceProtection	HasTechSupportAccess		HasOnlineTV	\
0	No	No		No	
1	Yes	No		Yes	
2	No	No		No	

3	No	Yes	Yes
4	No internet service	No internet service	No internet service
...	...	...	...
1756	Yes	No	Yes
1757	No internet service	No internet service	No internet service
1758	Yes	No	Yes
1759	No	No	No
1760	Yes	Yes	Yes

	HasMovieSubscription	HasContractPhone	IsBillingPaperless	\
0	No	One year	0	
1	Yes	Month-to-month	1	
2	No	Month-to-month	1	
3	No	Two year	0	
4	No internet service	Two year	0	
...	...	...	...	
1756	Yes	Two year	1	
1757	No internet service	Two year	0	
1758	No	Month-to-month	1	
1759	Yes	Month-to-month	1	
1760	Yes	Two year	1	

	PaymentMethod	ClientPeriodGroup
AverageMonthlySpend	\	
0	Credit card (automatic)	5
56.829627		
1	Electronic check	4
103.591022		
2	Electronic check	1
76.093599		
3	Credit card (automatic)	9
72.662578		
4	Bank transfer (automatic)	9
19.571803		
...	...	...
.		..
1756	Bank transfer (automatic)	9
106.408301		
1757	Mailed check	1
20.949839		
1758	Electronic check	0
85.720714		
1759	Electronic check	2
85.304814		
1760	Bank transfer (automatic)	9
63.423523		

	NotAlone	IsAloneSenior	HasMultipleServices
0	1	0	3
1	1	0	5

2	0	0	2
3	1	0	6
4	1	0	1
...	...	...	...
1756	1	0	6
1757	1	0	1
1758	1	0	4
1759	1	0	4
1760	0	0	7

[1761 rows x 24 columns]

```
# check_col_values(train_generated)
```

Посмотрим на корреляции с помощью phik, так как библиотека работает с числовыми и категориальными признаками.

```
corr_df = train_generated.phik_matrix()

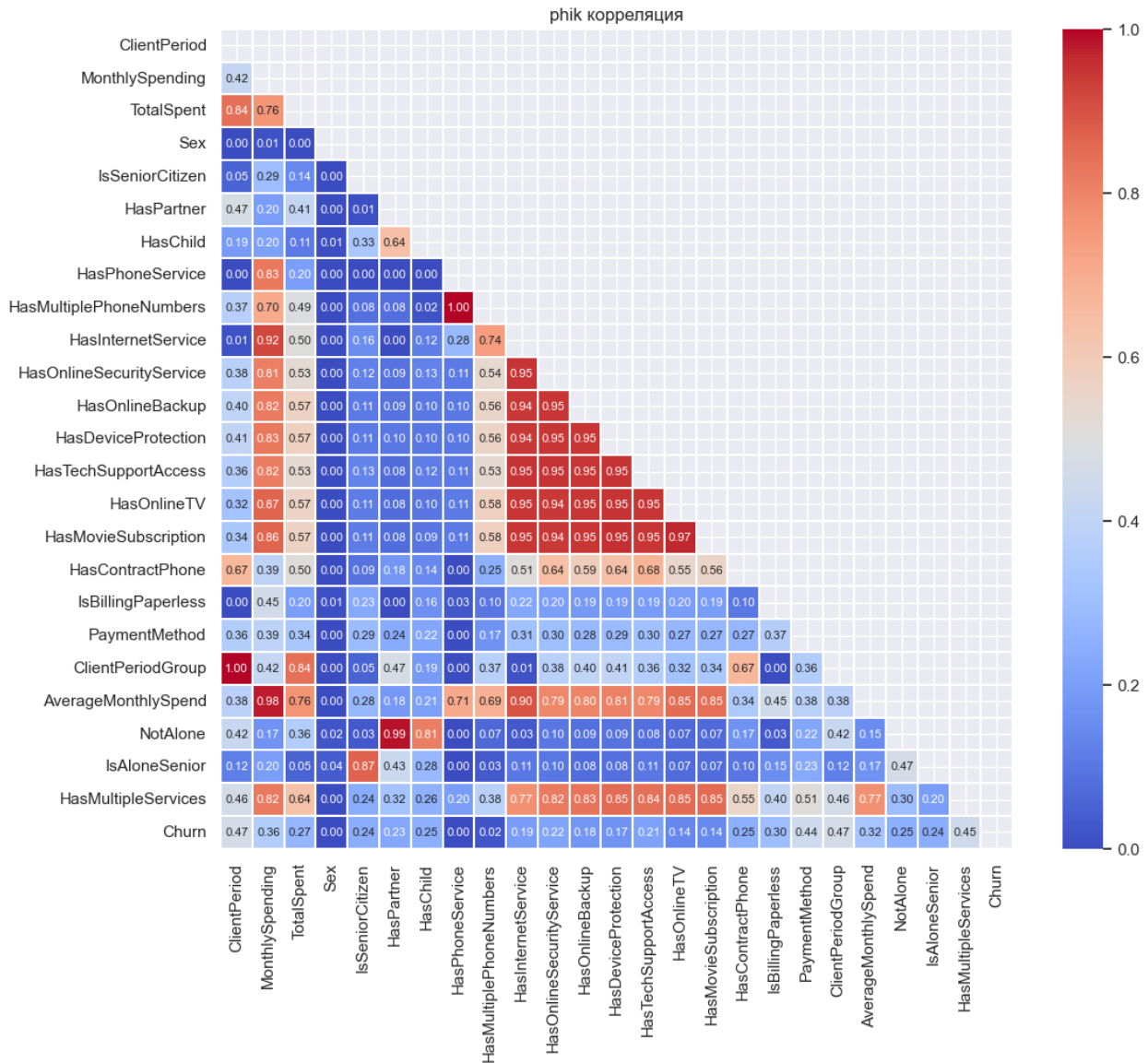
# corr_df

interval columns not set, guessing: ['ClientPeriod',
'MonthlySpending', 'TotalSpent', 'Sex', 'IsSeniorCitizen',
'HasPartner', 'HasChild', 'IsBillingPaperless', 'ClientPeriodGroup',
'AverageMonthlySpend', 'NotAlone', 'IsAloneSenior',
'HasMultipleServices', 'Churn']

plt.figure(figsize=(12, 10))
mask = np.triu(np.ones_like(corr_df, dtype=bool))
sns.heatmap(corr_df, mask=mask, annot=True, cmap='coolwarm',
fmt='.2f', annot_kws={'size': 8}, linewidths=0.3)

plt.title('phik корреляция')
plt.xticks(rotation=90)

plt.show();
```



Наблюдается сильная мультиколлинеарность нецелевых признаков. Попробуем в дальнейшем отбросить некоторые из них.

Отбор признаков

Все числовые признаки

```
num_cols = [
    'ClientPeriod',
    'AverageMonthlySpend',
    'HasMultipleServices',
    'NotAlone',
    'TotalSpent',
    'MonthlySpending',
    'ClientPeriodGroup',
```

```

        'IsAloneSenior',
        'IsSeniorCitizen',
    ]

    # Все категориальные признаки

    cat_cols = [
        'PaymentMethod',
        'IsBillingPaperless',
        'HasContractPhone',
        'HasInternetService',
        'Sex',
        'HasPartner',
        'HasChild',
        'HasPhoneService',
        'HasMultiplePhoneNumbers',
        'HasOnlineSecurityService',
        'HasOnlineBackup',
        'HasDeviceProtection',
        'HasTechSupportAccess',
        'HasOnlineTV',
        'HasMovieSubscription',
    ]

    # Все признаки
    features = num_cols + cat_cols

```

Разбиваем данные на X_train, y_train, X_test

```

X_train = train_generated[features]
y_train = train_generated['Churn']
X_test = test_generated[features]

```

Методы отбора признаков

Метод отбора признаков с высокой мультиколлинеарностью числовых признаков

```

from statsmodels.stats.outliers_influence import
variance_inflation_factor
import statsmodels.api as sm

X_num = train_generated[num_cols].copy()

X_with_const = sm.add_constant(X_num)

vif_values = []
for i in range(X_with_const.shape[1]):
    vif = variance_inflation_factor(X_with_const.values, i)
    vif_values.append(vif)

```

```

vif_df = pd.DataFrame({
    'feature': X_with_const.columns,
    'VIF': vif_values
})

vif_df = vif_df[vif_df['feature'] != 'const']

vif_df.sort_values('VIF', ascending=False).reset_index(drop=True)

```

	feature	VIF
0	ClientPeriod	143.504072
1	ClientPeriodGroup	138.146674
2	MonthlySpending	86.228645
3	AverageMonthlySpend	82.784594
4	TotalSpent	10.258401
5	HasMultipleServices	4.159321
6	IsAloneSenior	2.155817
7	IsSeniorCitizen	2.021265
8	NotAlone	1.315882

Признаки с высокими показателями мультиколлинеарности - кандидаты на удаление из списка для обучения модели.

Попробуем отобрать признаки на основе матрицы корреляции phik (матрица ранее уже была реализована).

```

threshold = 0.1 # порог отбора признаков

# Корреляцию нецелевых признаков с целевым признаком
corr_with_target = corr_df['Churn'].abs().sort_values(ascending=False)

# Список отобранных признаков
phik_features = corr_with_target[corr_with_target >
threshold].index.tolist()
phik_features.remove('Churn')

phik_features

```

```

['ClientPeriodGroup',
 'ClientPeriod',
 'HasMultipleServices',
 'PaymentMethod',
 'MonthlySpending',
 'AverageMonthlySpend',
 'IsBillingPaperless',
 'TotalSpent',
 'HasChild',
 'HasContractPhone',
 'NotAlone',

```

```
'IsSeniorCitizen',  
'IsAloneSenior',  
'HasPartner',  
'HasOnlineSecurityService',  
'HasTechSupportAccess',  
'HasInternetService',  
'HasOnlineBackup',  
'HasDeviceProtection',  
'HasMovieSubscription',  
'HasOnlineTV']
```

Подбор признаков с помощью ml-модели. Сразу попробуем через CatBoost.

```
X_train_cat = X_train.copy()  
X_test_cat = X_test.copy()  
  
num_features = [col for col in num_cols if col in X_train.columns]  
cat_features = [col for col in cat_cols if col in X_train.columns]  
  
# индексы категориальных колонок для catboost  
cat_indices = [i for i, col in enumerate(X_train_cat.columns) if col  
in cat_features]
```

Коммент

```
catboost = CatBoostClassifier(iterations=500,  
                              random_state=RANDOM_STATE,  
                              verbose=False,  
                              cat_features=cat_indices)  
  
catboost.fit(X_train_cat, y_train)  
  
# Важность признаков  
importance_cat = catboost.get_feature_importance()  
  
feature_importance_cat = pd.DataFrame({'feature': X_train_cat.columns,  
                                       'importance':  
importance_cat}).sort_values('importance', ascending=False)  
  
feature_importance_cat['feature'].to_list()  
  
['HasContractPhone',  
'ClientPeriod',  
'AverageMonthlySpend',  
'TotalSpent',  
'MonthlySpending',  
'HasOnlineSecurityService',  
'HasMultiplePhoneNumbers',  
'PaymentMethod',  
'HasInternetService',
```

```
'ClientPeriodGroup',
'HasOnlineBackup',
'HasTechSupportAccess',
'IsBillingPaperless',
'HasMovieSubscription',
'Sex',
'HasMultipleServices',
'HasDeviceProtection',
'HasOnlineTV',
'IsSeniorCitizen',
'HasChild',
'IsAloneSenior',
'HasPartner',
'NotAlone',
'HasPhoneService']
```

Лучшие признаки

```
# phik
# Train ROC-AUC: 0.8491

phik_features = [
    'ClientPeriodGroup',
    'ClientPeriod',
    'HasMultipleServices',
    'PaymentMethod',
    'MonthlySpending',
    'AverageMonthlySpend',
    'IsBillingPaperless',
    'TotalSpent',
    'HasChild',
    'HasContractPhone',
    'NotAlone',
    'IsSeniorCitizen',
    'IsAloneSenior',
    'HasPartner',
    'HasOnlineSecurityService',
    'HasTechSupportAccess',
    'HasInternetService',
    'HasOnlineBackup',
    'HasDeviceProtection',
    'HasMovieSubscription',
    'HasOnlineTV'
]

# catboost
# Train ROC-AUC: 0.8498
```

BEST!!!

```
cb_features = [  
    'HasContractPhone',  
    'ClientPeriod',  
    'AverageMonthlySpend',  
    'TotalSpent',  
    'MonthlySpending',  
    'HasOnlineSecurityService',  
    'HasMultiplePhoneNumbers',  
    'PaymentMethod',  
    'HasInternetService',  
    'ClientPeriodGroup',  
    'HasOnlineBackup',  
    'HasTechSupportAccess',  
    'IsBillingPaperless',  
    'HasMovieSubscription',  
    'Sex',  
    'HasMultipleServices',  
    'HasDeviceProtection',  
    'HasOnlineTV',  
    'IsSeniorCitizen',  
    'HasChild',  
    # 'IsAloneSenior',  
    # 'HasPartner',  
    # 'NotAlone',  
    # 'HasPhoneService'  
]
```

Несмотря на сильную взаимосвязь нецелевых признаков, сокращение некоторых таких признаков ухудшает метрику ROC-AUC модели. По условию нам нужен максимальный результат.

```
best_features = cb_features
```

Обучение и выбор модели

Применение линейных моделей (3 балла)

1) Обработайте данные для того, чтобы к ним можно было применить LogisticRegression. Т.е. отнормируйте числовые признаки, а категориальные закодируйте с помощью one-hot-encoding'a.

2) С помощью кроссвалидации или разделения на train/valid выборку протестируйте разные значения гиперпараметра C и выберите лучший (можно тестировать C=100, 10, 1, 0.1, 0.01, 0.001) по метрике ROC-AUC.

Если вы разделяете на train/valid, то используйте LogisticRegressionCV. Он сам при вызове .fit() подберет параметр C. (не забудьте передать scoring='roc_auc', чтобы при

кроссвалидации сравнивались значения этой метрики, и `refit=True`, чтобы при потом модель обучилась на всем датасете с лучшим параметром C).

(более сложный вариант) Если вы будете использовать кроссвалидацию, то преобразования данных и `LogisticRegression` нужно соединить в один `Pipeline` с помощью `make_pipeline`, как это делалось во втором семинаре. Потом `pipeline` надо передать в `GridSearchCV`. Для `one-hot-encoding`'а можно использовать комбинацию `LabelEncoder` + `OneHotEncoder` (сначала превращаем строки в числа, а потом числа превращаем в `one-hot` вектора.)

Выпишите какое лучшее качество и с какими параметрами вам удалось получить

```
from sklearn.linear_model import LogisticRegression,
LogisticRegressionCV
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, RobustScaler,
LabelEncoder
from sklearn.pipeline import make_pipeline
```

Разбиваем данные на `X_train`, `y_train`, `X_test` с учетом выбранных лучших признаков

```
num_features = [col for col in num_cols if col in best_features]
num_features

['ClientPeriod',
 'AverageMonthlySpend',
 'HasMultipleServices',
 'TotalSpent',
 'MonthlySpending',
 'ClientPeriodGroup',
 'IsSeniorCitizen']

cat_features = [col for col in cat_cols if col in best_features]
cat_features

['PaymentMethod',
 'IsBillingPaperless',
 'HasContractPhone',
 'HasInternetService',
 'Sex',
 'HasChild',
 'HasMultiplePhoneNumbers',
 'HasOnlineSecurityService',
 'HasOnlineBackup',
 'HasDeviceProtection',
 'HasTechSupportAccess',
 'HasOnlineTV',
 'HasMovieSubscription']
```

```
X_train = train_generated[num_features + cat_features].copy()
y_train = train_generated['Churn'].copy()
X_test = test_generated[num_features + cat_features].copy()
```

Pipeline для Логистической регрессии

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(use_cat_names=True,
            handle_unknown='ignore'), cat_features)
    ])

logreg_cv = LogisticRegressionCV(Cs=[0.001, 0.01, 0.1, 1, 10, 100],
    cv=5,
    scoring='roc_auc',
    random_state=RANDOM_STATE,
    max_iter=1000,
    class_weight='balanced',
    n_jobs=-1)

pipeline_lr = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', logreg_cv)
])

pipeline_lr.fit(X_train, y_train)

Pipeline(steps=[('preprocessor',
    ColumnTransformer(transformers=[('num',
    StandardScaler(),
    ['ClientPeriod',
    'AverageMonthlySpend',
    'HasMultipleServices',
    'TotalSpent',
    'MonthlySpending',
    'ClientPeriodGroup',
    'IsSeniorCitizen']]),
    ('cat',
    OneHotEncoder(handle_unknown='ignore',
    use_cat_names=True),
    ['PaymentMethod',
    'IsBillingPaperless',
```

```

        'HasContractPhone',
'HasInternetService', 'Sex',
        'HasChild',
'HasMultiplePhoneNumbers',
'HasOnlineSecurityService',
        'HasOnlineBackup',
'HasDeviceProtection',
'HasTechSupportAccess',
        'HasOnlineTV',
'HasMovieSubscription']]])),
        ('classifier',
         LogisticRegressionCV(Cs=[0.001, 0.01, 0.1, 1, 10,
100],
                             class_weight='balanced', cv=5,
                             max_iter=1000, n_jobs=-1,
random_state=42,
                             scoring='roc_auc'))))

y_pred_train_lr = pipeline_lr.predict_proba(X_train)[: , 1]
y_pred_test_lr = pipeline_lr.predict_proba(X_test)[: , 1]

# Результаты

best_c = pipeline_lr.named_steps['classifier'].C_[0]
roc_auc_lr = roc_auc_score(y_train, y_pred_train_lr)

print(f'Лучший параметр C: {best_c}')
print(f'Train ROC-AUC: {roc_auc_lr:.4f}')

# BEST Train ROC-AUC: 0.8498

Лучший параметр C: 10.0
Train ROC-AUC: 0.8498

# submission['Churn'] = y_pred_test_lr
# submission.to_csv(root + 'my_submission_lr.csv', index=False)

```

ROC-AUC вашего решения равен 0.847045373751737.

GridSearchCV Подберем гиперпараметры Логистической регрессии.

```

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=RANDOM_STATE,

```

```

max_iter=1000))
])

# Сетка гиперпараметров

param_grid = {
    'classifier__C': [0.001, 0.01, 0.1, 1, 10, 100],
    'classifier__class_weight': ['balanced', None],
    'classifier__solver': ['lbfgs', 'liblinear', 'newton-cg', 'sag',
    'saga'],
    # 'classifier__penalty': ['l2']
}

grid_search = GridSearchCV(estimator=pipeline,
                           param_grid=param_grid,
                           cv=5,
                           scoring='roc_auc',
                           n_jobs=-1,
                           verbose=1)

grid_search.fit(X_train, y_train)

Fitting 5 folds for each of 60 candidates, totalling 300 fits

GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('preprocessor',
ColumnTransformer(transformers=[('num',
StandardScaler(),
['ClientPeriod',
'AverageMonthlySpend',
'HasMultipleServices',
'TotalSpent',
'MonthlySpending',
'ClientPeriodGroup',
'IsSeniorCitizen'])],
('cat',
OneHotEncoder(handle_unknown='ignore',
use_cat_names=True),

```

```

['PaymentMethod',
'IsBillingPaperless',...
'HasDeviceProtection',
'HasTechSupportAccess',
'HasOnlineTV',
'HasMovieSubscription']]])),
                                ('classifier',
LogisticRegression(max_iter=1000,
random_state=42))),
                                n_jobs=-1,
                                param_grid={'classifier__C': [0.001, 0.01, 0.1, 1, 10,
100],
                                'classifier__class_weight': ['balanced',
None],
                                'classifier__solver': ['lbfgs', 'liblinear',
'newton-cg', 'sag',
'saga']},
                                scoring='roc_auc', verbose=1)

# Предсказания с лучшей моделью
y_pred_train_lr_gs = grid_search.predict_proba(X_train)[: , 1]
y_pred_test_lr_gs = grid_search.predict_proba(X_test)[: , 1]

train_auc = roc_auc_score(y_train, y_pred_train_lr_gs)

# Результаты

print('Лучшие параметры:', grid_search.best_params_)
print(f'Лучший ROC-AUC на кросс-валидации:
{grid_search.best_score_:.4f}')
print(f'Train ROC-AUC: {train_auc:.4f}')

# На тесте получился хуже результат, чем предыдущая реализация
Лучшие параметры: {'classifier__C': 100, 'classifier__class_weight':
None, 'classifier__solver': 'sag'}
Лучший ROC-AUC на кросс-валидации: 0.8464
Train ROC-AUC: 0.8500

# submission['Churn'] = y_pred_test
# submission.to_csv(root + 'my_submission_lr_gs.csv', index=False)

```

Optuna Применим мощный инструмент для перебора гиперпараметров и фичей.
Подберем гиперпараметры Логистической регрессии.


```

# study_lr = optuna.create_study(direction='maximize')
# study_lr.optimize(objective_lr, n_trials=200,
# show_progress_bar=True)

# best_params_lr = study_lr.best_params
# best_params_lr

# # {'C': 16.774054266041745, 'solver': 'lbfgs', 'max_iter': 843}

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(use_cat_names=True,
handle_unknown='ignore'), cat_features)
    ])

best_params_lr = {'C': 16.774054266041745, 'solver': 'lbfgs',
'max_iter': 843}

final_lr = LogisticRegression(**best_params_lr,
                               random_state=RANDOM_STATE,
                               n_jobs=-1)

pipeline_lr = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', final_lr)
])

pipeline_lr.fit(X_train, y_train)

Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('num',
StandardScaler(),
                                                ['ClientPeriod',
'AverageMonthlySpend',
'HasMultipleServices',
                                                'TotalSpent',
'MonthlySpending',
'ClientPeriodGroup',
'IsSeniorCitizen'])],
                  ('cat',
OneHotEncoder(handle_unknown='ignore',
use_cat_names=True),
                  ['PaymentMethod',

```

```

'IsBillingPaperless',
'HasContractPhone',
'HasInternetService', 'Sex',
'HasChild',
'HasMultiplePhoneNumbers',
'HasOnlineSecurityService',
'HasOnlineBackup',
'HasDeviceProtection',
'HasTechSupportAccess',
'HasOnlineTV',
'HasMovieSubscription']]])),
('classifier',
LogisticRegression(C=16.774054266041745,
max_iter=843,
n_jobs=-1, random_state=42)))

y_pred_train_lr_optuna = pipeline_lr.predict_proba(X_train)[: , 1]
y_pred_test_lr_optuna = pipeline_lr.predict_proba(X_test)[: , 1]
roc_auc_lr_optuna = roc_auc_score(y_train, y_pred_train_lr_optuna)
print(f'Train ROC-AUC: {roc_auc_lr_optuna:.4f}')
# Train ROC-AUC: 0.8500
Train ROC-AUC: 0.8500
# submission['Churn'] = y_pred_test_lr_optuna
# submission.to_csv(root + 'my_submission_lr_optuna.csv', index=False)

```

ROC-AUC вашего решения равен 0.8470340626312898

```

# Результат на тесте практически идентичен результату, полученному с
помощью LogisticRegressionCV
# 0.8470340626312898 против 0.847045373751737
print('Разница:', 0.847045373751737 - 0.8470340626312898)
Разница: 1.1311120447277112e-05

```

Применение градиентного бустинга (2 балла)

Если вы хотите получить баллы за точный ответ, то стоит попробовать градиентный бустинг. Часто градиентный бустинг с параметрами по умолчанию даст вам 80% результата за 0% усилий.

Мы будем использовать catboost, поэтому нам не надо кодировать категориальные признаки. Catboost сделает это сам (в .fit() надо передать cat_features=cat_cols). А численные признаки нормировать для моделей, основанных на деревьях не нужно.

1) Разделите выборку на train/valid. Протестируйте catboost со стандартными параметрами.

2) Протестируйте разные значения параметров количества деревьев и learning_rate'a и выберите лучшую по метрике ROC-AUC комбинацию.

(Дополнительно) Есть некоторые сложности с тем, чтобы использовать CatBoostClassifier вместе с GridSearchCV, поэтому мы не просим использовать кроссвалидацию. Но можете попробовать)

Выпишите, какое лучшее качество и с какими параметрами вам удалось получить

Разбиваем данные на X_train, y_train, X_test с учетом выбранных лучших признаков

```
num_features = [col for col in num_cols if col in best_features]
num_features

['ClientPeriod',
 'AverageMonthlySpend',
 'HasMultipleServices',
 'TotalSpent',
 'MonthlySpending',
 'ClientPeriodGroup',
 'IsSeniorCitizen']

cat_features = [col for col in cat_cols if col in best_features]
cat_features

['PaymentMethod',
 'IsBillingPaperless',
 'HasContractPhone',
 'HasInternetService',
 'Sex',
 'HasChild',
 'HasMultiplePhoneNumbers',
 'HasOnlineSecurityService',
 'HasOnlineBackup',
 'HasDeviceProtection',
 'HasTechSupportAccess',
 'HasOnlineTV',
 'HasMovieSubscription']

X_train = train_generated[num_features + cat_features].copy()
y_train = train_generated['Churn'].copy()
X_test = test_generated[num_features + cat_features].copy()

cb = CatBoostClassifier(iterations=1000,
                        cat_features=cat_features,
```

```
random_seed=RANDOM_STATE,  
eval_metric='AUC')
```

```
# Создаем пулы для catboost
```

```
train_pool = Pool(data=X_train,  
                  label=y_train,  
                  cat_features=cat_features  
                  )
```

```
test_pool = Pool(data=X_test,  
                 cat_features=cat_features  
                 )
```

```
cb.fit(train_pool)
```

```
Learning rate set to 0.020931
```

0:	total: 36.3ms	remaining: 36.3s
1:	total: 65.4ms	remaining: 32.7s
2:	total: 94.1ms	remaining: 31.3s
3:	total: 122ms	remaining: 30.3s
4:	total: 152ms	remaining: 30.2s
5:	total: 181ms	remaining: 29.9s
6:	total: 208ms	remaining: 29.6s
7:	total: 236ms	remaining: 29.2s
8:	total: 267ms	remaining: 29.4s
9:	total: 286ms	remaining: 28.3s
10:	total: 314ms	remaining: 28.3s
11:	total: 341ms	remaining: 28.1s
12:	total: 371ms	remaining: 28.2s
13:	total: 402ms	remaining: 28.3s
14:	total: 433ms	remaining: 28.4s
15:	total: 460ms	remaining: 28.3s
16:	total: 487ms	remaining: 28.2s
17:	total: 515ms	remaining: 28.1s
18:	total: 542ms	remaining: 28s
19:	total: 571ms	remaining: 28s
20:	total: 600ms	remaining: 28s
21:	total: 628ms	remaining: 27.9s
22:	total: 656ms	remaining: 27.9s
23:	total: 680ms	remaining: 27.7s
24:	total: 708ms	remaining: 27.6s
25:	total: 731ms	remaining: 27.4s
26:	total: 760ms	remaining: 27.4s
27:	total: 786ms	remaining: 27.3s
28:	total: 815ms	remaining: 27.3s
29:	total: 842ms	remaining: 27.2s
30:	total: 872ms	remaining: 27.3s
31:	total: 902ms	remaining: 27.3s
32:	total: 930ms	remaining: 27.2s

33:	total: 957ms	remaining: 27.2s
34:	total: 983ms	remaining: 27.1s
35:	total: 1.01s	remaining: 27s
36:	total: 1.04s	remaining: 27s
37:	total: 1.06s	remaining: 26.9s
38:	total: 1.09s	remaining: 26.8s
39:	total: 1.12s	remaining: 26.8s
40:	total: 1.15s	remaining: 26.9s
41:	total: 1.18s	remaining: 26.8s
42:	total: 1.2s	remaining: 26.8s
43:	total: 1.21s	remaining: 26.3s
44:	total: 1.24s	remaining: 26.3s
45:	total: 1.27s	remaining: 26.3s
46:	total: 1.29s	remaining: 26.2s
47:	total: 1.31s	remaining: 26.1s
48:	total: 1.34s	remaining: 26.1s
49:	total: 1.37s	remaining: 26s
50:	total: 1.4s	remaining: 26s
51:	total: 1.42s	remaining: 25.9s
52:	total: 1.44s	remaining: 25.7s
53:	total: 1.46s	remaining: 25.6s
54:	total: 1.49s	remaining: 25.6s
55:	total: 1.51s	remaining: 25.5s
56:	total: 1.54s	remaining: 25.6s
57:	total: 1.57s	remaining: 25.5s
58:	total: 1.58s	remaining: 25.2s
59:	total: 1.61s	remaining: 25.2s
60:	total: 1.64s	remaining: 25.2s
61:	total: 1.66s	remaining: 25.2s
62:	total: 1.69s	remaining: 25.2s
63:	total: 1.72s	remaining: 25.1s
64:	total: 1.75s	remaining: 25.1s
65:	total: 1.78s	remaining: 25.2s
66:	total: 1.8s	remaining: 25.1s
67:	total: 1.82s	remaining: 25s
68:	total: 1.85s	remaining: 25s
69:	total: 1.88s	remaining: 25s
70:	total: 1.91s	remaining: 24.9s
71:	total: 1.94s	remaining: 24.9s
72:	total: 1.97s	remaining: 25s
73:	total: 2s	remaining: 25s
74:	total: 2.02s	remaining: 25s
75:	total: 2.05s	remaining: 24.9s
76:	total: 2.08s	remaining: 24.9s
77:	total: 2.11s	remaining: 24.9s
78:	total: 2.13s	remaining: 24.9s
79:	total: 2.16s	remaining: 24.9s
80:	total: 2.19s	remaining: 24.9s
81:	total: 2.2s	remaining: 24.7s

82:	total: 2.23s	remaining: 24.7s
83:	total: 2.26s	remaining: 24.7s
84:	total: 2.28s	remaining: 24.5s
85:	total: 2.29s	remaining: 24.4s
86:	total: 2.32s	remaining: 24.3s
87:	total: 2.35s	remaining: 24.3s
88:	total: 2.38s	remaining: 24.3s
89:	total: 2.41s	remaining: 24.4s
90:	total: 2.44s	remaining: 24.4s
91:	total: 2.47s	remaining: 24.4s
92:	total: 2.5s	remaining: 24.4s
93:	total: 2.51s	remaining: 24.2s
94:	total: 2.54s	remaining: 24.2s
95:	total: 2.57s	remaining: 24.2s
96:	total: 2.6s	remaining: 24.2s
97:	total: 2.61s	remaining: 24.1s
98:	total: 2.64s	remaining: 24.1s
99:	total: 2.68s	remaining: 24.1s
100:	total: 2.71s	remaining: 24.1s
101:	total: 2.73s	remaining: 24.1s
102:	total: 2.76s	remaining: 24s
103:	total: 2.79s	remaining: 24s
104:	total: 2.81s	remaining: 24s
105:	total: 2.84s	remaining: 23.9s
106:	total: 2.87s	remaining: 23.9s
107:	total: 2.88s	remaining: 23.8s
108:	total: 2.91s	remaining: 23.8s
109:	total: 2.94s	remaining: 23.8s
110:	total: 2.97s	remaining: 23.8s
111:	total: 3s	remaining: 23.8s
112:	total: 3.03s	remaining: 23.8s
113:	total: 3.06s	remaining: 23.8s
114:	total: 3.08s	remaining: 23.7s
115:	total: 3.11s	remaining: 23.7s
116:	total: 3.14s	remaining: 23.7s
117:	total: 3.17s	remaining: 23.7s
118:	total: 3.2s	remaining: 23.7s
119:	total: 3.22s	remaining: 23.6s
120:	total: 3.25s	remaining: 23.6s
121:	total: 3.27s	remaining: 23.6s
122:	total: 3.3s	remaining: 23.5s
123:	total: 3.33s	remaining: 23.5s
124:	total: 3.36s	remaining: 23.5s
125:	total: 3.38s	remaining: 23.5s
126:	total: 3.41s	remaining: 23.5s
127:	total: 3.44s	remaining: 23.4s
128:	total: 3.47s	remaining: 23.4s
129:	total: 3.49s	remaining: 23.4s
130:	total: 3.52s	remaining: 23.4s

131:	total: 3.55s	remaining: 23.3s
132:	total: 3.58s	remaining: 23.3s
133:	total: 3.6s	remaining: 23.3s
134:	total: 3.63s	remaining: 23.3s
135:	total: 3.66s	remaining: 23.3s
136:	total: 3.69s	remaining: 23.3s
137:	total: 3.72s	remaining: 23.2s
138:	total: 3.75s	remaining: 23.2s
139:	total: 3.77s	remaining: 23.2s
140:	total: 3.8s	remaining: 23.2s
141:	total: 3.84s	remaining: 23.2s
142:	total: 3.87s	remaining: 23.2s
143:	total: 3.9s	remaining: 23.2s
144:	total: 3.92s	remaining: 23.1s
145:	total: 3.94s	remaining: 23.1s
146:	total: 3.95s	remaining: 22.9s
147:	total: 3.98s	remaining: 22.9s
148:	total: 4.01s	remaining: 22.9s
149:	total: 4.04s	remaining: 22.9s
150:	total: 4.05s	remaining: 22.8s
151:	total: 4.08s	remaining: 22.7s
152:	total: 4.11s	remaining: 22.7s
153:	total: 4.13s	remaining: 22.7s
154:	total: 4.16s	remaining: 22.7s
155:	total: 4.17s	remaining: 22.6s
156:	total: 4.2s	remaining: 22.6s
157:	total: 4.23s	remaining: 22.5s
158:	total: 4.26s	remaining: 22.5s
159:	total: 4.29s	remaining: 22.6s
160:	total: 4.33s	remaining: 22.6s
161:	total: 4.36s	remaining: 22.5s
162:	total: 4.37s	remaining: 22.4s
163:	total: 4.38s	remaining: 22.3s
164:	total: 4.41s	remaining: 22.3s
165:	total: 4.42s	remaining: 22.2s
166:	total: 4.45s	remaining: 22.2s
167:	total: 4.46s	remaining: 22.1s
168:	total: 4.49s	remaining: 22.1s
169:	total: 4.51s	remaining: 22s
170:	total: 4.54s	remaining: 22s
171:	total: 4.57s	remaining: 22s
172:	total: 4.6s	remaining: 22s
173:	total: 4.63s	remaining: 22s
174:	total: 4.66s	remaining: 22s
175:	total: 4.68s	remaining: 21.9s
176:	total: 4.71s	remaining: 21.9s
177:	total: 4.74s	remaining: 21.9s
178:	total: 4.77s	remaining: 21.9s
179:	total: 4.8s	remaining: 21.9s

180:	total: 4.83s	remaining: 21.9s
181:	total: 4.86s	remaining: 21.8s
182:	total: 4.89s	remaining: 21.8s
183:	total: 4.92s	remaining: 21.8s
184:	total: 4.94s	remaining: 21.8s
185:	total: 4.97s	remaining: 21.7s
186:	total: 5s	remaining: 21.7s
187:	total: 5.02s	remaining: 21.7s
188:	total: 5.05s	remaining: 21.7s
189:	total: 5.09s	remaining: 21.7s
190:	total: 5.12s	remaining: 21.7s
191:	total: 5.15s	remaining: 21.7s
192:	total: 5.16s	remaining: 21.6s
193:	total: 5.19s	remaining: 21.6s
194:	total: 5.2s	remaining: 21.5s
195:	total: 5.23s	remaining: 21.4s
196:	total: 5.23s	remaining: 21.3s
197:	total: 5.26s	remaining: 21.3s
198:	total: 5.27s	remaining: 21.2s
199:	total: 5.3s	remaining: 21.2s
200:	total: 5.32s	remaining: 21.2s
201:	total: 5.33s	remaining: 21.1s
202:	total: 5.36s	remaining: 21s
203:	total: 5.37s	remaining: 21s
204:	total: 5.4s	remaining: 20.9s
205:	total: 5.42s	remaining: 20.9s
206:	total: 5.44s	remaining: 20.8s
207:	total: 5.47s	remaining: 20.8s
208:	total: 5.48s	remaining: 20.7s
209:	total: 5.51s	remaining: 20.7s
210:	total: 5.54s	remaining: 20.7s
211:	total: 5.57s	remaining: 20.7s
212:	total: 5.59s	remaining: 20.7s
213:	total: 5.62s	remaining: 20.6s
214:	total: 5.63s	remaining: 20.6s
215:	total: 5.66s	remaining: 20.5s
216:	total: 5.69s	remaining: 20.5s
217:	total: 5.71s	remaining: 20.5s
218:	total: 5.74s	remaining: 20.5s
219:	total: 5.76s	remaining: 20.4s
220:	total: 5.76s	remaining: 20.3s
221:	total: 5.79s	remaining: 20.3s
222:	total: 5.82s	remaining: 20.3s
223:	total: 5.84s	remaining: 20.3s
224:	total: 5.88s	remaining: 20.2s
225:	total: 5.91s	remaining: 20.2s
226:	total: 5.94s	remaining: 20.2s
227:	total: 5.97s	remaining: 20.2s
228:	total: 6s	remaining: 20.2s

229:	total: 6.03s	remaining: 20.2s
230:	total: 6.06s	remaining: 20.2s
231:	total: 6.08s	remaining: 20.1s
232:	total: 6.11s	remaining: 20.1s
233:	total: 6.14s	remaining: 20.1s
234:	total: 6.15s	remaining: 20s
235:	total: 6.16s	remaining: 19.9s
236:	total: 6.18s	remaining: 19.9s
237:	total: 6.21s	remaining: 19.9s
238:	total: 6.24s	remaining: 19.9s
239:	total: 6.25s	remaining: 19.8s
240:	total: 6.28s	remaining: 19.8s
241:	total: 6.3s	remaining: 19.7s
242:	total: 6.33s	remaining: 19.7s
243:	total: 6.36s	remaining: 19.7s
244:	total: 6.39s	remaining: 19.7s
245:	total: 6.42s	remaining: 19.7s
246:	total: 6.45s	remaining: 19.7s
247:	total: 6.47s	remaining: 19.6s
248:	total: 6.5s	remaining: 19.6s
249:	total: 6.53s	remaining: 19.6s
250:	total: 6.54s	remaining: 19.5s
251:	total: 6.57s	remaining: 19.5s
252:	total: 6.58s	remaining: 19.4s
253:	total: 6.61s	remaining: 19.4s
254:	total: 6.64s	remaining: 19.4s
255:	total: 6.67s	remaining: 19.4s
256:	total: 6.7s	remaining: 19.4s
257:	total: 6.72s	remaining: 19.3s
258:	total: 6.74s	remaining: 19.3s
259:	total: 6.76s	remaining: 19.3s
260:	total: 6.78s	remaining: 19.2s
261:	total: 6.81s	remaining: 19.2s
262:	total: 6.84s	remaining: 19.2s
263:	total: 6.87s	remaining: 19.1s
264:	total: 6.88s	remaining: 19.1s
265:	total: 6.9s	remaining: 19s
266:	total: 6.93s	remaining: 19s
267:	total: 6.96s	remaining: 19s
268:	total: 6.98s	remaining: 19s
269:	total: 7.01s	remaining: 19s
270:	total: 7.05s	remaining: 19s
271:	total: 7.08s	remaining: 18.9s
272:	total: 7.11s	remaining: 18.9s
273:	total: 7.13s	remaining: 18.9s
274:	total: 7.16s	remaining: 18.9s
275:	total: 7.17s	remaining: 18.8s
276:	total: 7.2s	remaining: 18.8s
277:	total: 7.23s	remaining: 18.8s

278:	total: 7.25s	remaining: 18.7s
279:	total: 7.28s	remaining: 18.7s
280:	total: 7.3s	remaining: 18.7s
281:	total: 7.33s	remaining: 18.7s
282:	total: 7.36s	remaining: 18.6s
283:	total: 7.37s	remaining: 18.6s
284:	total: 7.4s	remaining: 18.6s
285:	total: 7.42s	remaining: 18.5s
286:	total: 7.46s	remaining: 18.5s
287:	total: 7.49s	remaining: 18.5s
288:	total: 7.5s	remaining: 18.5s
289:	total: 7.53s	remaining: 18.4s
290:	total: 7.56s	remaining: 18.4s
291:	total: 7.57s	remaining: 18.3s
292:	total: 7.58s	remaining: 18.3s
293:	total: 7.61s	remaining: 18.3s
294:	total: 7.63s	remaining: 18.2s
295:	total: 7.66s	remaining: 18.2s
296:	total: 7.68s	remaining: 18.2s
297:	total: 7.71s	remaining: 18.2s
298:	total: 7.74s	remaining: 18.2s
299:	total: 7.77s	remaining: 18.1s
300:	total: 7.8s	remaining: 18.1s
301:	total: 7.84s	remaining: 18.1s
302:	total: 7.86s	remaining: 18.1s
303:	total: 7.89s	remaining: 18.1s
304:	total: 7.91s	remaining: 18s
305:	total: 7.92s	remaining: 18s
306:	total: 7.96s	remaining: 18s
307:	total: 7.99s	remaining: 18s
308:	total: 8.03s	remaining: 18s
309:	total: 8.06s	remaining: 17.9s
310:	total: 8.09s	remaining: 17.9s
311:	total: 8.1s	remaining: 17.9s
312:	total: 8.12s	remaining: 17.8s
313:	total: 8.15s	remaining: 17.8s
314:	total: 8.18s	remaining: 17.8s
315:	total: 8.19s	remaining: 17.7s
316:	total: 8.22s	remaining: 17.7s
317:	total: 8.22s	remaining: 17.6s
318:	total: 8.26s	remaining: 17.6s
319:	total: 8.29s	remaining: 17.6s
320:	total: 8.31s	remaining: 17.6s
321:	total: 8.33s	remaining: 17.5s
322:	total: 8.35s	remaining: 17.5s
323:	total: 8.37s	remaining: 17.5s
324:	total: 8.4s	remaining: 17.5s
325:	total: 8.42s	remaining: 17.4s
326:	total: 8.43s	remaining: 17.4s

327:	total: 8.46s	remaining: 17.3s
328:	total: 8.49s	remaining: 17.3s
329:	total: 8.52s	remaining: 17.3s
330:	total: 8.54s	remaining: 17.3s
331:	total: 8.57s	remaining: 17.2s
332:	total: 8.61s	remaining: 17.2s
333:	total: 8.63s	remaining: 17.2s
334:	total: 8.67s	remaining: 17.2s
335:	total: 8.7s	remaining: 17.2s
336:	total: 8.72s	remaining: 17.2s
337:	total: 8.75s	remaining: 17.1s
338:	total: 8.78s	remaining: 17.1s
339:	total: 8.8s	remaining: 17.1s
340:	total: 8.83s	remaining: 17.1s
341:	total: 8.86s	remaining: 17s
342:	total: 8.89s	remaining: 17s
343:	total: 8.91s	remaining: 17s
344:	total: 8.94s	remaining: 17s
345:	total: 8.97s	remaining: 17s
346:	total: 8.98s	remaining: 16.9s
347:	total: 9.01s	remaining: 16.9s
348:	total: 9.03s	remaining: 16.8s
349:	total: 9.06s	remaining: 16.8s
350:	total: 9.09s	remaining: 16.8s
351:	total: 9.12s	remaining: 16.8s
352:	total: 9.14s	remaining: 16.8s
353:	total: 9.17s	remaining: 16.7s
354:	total: 9.2s	remaining: 16.7s
355:	total: 9.23s	remaining: 16.7s
356:	total: 9.26s	remaining: 16.7s
357:	total: 9.29s	remaining: 16.7s
358:	total: 9.3s	remaining: 16.6s
359:	total: 9.33s	remaining: 16.6s
360:	total: 9.36s	remaining: 16.6s
361:	total: 9.39s	remaining: 16.5s
362:	total: 9.42s	remaining: 16.5s
363:	total: 9.44s	remaining: 16.5s
364:	total: 9.47s	remaining: 16.5s
365:	total: 9.5s	remaining: 16.5s
366:	total: 9.53s	remaining: 16.4s
367:	total: 9.56s	remaining: 16.4s
368:	total: 9.59s	remaining: 16.4s
369:	total: 9.62s	remaining: 16.4s
370:	total: 9.63s	remaining: 16.3s
371:	total: 9.63s	remaining: 16.3s
372:	total: 9.64s	remaining: 16.2s
373:	total: 9.67s	remaining: 16.2s
374:	total: 9.7s	remaining: 16.2s
375:	total: 9.73s	remaining: 16.1s

376: total: 9.75s remaining: 16.1s
377: total: 9.78s remaining: 16.1s
378: total: 9.82s remaining: 16.1s
379: total: 9.85s remaining: 16.1s
380: total: 9.88s remaining: 16.1s
381: total: 9.91s remaining: 16s
382: total: 9.94s remaining: 16s
383: total: 9.98s remaining: 16s
384: total: 10s remaining: 16s
385: total: 10s remaining: 16s
386: total: 10.1s remaining: 15.9s
387: total: 10.1s remaining: 15.9s
388: total: 10.1s remaining: 15.9s
389: total: 10.1s remaining: 15.9s
390: total: 10.2s remaining: 15.8s
391: total: 10.2s remaining: 15.8s
392: total: 10.2s remaining: 15.8s
393: total: 10.2s remaining: 15.7s
394: total: 10.2s remaining: 15.7s
395: total: 10.3s remaining: 15.7s
396: total: 10.3s remaining: 15.6s
397: total: 10.3s remaining: 15.6s
398: total: 10.3s remaining: 15.6s
399: total: 10.4s remaining: 15.6s
400: total: 10.4s remaining: 15.5s
401: total: 10.4s remaining: 15.5s
402: total: 10.5s remaining: 15.5s
403: total: 10.5s remaining: 15.5s
404: total: 10.5s remaining: 15.5s
405: total: 10.5s remaining: 15.4s
406: total: 10.6s remaining: 15.4s
407: total: 10.6s remaining: 15.4s
408: total: 10.6s remaining: 15.3s
409: total: 10.6s remaining: 15.3s
410: total: 10.7s remaining: 15.3s
411: total: 10.7s remaining: 15.3s
412: total: 10.7s remaining: 15.2s
413: total: 10.8s remaining: 15.2s
414: total: 10.8s remaining: 15.2s
415: total: 10.8s remaining: 15.2s
416: total: 10.8s remaining: 15.2s
417: total: 10.9s remaining: 15.1s
418: total: 10.9s remaining: 15.1s
419: total: 10.9s remaining: 15.1s
420: total: 11s remaining: 15.1s
421: total: 11s remaining: 15s
422: total: 11s remaining: 15s
423: total: 11s remaining: 15s
424: total: 11.1s remaining: 15s

425:	total: 11.1s	remaining: 14.9s
426:	total: 11.1s	remaining: 14.9s
427:	total: 11.2s	remaining: 14.9s
428:	total: 11.2s	remaining: 14.9s
429:	total: 11.2s	remaining: 14.9s
430:	total: 11.2s	remaining: 14.8s
431:	total: 11.3s	remaining: 14.8s
432:	total: 11.3s	remaining: 14.8s
433:	total: 11.3s	remaining: 14.8s
434:	total: 11.4s	remaining: 14.7s
435:	total: 11.4s	remaining: 14.7s
436:	total: 11.4s	remaining: 14.7s
437:	total: 11.4s	remaining: 14.7s
438:	total: 11.5s	remaining: 14.7s
439:	total: 11.5s	remaining: 14.6s
440:	total: 11.5s	remaining: 14.6s
441:	total: 11.6s	remaining: 14.6s
442:	total: 11.6s	remaining: 14.6s
443:	total: 11.6s	remaining: 14.5s
444:	total: 11.6s	remaining: 14.5s
445:	total: 11.7s	remaining: 14.5s
446:	total: 11.7s	remaining: 14.5s
447:	total: 11.7s	remaining: 14.4s
448:	total: 11.7s	remaining: 14.4s
449:	total: 11.8s	remaining: 14.4s
450:	total: 11.8s	remaining: 14.4s
451:	total: 11.8s	remaining: 14.3s
452:	total: 11.9s	remaining: 14.3s
453:	total: 11.9s	remaining: 14.3s
454:	total: 11.9s	remaining: 14.3s
455:	total: 11.9s	remaining: 14.2s
456:	total: 12s	remaining: 14.2s
457:	total: 12s	remaining: 14.2s
458:	total: 12s	remaining: 14.2s
459:	total: 12.1s	remaining: 14.2s
460:	total: 12.1s	remaining: 14.1s
461:	total: 12.1s	remaining: 14.1s
462:	total: 12.1s	remaining: 14.1s
463:	total: 12.2s	remaining: 14.1s
464:	total: 12.2s	remaining: 14s
465:	total: 12.2s	remaining: 14s
466:	total: 12.3s	remaining: 14s
467:	total: 12.3s	remaining: 14s
468:	total: 12.3s	remaining: 13.9s
469:	total: 12.3s	remaining: 13.9s
470:	total: 12.4s	remaining: 13.9s
471:	total: 12.4s	remaining: 13.9s
472:	total: 12.4s	remaining: 13.9s
473:	total: 12.5s	remaining: 13.8s

474:	total:	12.5s	remaining:	13.8s
475:	total:	12.5s	remaining:	13.8s
476:	total:	12.6s	remaining:	13.8s
477:	total:	12.6s	remaining:	13.7s
478:	total:	12.6s	remaining:	13.7s
479:	total:	12.7s	remaining:	13.7s
480:	total:	12.7s	remaining:	13.7s
481:	total:	12.7s	remaining:	13.7s
482:	total:	12.7s	remaining:	13.6s
483:	total:	12.8s	remaining:	13.6s
484:	total:	12.8s	remaining:	13.6s
485:	total:	12.8s	remaining:	13.6s
486:	total:	12.8s	remaining:	13.5s
487:	total:	12.9s	remaining:	13.5s
488:	total:	12.9s	remaining:	13.5s
489:	total:	12.9s	remaining:	13.5s
490:	total:	13s	remaining:	13.4s
491:	total:	13s	remaining:	13.4s
492:	total:	13s	remaining:	13.4s
493:	total:	13s	remaining:	13.4s
494:	total:	13.1s	remaining:	13.3s
495:	total:	13.1s	remaining:	13.3s
496:	total:	13.1s	remaining:	13.3s
497:	total:	13.2s	remaining:	13.3s
498:	total:	13.2s	remaining:	13.2s
499:	total:	13.2s	remaining:	13.2s
500:	total:	13.2s	remaining:	13.2s
501:	total:	13.3s	remaining:	13.2s
502:	total:	13.3s	remaining:	13.1s
503:	total:	13.3s	remaining:	13.1s
504:	total:	13.4s	remaining:	13.1s
505:	total:	13.4s	remaining:	13.1s
506:	total:	13.4s	remaining:	13s
507:	total:	13.4s	remaining:	13s
508:	total:	13.5s	remaining:	13s
509:	total:	13.5s	remaining:	13s
510:	total:	13.5s	remaining:	12.9s
511:	total:	13.6s	remaining:	12.9s
512:	total:	13.6s	remaining:	12.9s
513:	total:	13.6s	remaining:	12.9s
514:	total:	13.6s	remaining:	12.8s
515:	total:	13.7s	remaining:	12.8s
516:	total:	13.7s	remaining:	12.8s
517:	total:	13.7s	remaining:	12.8s
518:	total:	13.8s	remaining:	12.8s
519:	total:	13.8s	remaining:	12.7s
520:	total:	13.8s	remaining:	12.7s
521:	total:	13.8s	remaining:	12.7s
522:	total:	13.9s	remaining:	12.7s
523:	total:	13.9s	remaining:	12.6s

524: total: 13.9s remaining: 12.6s
525: total: 13.9s remaining: 12.6s
526: total: 14s remaining: 12.5s
527: total: 14s remaining: 12.5s
528: total: 14s remaining: 12.5s
529: total: 14.1s remaining: 12.5s
530: total: 14.1s remaining: 12.4s
531: total: 14.1s remaining: 12.4s
532: total: 14.1s remaining: 12.4s
533: total: 14.2s remaining: 12.4s
534: total: 14.2s remaining: 12.3s
535: total: 14.2s remaining: 12.3s
536: total: 14.3s remaining: 12.3s
537: total: 14.3s remaining: 12.3s
538: total: 14.3s remaining: 12.2s
539: total: 14.3s remaining: 12.2s
540: total: 14.4s remaining: 12.2s
541: total: 14.4s remaining: 12.2s
542: total: 14.4s remaining: 12.1s
543: total: 14.5s remaining: 12.1s
544: total: 14.5s remaining: 12.1s
545: total: 14.5s remaining: 12.1s
546: total: 14.6s remaining: 12.1s
547: total: 14.6s remaining: 12s
548: total: 14.6s remaining: 12s
549: total: 14.6s remaining: 12s
550: total: 14.7s remaining: 11.9s
551: total: 14.7s remaining: 11.9s
552: total: 14.7s remaining: 11.9s
553: total: 14.7s remaining: 11.9s
554: total: 14.8s remaining: 11.8s
555: total: 14.8s remaining: 11.8s
556: total: 14.8s remaining: 11.8s
557: total: 14.9s remaining: 11.8s
558: total: 14.9s remaining: 11.8s
559: total: 14.9s remaining: 11.7s
560: total: 15s remaining: 11.7s
561: total: 15s remaining: 11.7s
562: total: 15s remaining: 11.6s
563: total: 15.1s remaining: 11.6s
564: total: 15.1s remaining: 11.6s
565: total: 15.2s remaining: 11.6s
566: total: 15.2s remaining: 11.6s
567: total: 15.2s remaining: 11.6s
568: total: 15.2s remaining: 11.5s
569: total: 15.3s remaining: 11.5s
570: total: 15.3s remaining: 11.5s
571: total: 15.3s remaining: 11.5s
572: total: 15.4s remaining: 11.4s

573:	total: 15.4s	remaining: 11.4s
574:	total: 15.4s	remaining: 11.4s
575:	total: 15.4s	remaining: 11.4s
576:	total: 15.5s	remaining: 11.3s
577:	total: 15.5s	remaining: 11.3s
578:	total: 15.5s	remaining: 11.3s
579:	total: 15.5s	remaining: 11.3s
580:	total: 15.6s	remaining: 11.2s
581:	total: 15.6s	remaining: 11.2s
582:	total: 15.6s	remaining: 11.2s
583:	total: 15.7s	remaining: 11.2s
584:	total: 15.7s	remaining: 11.1s
585:	total: 15.7s	remaining: 11.1s
586:	total: 15.7s	remaining: 11.1s
587:	total: 15.8s	remaining: 11.1s
588:	total: 15.8s	remaining: 11s
589:	total: 15.8s	remaining: 11s
590:	total: 15.9s	remaining: 11s
591:	total: 15.9s	remaining: 11s
592:	total: 15.9s	remaining: 10.9s
593:	total: 16s	remaining: 10.9s
594:	total: 16s	remaining: 10.9s
595:	total: 16s	remaining: 10.9s
596:	total: 16s	remaining: 10.8s
597:	total: 16.1s	remaining: 10.8s
598:	total: 16.1s	remaining: 10.8s
599:	total: 16.1s	remaining: 10.7s
600:	total: 16.1s	remaining: 10.7s
601:	total: 16.2s	remaining: 10.7s
602:	total: 16.2s	remaining: 10.7s
603:	total: 16.2s	remaining: 10.6s
604:	total: 16.3s	remaining: 10.6s
605:	total: 16.3s	remaining: 10.6s
606:	total: 16.3s	remaining: 10.6s
607:	total: 16.3s	remaining: 10.5s
608:	total: 16.4s	remaining: 10.5s
609:	total: 16.4s	remaining: 10.5s
610:	total: 16.4s	remaining: 10.5s
611:	total: 16.4s	remaining: 10.4s
612:	total: 16.5s	remaining: 10.4s
613:	total: 16.5s	remaining: 10.4s
614:	total: 16.5s	remaining: 10.3s
615:	total: 16.6s	remaining: 10.3s
616:	total: 16.6s	remaining: 10.3s
617:	total: 16.6s	remaining: 10.3s
618:	total: 16.6s	remaining: 10.2s
619:	total: 16.7s	remaining: 10.2s
620:	total: 16.7s	remaining: 10.2s
621:	total: 16.7s	remaining: 10.2s

622:	total: 16.8s	remaining: 10.1s
623:	total: 16.8s	remaining: 10.1s
624:	total: 16.8s	remaining: 10.1s
625:	total: 16.8s	remaining: 10.1s
626:	total: 16.9s	remaining: 10s
627:	total: 16.9s	remaining: 10s
628:	total: 16.9s	remaining: 9.98s
629:	total: 17s	remaining: 9.96s
630:	total: 17s	remaining: 9.93s
631:	total: 17s	remaining: 9.9s
632:	total: 17s	remaining: 9.88s
633:	total: 17.1s	remaining: 9.85s
634:	total: 17.1s	remaining: 9.83s
635:	total: 17.1s	remaining: 9.8s
636:	total: 17.2s	remaining: 9.78s
637:	total: 17.2s	remaining: 9.75s
638:	total: 17.2s	remaining: 9.72s
639:	total: 17.2s	remaining: 9.7s
640:	total: 17.3s	remaining: 9.67s
641:	total: 17.3s	remaining: 9.64s
642:	total: 17.3s	remaining: 9.62s
643:	total: 17.4s	remaining: 9.59s
644:	total: 17.4s	remaining: 9.57s
645:	total: 17.4s	remaining: 9.54s
646:	total: 17.4s	remaining: 9.51s
647:	total: 17.5s	remaining: 9.49s
648:	total: 17.5s	remaining: 9.46s
649:	total: 17.5s	remaining: 9.44s
650:	total: 17.6s	remaining: 9.41s
651:	total: 17.6s	remaining: 9.38s
652:	total: 17.6s	remaining: 9.36s
653:	total: 17.6s	remaining: 9.33s
654:	total: 17.7s	remaining: 9.3s
655:	total: 17.7s	remaining: 9.28s
656:	total: 17.7s	remaining: 9.25s
657:	total: 17.7s	remaining: 9.22s
658:	total: 17.8s	remaining: 9.2s
659:	total: 17.8s	remaining: 9.17s
660:	total: 17.8s	remaining: 9.14s
661:	total: 17.9s	remaining: 9.12s
662:	total: 17.9s	remaining: 9.09s
663:	total: 17.9s	remaining: 9.06s
664:	total: 17.9s	remaining: 9.04s
665:	total: 18s	remaining: 9.01s
666:	total: 18s	remaining: 8.98s
667:	total: 18s	remaining: 8.96s
668:	total: 18s	remaining: 8.93s
669:	total: 18.1s	remaining: 8.9s
670:	total: 18.1s	remaining: 8.88s

671:	total: 18.1s	remaining: 8.85s
672:	total: 18.2s	remaining: 8.82s
673:	total: 18.2s	remaining: 8.79s
674:	total: 18.2s	remaining: 8.77s
675:	total: 18.2s	remaining: 8.74s
676:	total: 18.3s	remaining: 8.72s
677:	total: 18.3s	remaining: 8.69s
678:	total: 18.3s	remaining: 8.66s
679:	total: 18.4s	remaining: 8.64s
680:	total: 18.4s	remaining: 8.61s
681:	total: 18.4s	remaining: 8.58s
682:	total: 18.4s	remaining: 8.56s
683:	total: 18.5s	remaining: 8.53s
684:	total: 18.5s	remaining: 8.51s
685:	total: 18.5s	remaining: 8.48s
686:	total: 18.6s	remaining: 8.45s
687:	total: 18.6s	remaining: 8.43s
688:	total: 18.6s	remaining: 8.4s
689:	total: 18.6s	remaining: 8.38s
690:	total: 18.7s	remaining: 8.35s
691:	total: 18.7s	remaining: 8.32s
692:	total: 18.7s	remaining: 8.3s
693:	total: 18.8s	remaining: 8.27s
694:	total: 18.8s	remaining: 8.24s
695:	total: 18.8s	remaining: 8.22s
696:	total: 18.9s	remaining: 8.19s
697:	total: 18.9s	remaining: 8.17s
698:	total: 18.9s	remaining: 8.14s
699:	total: 18.9s	remaining: 8.12s
700:	total: 19s	remaining: 8.09s
701:	total: 19s	remaining: 8.06s
702:	total: 19s	remaining: 8.04s
703:	total: 19s	remaining: 8.01s
704:	total: 19.1s	remaining: 7.98s
705:	total: 19.1s	remaining: 7.96s
706:	total: 19.1s	remaining: 7.93s
707:	total: 19.2s	remaining: 7.9s
708:	total: 19.2s	remaining: 7.88s
709:	total: 19.2s	remaining: 7.85s
710:	total: 19.2s	remaining: 7.82s
711:	total: 19.3s	remaining: 7.79s
712:	total: 19.3s	remaining: 7.77s
713:	total: 19.3s	remaining: 7.74s
714:	total: 19.4s	remaining: 7.71s
715:	total: 19.4s	remaining: 7.69s
716:	total: 19.4s	remaining: 7.66s
717:	total: 19.4s	remaining: 7.64s
718:	total: 19.5s	remaining: 7.61s
719:	total: 19.5s	remaining: 7.58s

720:	total: 19.5s	remaining: 7.56s
721:	total: 19.6s	remaining: 7.53s
722:	total: 19.6s	remaining: 7.5s
723:	total: 19.6s	remaining: 7.48s
724:	total: 19.6s	remaining: 7.45s
725:	total: 19.7s	remaining: 7.42s
726:	total: 19.7s	remaining: 7.4s
727:	total: 19.7s	remaining: 7.37s
728:	total: 19.8s	remaining: 7.34s
729:	total: 19.8s	remaining: 7.32s
730:	total: 19.8s	remaining: 7.29s
731:	total: 19.8s	remaining: 7.26s
732:	total: 19.9s	remaining: 7.24s
733:	total: 19.9s	remaining: 7.21s
734:	total: 19.9s	remaining: 7.18s
735:	total: 20s	remaining: 7.16s
736:	total: 20s	remaining: 7.13s
737:	total: 20s	remaining: 7.1s
738:	total: 20s	remaining: 7.08s
739:	total: 20.1s	remaining: 7.05s
740:	total: 20.1s	remaining: 7.02s
741:	total: 20.1s	remaining: 7s
742:	total: 20.2s	remaining: 6.97s
743:	total: 20.2s	remaining: 6.94s
744:	total: 20.2s	remaining: 6.92s
745:	total: 20.2s	remaining: 6.89s
746:	total: 20.3s	remaining: 6.86s
747:	total: 20.3s	remaining: 6.84s
748:	total: 20.3s	remaining: 6.81s
749:	total: 20.3s	remaining: 6.78s
750:	total: 20.4s	remaining: 6.76s
751:	total: 20.4s	remaining: 6.73s
752:	total: 20.4s	remaining: 6.7s
753:	total: 20.5s	remaining: 6.67s
754:	total: 20.5s	remaining: 6.65s
755:	total: 20.5s	remaining: 6.62s
756:	total: 20.5s	remaining: 6.59s
757:	total: 20.6s	remaining: 6.57s
758:	total: 20.6s	remaining: 6.54s
759:	total: 20.6s	remaining: 6.51s
760:	total: 20.7s	remaining: 6.49s
761:	total: 20.7s	remaining: 6.46s
762:	total: 20.7s	remaining: 6.43s
763:	total: 20.7s	remaining: 6.41s
764:	total: 20.8s	remaining: 6.38s
765:	total: 20.8s	remaining: 6.35s
766:	total: 20.8s	remaining: 6.33s
767:	total: 20.9s	remaining: 6.3s
768:	total: 20.9s	remaining: 6.27s

769: total: 20.9s remaining: 6.25s
770: total: 20.9s remaining: 6.22s
771: total: 21s remaining: 6.19s
772: total: 21s remaining: 6.17s
773: total: 21s remaining: 6.14s
774: total: 21s remaining: 6.11s
775: total: 21.1s remaining: 6.08s
776: total: 21.1s remaining: 6.06s
777: total: 21.1s remaining: 6.03s
778: total: 21.2s remaining: 6s
779: total: 21.2s remaining: 5.98s
780: total: 21.2s remaining: 5.95s
781: total: 21.2s remaining: 5.92s
782: total: 21.3s remaining: 5.9s
783: total: 21.3s remaining: 5.87s
784: total: 21.3s remaining: 5.84s
785: total: 21.4s remaining: 5.82s
786: total: 21.4s remaining: 5.79s
787: total: 21.4s remaining: 5.76s
788: total: 21.4s remaining: 5.74s
789: total: 21.5s remaining: 5.71s
790: total: 21.5s remaining: 5.68s
791: total: 21.5s remaining: 5.65s
792: total: 21.6s remaining: 5.63s
793: total: 21.6s remaining: 5.6s
794: total: 21.6s remaining: 5.57s
795: total: 21.6s remaining: 5.55s
796: total: 21.7s remaining: 5.52s
797: total: 21.7s remaining: 5.49s
798: total: 21.7s remaining: 5.47s
799: total: 21.8s remaining: 5.44s
800: total: 21.8s remaining: 5.41s
801: total: 21.8s remaining: 5.39s
802: total: 21.8s remaining: 5.36s
803: total: 21.9s remaining: 5.33s
804: total: 21.9s remaining: 5.31s
805: total: 21.9s remaining: 5.28s
806: total: 22s remaining: 5.25s
807: total: 22s remaining: 5.23s
808: total: 22s remaining: 5.2s
809: total: 22s remaining: 5.17s
810: total: 22.1s remaining: 5.14s
811: total: 22.1s remaining: 5.12s
812: total: 22.1s remaining: 5.09s
813: total: 22.2s remaining: 5.06s
814: total: 22.2s remaining: 5.04s
815: total: 22.2s remaining: 5.01s
816: total: 22.2s remaining: 4.98s
817: total: 22.3s remaining: 4.96s

818:	total: 22.3s	remaining: 4.93s
819:	total: 22.3s	remaining: 4.9s
820:	total: 22.4s	remaining: 4.87s
821:	total: 22.4s	remaining: 4.85s
822:	total: 22.4s	remaining: 4.82s
823:	total: 22.4s	remaining: 4.79s
824:	total: 22.5s	remaining: 4.77s
825:	total: 22.5s	remaining: 4.74s
826:	total: 22.5s	remaining: 4.71s
827:	total: 22.6s	remaining: 4.69s
828:	total: 22.6s	remaining: 4.66s
829:	total: 22.6s	remaining: 4.63s
830:	total: 22.6s	remaining: 4.61s
831:	total: 22.7s	remaining: 4.58s
832:	total: 22.7s	remaining: 4.55s
833:	total: 22.7s	remaining: 4.53s
834:	total: 22.8s	remaining: 4.5s
835:	total: 22.8s	remaining: 4.47s
836:	total: 22.8s	remaining: 4.45s
837:	total: 22.9s	remaining: 4.42s
838:	total: 22.9s	remaining: 4.39s
839:	total: 22.9s	remaining: 4.37s
840:	total: 23s	remaining: 4.34s
841:	total: 23s	remaining: 4.31s
842:	total: 23s	remaining: 4.29s
843:	total: 23.1s	remaining: 4.26s
844:	total: 23.1s	remaining: 4.24s
845:	total: 23.1s	remaining: 4.21s
846:	total: 23.2s	remaining: 4.18s
847:	total: 23.2s	remaining: 4.16s
848:	total: 23.2s	remaining: 4.13s
849:	total: 23.3s	remaining: 4.1s
850:	total: 23.3s	remaining: 4.08s
851:	total: 23.3s	remaining: 4.05s
852:	total: 23.3s	remaining: 4.02s
853:	total: 23.4s	remaining: 4s
854:	total: 23.4s	remaining: 3.97s
855:	total: 23.4s	remaining: 3.94s
856:	total: 23.5s	remaining: 3.91s
857:	total: 23.5s	remaining: 3.88s
858:	total: 23.5s	remaining: 3.86s
859:	total: 23.5s	remaining: 3.83s
860:	total: 23.6s	remaining: 3.8s
861:	total: 23.6s	remaining: 3.78s
862:	total: 23.6s	remaining: 3.75s
863:	total: 23.7s	remaining: 3.72s
864:	total: 23.7s	remaining: 3.7s
865:	total: 23.7s	remaining: 3.67s
866:	total: 23.7s	remaining: 3.64s

867:	total: 23.8s	remaining: 3.61s
868:	total: 23.8s	remaining: 3.59s
869:	total: 23.8s	remaining: 3.56s
870:	total: 23.8s	remaining: 3.53s
871:	total: 23.9s	remaining: 3.5s
872:	total: 23.9s	remaining: 3.48s
873:	total: 23.9s	remaining: 3.45s
874:	total: 24s	remaining: 3.42s
875:	total: 24s	remaining: 3.4s
876:	total: 24s	remaining: 3.37s
877:	total: 24.1s	remaining: 3.34s
878:	total: 24.1s	remaining: 3.31s
879:	total: 24.1s	remaining: 3.29s
880:	total: 24.1s	remaining: 3.26s
881:	total: 24.2s	remaining: 3.23s
882:	total: 24.2s	remaining: 3.21s
883:	total: 24.2s	remaining: 3.18s
884:	total: 24.2s	remaining: 3.15s
885:	total: 24.3s	remaining: 3.12s
886:	total: 24.3s	remaining: 3.1s
887:	total: 24.3s	remaining: 3.07s
888:	total: 24.4s	remaining: 3.04s
889:	total: 24.4s	remaining: 3.01s
890:	total: 24.4s	remaining: 2.99s
891:	total: 24.4s	remaining: 2.96s
892:	total: 24.5s	remaining: 2.93s
893:	total: 24.5s	remaining: 2.91s
894:	total: 24.5s	remaining: 2.88s
895:	total: 24.6s	remaining: 2.85s
896:	total: 24.6s	remaining: 2.82s
897:	total: 24.6s	remaining: 2.8s
898:	total: 24.7s	remaining: 2.77s
899:	total: 24.7s	remaining: 2.74s
900:	total: 24.7s	remaining: 2.71s
901:	total: 24.7s	remaining: 2.69s
902:	total: 24.8s	remaining: 2.66s
903:	total: 24.8s	remaining: 2.63s
904:	total: 24.8s	remaining: 2.6s
905:	total: 24.9s	remaining: 2.58s
906:	total: 24.9s	remaining: 2.55s
907:	total: 24.9s	remaining: 2.52s
908:	total: 24.9s	remaining: 2.5s
909:	total: 25s	remaining: 2.47s
910:	total: 25s	remaining: 2.44s
911:	total: 25s	remaining: 2.42s
912:	total: 25.1s	remaining: 2.39s
913:	total: 25.1s	remaining: 2.36s
914:	total: 25.1s	remaining: 2.33s
915:	total: 25.1s	remaining: 2.31s

916:	total: 25.2s	remaining: 2.28s
917:	total: 25.2s	remaining: 2.25s
918:	total: 25.2s	remaining: 2.22s
919:	total: 25.3s	remaining: 2.2s
920:	total: 25.3s	remaining: 2.17s
921:	total: 25.3s	remaining: 2.14s
922:	total: 25.3s	remaining: 2.11s
923:	total: 25.4s	remaining: 2.09s
924:	total: 25.4s	remaining: 2.06s
925:	total: 25.4s	remaining: 2.03s
926:	total: 25.5s	remaining: 2s
927:	total: 25.5s	remaining: 1.98s
928:	total: 25.5s	remaining: 1.95s
929:	total: 25.5s	remaining: 1.92s
930:	total: 25.6s	remaining: 1.89s
931:	total: 25.6s	remaining: 1.87s
932:	total: 25.6s	remaining: 1.84s
933:	total: 25.6s	remaining: 1.81s
934:	total: 25.7s	remaining: 1.78s
935:	total: 25.7s	remaining: 1.76s
936:	total: 25.7s	remaining: 1.73s
937:	total: 25.8s	remaining: 1.7s
938:	total: 25.8s	remaining: 1.68s
939:	total: 25.8s	remaining: 1.65s
940:	total: 25.9s	remaining: 1.62s
941:	total: 25.9s	remaining: 1.59s
942:	total: 25.9s	remaining: 1.57s
943:	total: 25.9s	remaining: 1.54s
944:	total: 26s	remaining: 1.51s
945:	total: 26s	remaining: 1.48s
946:	total: 26s	remaining: 1.46s
947:	total: 26.1s	remaining: 1.43s
948:	total: 26.1s	remaining: 1.4s
949:	total: 26.1s	remaining: 1.37s
950:	total: 26.1s	remaining: 1.35s
951:	total: 26.2s	remaining: 1.32s
952:	total: 26.2s	remaining: 1.29s
953:	total: 26.2s	remaining: 1.26s
954:	total: 26.3s	remaining: 1.24s
955:	total: 26.3s	remaining: 1.21s
956:	total: 26.3s	remaining: 1.18s
957:	total: 26.3s	remaining: 1.15s
958:	total: 26.4s	remaining: 1.13s
959:	total: 26.4s	remaining: 1.1s
960:	total: 26.4s	remaining: 1.07s
961:	total: 26.5s	remaining: 1.04s
962:	total: 26.5s	remaining: 1.02s
963:	total: 26.5s	remaining: 990ms
964:	total: 26.5s	remaining: 963ms

```
965: total: 26.6s    remaining: 935ms
966: total: 26.6s    remaining: 908ms
967: total: 26.6s    remaining: 880ms
968: total: 26.7s    remaining: 853ms
969: total: 26.7s    remaining: 825ms
970: total: 26.7s    remaining: 798ms
971: total: 26.7s    remaining: 771ms
972: total: 26.8s    remaining: 743ms
973: total: 26.8s    remaining: 716ms
974: total: 26.8s    remaining: 688ms
975: total: 26.9s    remaining: 661ms
976: total: 26.9s    remaining: 634ms
977: total: 26.9s    remaining: 606ms
978: total: 27s      remaining: 579ms
979: total: 27s      remaining: 551ms
980: total: 27s      remaining: 524ms
981: total: 27.1s    remaining: 496ms
982: total: 27.1s    remaining: 469ms
983: total: 27.1s    remaining: 441ms
984: total: 27.2s    remaining: 413ms
985: total: 27.2s    remaining: 386ms
986: total: 27.2s    remaining: 358ms
987: total: 27.2s    remaining: 331ms
988: total: 27.3s    remaining: 303ms
989: total: 27.3s    remaining: 276ms
990: total: 27.3s    remaining: 248ms
991: total: 27.4s    remaining: 221ms
992: total: 27.4s    remaining: 193ms
993: total: 27.4s    remaining: 166ms
994: total: 27.5s    remaining: 138ms
995: total: 27.5s    remaining: 110ms
996: total: 27.5s    remaining: 82.8ms
997: total: 27.5s    remaining: 55.2ms
998: total: 27.6s    remaining: 27.6ms
999: total: 27.6s    remaining: 0us
```

```
<catboost.core.CatBoostClassifier at 0x274d5aa0a40>
```

```
y_pred_train_cb = cb.predict_proba(X_train)[: , 1]
y_pred_test_cb = cb.predict_proba(X_test)[: , 1]
```

```
roc_auc_cb = roc_auc_score(y_train, y_pred_train_cb)
```

```
print(f'Train ROC-AUC: {roc_auc_cb:.4f}')
```

```
# Train ROC-AUC: 0.8937
```

```
Train ROC-AUC: 0.8937
```

```
# submission['Churn'] = y_pred_test_cb
# submission.to_csv(root + 'my_submission_cb.csv', index=False)
```

ROC-AUC вашего решения равен 0.8514413599198526.

Уже лучше Логистической регрессии. Даже со стандартными гиперпараметрами.

Попробуем через кросс-валидацию.

```
X_train_cb, X_val_cb, y_train_cb, y_val_cb = train_test_split(X_train,
y_train, test_size=0.2,
random_state=RANDOM_STATE, stratify=y_train)

train_pool = Pool(data=X_train_cb,
                  label=y_train_cb,
                  cat_features=cat_features
                  )

val_pool = Pool(data=X_val_cb,
                label=y_val_cb,
                cat_features=cat_features
                )

cb_cv = CatBoostClassifier(
    iterations=2000,
    cat_features=cat_features,
    random_seed=RANDOM_STATE,
    eval_metric='AUC',
    verbose=50,
    early_stopping_rounds=100
)

cb_cv.fit(
    train_pool,
    eval_set=val_pool,
    verbose=50,
    plot=False
)
```

Learning rate set to 0.033428

```
0:   test: 0.8186230 best: 0.8186230 (0)   total: 31.7ms   remaining:
1m 3s
50:  test: 0.8495253 best: 0.8497082 (49) total: 1.45s   remaining:
55.3s
100: test: 0.8521921 best: 0.8521921 (100) total: 2.67s   remaining:
50.3s
150: test: 0.8532453 best: 0.8532453 (150) total: 3.87s   remaining:
47.4s
```

```
200: test: 0.8534939 best: 0.8538950 (178) total: 5.13s remaining:
45.9s
250: test: 0.8541037 best: 0.8541647 (238) total: 6.56s remaining:
45.7s
300: test: 0.8539489 best: 0.8542304 (256) total: 7.94s remaining:
44.8s
350: test: 0.8541694 best: 0.8543946 (324) total: 9.4s remaining:
44.2s
400: test: 0.8535080 best: 0.8543946 (324) total: 10.8s remaining:
43.1s
Stopped by overfitting detector (100 iterations wait)
```

```
bestTest = 0.8543945734
bestIteration = 324
```

Shrink model to first 325 iterations.

```
<catboost.core.CatBoostClassifier at 0x2728000cd40>
```

```
best_iterations = cb_cv.get_best_iteration()
print(best_iterations)
```

```
324
```

```
train_pool = Pool(data=X_train,
                  label=y_train,
                  cat_features=cat_features
                  )
```

```
test_pool = Pool(data=X_test,
                 cat_features=cat_features
                 )
```

```
cb_cv_final = CatBoostClassifier(iterations=best_iterations,
                                cat_features=cat_features,
                                random_seed=RANDOM_STATE,
                                eval_metric='AUC',
                                verbose=50,
                                )
```

```
cb_cv_final.fit(train_pool)
```

Learning rate set to 0.058834

```
0: total: 47.9ms remaining: 15.5s
50: total: 1.38s remaining: 7.39s
100: total: 2.65s remaining: 5.85s
150: total: 4.09s remaining: 4.69s
200: total: 5.5s remaining: 3.36s
250: total: 6.96s remaining: 2.02s
300: total: 8.08s remaining: 617ms
323: total: 8.49s remaining: 0us
```

```

<catboost.core.CatBoostClassifier at 0x274c6d05a60>
y_pred_train_cb_cv_final = cb_cv_final.predict_proba(X_train)[: , 1]
y_pred_test_cb_cv_final = cb_cv_final.predict_proba(X_test)[: , 1]

roc_auc_cb_cv_final = roc_auc_score(y_train, y_pred_train_cb_cv_final)

print(f'Train ROC-AUC: {roc_auc_cb_cv_final:.4f}')
# Train ROC-AUC: 0.8937
Train ROC-AUC: 0.8907

# submission['Churn'] = y_pred_test_cb_cv_final
# submission.to_csv(root + 'my_submission_cb_cv.csv', index=False)

```

optuna Подберем гиперпараметры для Catboost

```

X_train = train_generated[num_features + cat_features].copy()
y_train = train_generated['Churn'].copy()
X_test = test_generated[num_features + cat_features].copy()

# Сначала модель с подобранными гиперпараметрами сильно переобучилась
Train ROC-AUC: 0.9794

def objective_cb(trial):
    params = {
        'iterations': trial.suggest_int('iterations', 300,
1200),
        'learning_rate': trial.suggest_float('learning_rate',
0.003, 0.05, log=True),
        'depth': trial.suggest_int('depth', 2, 5),
        'l2_leaf_reg': trial.suggest_float('l2_leaf_reg', 5,
20),
        'random_strength': trial.suggest_float('random_strength',
10, 30),
        'bagging_temperature':
trial.suggest_float('bagging_temperature', 0.5, 2.0),
        'min_data_in_leaf': trial.suggest_int('min_data_in_leaf',
10, 50),

        # Борьба с переобучением
        'subsample': trial.suggest_float('subsample', 0.4,
0.7),
        # Используем часть данных
        'colsample_bylevel':
trial.suggest_float('colsample_bylevel', 0.3, 0.6),
        # Используем
часть признаков
        'max_bin': trial.suggest_int('max_bin', 16, 64),
        # Уменьшаем количество бинов

```

```

        'cat_features':      cat_features,
        'random_seed':      RANDOM_STATE,
        'eval_metric':      'AUC',
        'verbose':          False,
        'thread_count':     -1,
    }

    X_train_cb, X_val_cb, y_train_cb, y_val_cb =
train_test_split(X_train, y_train, test_size=0.25,
random_state=RANDOM_STATE, stratify=y_train)

    train_pool = Pool(data=X_train_cb, label=y_train_cb,
cat_features=cat_features)
    val_pool = Pool(data=X_val_cb, label=y_val_cb,
cat_features=cat_features)

    model = CatBoostClassifier(**params)

    model.fit(train_pool,
              eval_set=val_pool,
              verbose=False,
              early_stopping_rounds=50
              )

    best_score = model.get_best_score()['validation']['AUC']

    return best_score

# study_cb = optuna.create_study(direction='maximize')
# study_cb.optimize(objective_cb, n_trials=1000, n_jobs=-1,
# show_progress_bar=True)

# best_params_cb = study_cb.best_params
# best_params_cb

# # {'iterations': 1177,
# #   'learning_rate': 0.04352833222664741,
# #   'depth': 2,
# #   'l2_leaf_reg': 8.712627216223945,
# #   'random_strength': 18.956025845929155,
# #   'bagging_temperature': 1.1644875612616532,
# #   'min_data_in_leaf': 40,
# #   'subsample': 0.5472929227565183,
# #   'colsample_bylevel': 0.4608651413955162,
# #   'max_bin': 16}

best_params_cb = {'iterations': 1177,
                  'learning_rate': 0.04352833222664741,
                  'depth': 2,

```

```
'l2_leaf_reg': 8.712627216223945,  
'random_strength': 18.956025845929155,  
'bagging_temperature': 1.1644875612616532,  
'min_data_in_leaf': 40,  
'subsample': 0.5472929227565183,  
'colsample_bylevel': 0.4608651413955162,  
'max_bin': 16}
```

```
train_pool_full = Pool(data=X_train,  
                        label=y_train,  
                        cat_features=cat_features  
                        )
```

```
test_pool = Pool(data=X_test,  
                  cat_features=cat_features  
                  )
```

```
cb_optuna = CatBoostClassifier(**best_params_cb)
```

```
cb_optuna.fit(train_pool_full)
```

0:	learn: 0.6742091	total: 4.18ms	remaining: 4.92s
1:	learn: 0.6647128	total: 8.29ms	remaining: 4.87s
2:	learn: 0.6438467	total: 14.4ms	remaining: 5.62s
3:	learn: 0.6362512	total: 18.1ms	remaining: 5.32s
4:	learn: 0.6258497	total: 24.4ms	remaining: 5.72s
5:	learn: 0.6110550	total: 29.6ms	remaining: 5.78s
6:	learn: 0.6018711	total: 36.2ms	remaining: 6.05s
7:	learn: 0.5921386	total: 42.6ms	remaining: 6.23s
8:	learn: 0.5870853	total: 47.2ms	remaining: 6.13s
9:	learn: 0.5800474	total: 53.5ms	remaining: 6.25s
10:	learn: 0.5732939	total: 59.1ms	remaining: 6.26s
11:	learn: 0.5664581	total: 65.5ms	remaining: 6.36s
12:	learn: 0.5550311	total: 71.1ms	remaining: 6.37s
13:	learn: 0.5524676	total: 76.1ms	remaining: 6.32s
14:	learn: 0.5467008	total: 82.2ms	remaining: 6.36s
15:	learn: 0.5446059	total: 85.2ms	remaining: 6.18s
16:	learn: 0.5426963	total: 88.1ms	remaining: 6.01s
17:	learn: 0.5386473	total: 93.1ms	remaining: 6s
18:	learn: 0.5305732	total: 98.8ms	remaining: 6.02s
19:	learn: 0.5256423	total: 104ms	remaining: 6.03s
20:	learn: 0.5244454	total: 107ms	remaining: 5.91s
21:	learn: 0.5214946	total: 113ms	remaining: 5.94s
22:	learn: 0.5189454	total: 118ms	remaining: 5.93s
23:	learn: 0.5171144	total: 123ms	remaining: 5.93s
24:	learn: 0.5128517	total: 130ms	remaining: 6.01s
25:	learn: 0.5121525	total: 134ms	remaining: 5.94s
26:	learn: 0.5079165	total: 140ms	remaining: 5.98s
27:	learn: 0.5043960	total: 147ms	remaining: 6.04s
28:	learn: 0.5039004	total: 151ms	remaining: 5.98s

29:	learn: 0.5010321	total: 158ms	remaining: 6.04s
30:	learn: 0.4957635	total: 165ms	remaining: 6.08s
31:	learn: 0.4906693	total: 170ms	remaining: 6.1s
32:	learn: 0.4879573	total: 176ms	remaining: 6.09s
33:	learn: 0.4877122	total: 179ms	remaining: 6.02s
34:	learn: 0.4850067	total: 184ms	remaining: 6.02s
35:	learn: 0.4826697	total: 190ms	remaining: 6.02s
36:	learn: 0.4797344	total: 196ms	remaining: 6.05s
37:	learn: 0.4789033	total: 202ms	remaining: 6.05s
38:	learn: 0.4756016	total: 208ms	remaining: 6.06s
39:	learn: 0.4742755	total: 213ms	remaining: 6.05s
40:	learn: 0.4736126	total: 218ms	remaining: 6.05s
41:	learn: 0.4727380	total: 223ms	remaining: 6.03s
42:	learn: 0.4710713	total: 229ms	remaining: 6.04s
43:	learn: 0.4698417	total: 235ms	remaining: 6.04s
44:	learn: 0.4688180	total: 239ms	remaining: 6.02s
45:	learn: 0.4664769	total: 245ms	remaining: 6.02s
46:	learn: 0.4655979	total: 250ms	remaining: 6.02s
47:	learn: 0.4653001	total: 255ms	remaining: 6.01s
48:	learn: 0.4640193	total: 261ms	remaining: 6.02s
49:	learn: 0.4632642	total: 267ms	remaining: 6.03s
50:	learn: 0.4607806	total: 272ms	remaining: 6.01s
51:	learn: 0.4607608	total: 276ms	remaining: 5.98s
52:	learn: 0.4597547	total: 283ms	remaining: 5.99s
53:	learn: 0.4593917	total: 288ms	remaining: 6s
54:	learn: 0.4583055	total: 294ms	remaining: 6s
55:	learn: 0.4568631	total: 300ms	remaining: 6s
56:	learn: 0.4560159	total: 306ms	remaining: 6.02s
57:	learn: 0.4553278	total: 313ms	remaining: 6.03s
58:	learn: 0.4545824	total: 319ms	remaining: 6.04s
59:	learn: 0.4539551	total: 324ms	remaining: 6.04s
60:	learn: 0.4535144	total: 331ms	remaining: 6.05s
61:	learn: 0.4525355	total: 336ms	remaining: 6.04s
62:	learn: 0.4514911	total: 341ms	remaining: 6.04s
63:	learn: 0.4505777	total: 347ms	remaining: 6.03s
64:	learn: 0.4505728	total: 352ms	remaining: 6.01s
65:	learn: 0.4504946	total: 357ms	remaining: 6.01s
66:	learn: 0.4500157	total: 362ms	remaining: 6s
67:	learn: 0.4497106	total: 367ms	remaining: 5.99s
68:	learn: 0.4495127	total: 372ms	remaining: 5.98s
69:	learn: 0.4485730	total: 378ms	remaining: 5.98s
70:	learn: 0.4485720	total: 381ms	remaining: 5.94s
71:	learn: 0.4484114	total: 387ms	remaining: 5.93s
72:	learn: 0.4467928	total: 392ms	remaining: 5.93s
73:	learn: 0.4462488	total: 397ms	remaining: 5.91s
74:	learn: 0.4461343	total: 402ms	remaining: 5.9s
75:	learn: 0.4461340	total: 405ms	remaining: 5.86s
76:	learn: 0.4461338	total: 408ms	remaining: 5.83s
77:	learn: 0.4459989	total: 413ms	remaining: 5.83s

78:	learn: 0.4448807	total: 419ms	remaining: 5.82s
79:	learn: 0.4445380	total: 424ms	remaining: 5.81s
80:	learn: 0.4445380	total: 427ms	remaining: 5.77s
81:	learn: 0.4443716	total: 432ms	remaining: 5.76s
82:	learn: 0.4443715	total: 435ms	remaining: 5.73s
83:	learn: 0.4443458	total: 440ms	remaining: 5.73s
84:	learn: 0.4436196	total: 445ms	remaining: 5.72s
85:	learn: 0.4430712	total: 451ms	remaining: 5.72s
86:	learn: 0.4429888	total: 457ms	remaining: 5.72s
87:	learn: 0.4419524	total: 463ms	remaining: 5.72s
88:	learn: 0.4413218	total: 468ms	remaining: 5.72s
89:	learn: 0.4413196	total: 473ms	remaining: 5.71s
90:	learn: 0.4413195	total: 477ms	remaining: 5.69s
91:	learn: 0.4412438	total: 483ms	remaining: 5.69s
92:	learn: 0.4404328	total: 489ms	remaining: 5.7s
93:	learn: 0.4403741	total: 495ms	remaining: 5.71s
94:	learn: 0.4396500	total: 502ms	remaining: 5.72s
95:	learn: 0.4391190	total: 507ms	remaining: 5.71s
96:	learn: 0.4387779	total: 512ms	remaining: 5.7s
97:	learn: 0.4382988	total: 517ms	remaining: 5.69s
98:	learn: 0.4377954	total: 521ms	remaining: 5.67s
99:	learn: 0.4377844	total: 527ms	remaining: 5.67s
100:	learn: 0.4377843	total: 530ms	remaining: 5.65s
101:	learn: 0.4369642	total: 535ms	remaining: 5.64s
102:	learn: 0.4369030	total: 541ms	remaining: 5.64s
103:	learn: 0.4365343	total: 545ms	remaining: 5.63s
104:	learn: 0.4365341	total: 548ms	remaining: 5.6s
105:	learn: 0.4362549	total: 553ms	remaining: 5.59s
106:	learn: 0.4362044	total: 559ms	remaining: 5.59s
107:	learn: 0.4358196	total: 564ms	remaining: 5.59s
108:	learn: 0.4351668	total: 570ms	remaining: 5.58s
109:	learn: 0.4351666	total: 573ms	remaining: 5.56s
110:	learn: 0.4350705	total: 579ms	remaining: 5.56s
111:	learn: 0.4348940	total: 585ms	remaining: 5.56s
112:	learn: 0.4348533	total: 591ms	remaining: 5.57s
113:	learn: 0.4345014	total: 597ms	remaining: 5.56s
114:	learn: 0.4341519	total: 602ms	remaining: 5.56s
115:	learn: 0.4337921	total: 608ms	remaining: 5.56s
116:	learn: 0.4337919	total: 611ms	remaining: 5.54s
117:	learn: 0.4333419	total: 617ms	remaining: 5.53s
118:	learn: 0.4333354	total: 623ms	remaining: 5.54s
119:	learn: 0.4330774	total: 628ms	remaining: 5.53s
120:	learn: 0.4330772	total: 631ms	remaining: 5.51s
121:	learn: 0.4325557	total: 637ms	remaining: 5.51s
122:	learn: 0.4322041	total: 643ms	remaining: 5.51s
123:	learn: 0.4322019	total: 649ms	remaining: 5.51s
124:	learn: 0.4321963	total: 655ms	remaining: 5.51s
125:	learn: 0.4321946	total: 660ms	remaining: 5.5s
126:	learn: 0.4316797	total: 665ms	remaining: 5.5s

127:	learn:	0.4316777	total:	671ms	remaining:	5.5s
128:	learn:	0.4316775	total:	674ms	remaining:	5.48s
129:	learn:	0.4313780	total:	680ms	remaining:	5.47s
130:	learn:	0.4310875	total:	686ms	remaining:	5.48s
131:	learn:	0.4305119	total:	692ms	remaining:	5.47s
132:	learn:	0.4305117	total:	695ms	remaining:	5.46s
133:	learn:	0.4305077	total:	701ms	remaining:	5.45s
134:	learn:	0.4305076	total:	704ms	remaining:	5.43s
135:	learn:	0.4304937	total:	710ms	remaining:	5.43s
136:	learn:	0.4303254	total:	715ms	remaining:	5.43s
137:	learn:	0.4303241	total:	720ms	remaining:	5.42s
138:	learn:	0.4300085	total:	725ms	remaining:	5.42s
139:	learn:	0.4300084	total:	728ms	remaining:	5.39s
140:	learn:	0.4297662	total:	734ms	remaining:	5.39s
141:	learn:	0.4295050	total:	739ms	remaining:	5.38s
142:	learn:	0.4295049	total:	742ms	remaining:	5.36s
143:	learn:	0.4290853	total:	747ms	remaining:	5.36s
144:	learn:	0.4289121	total:	751ms	remaining:	5.35s
145:	learn:	0.4289119	total:	754ms	remaining:	5.32s
146:	learn:	0.4285250	total:	759ms	remaining:	5.32s
147:	learn:	0.4281540	total:	763ms	remaining:	5.3s
148:	learn:	0.4278231	total:	767ms	remaining:	5.29s
149:	learn:	0.4277302	total:	773ms	remaining:	5.29s
150:	learn:	0.4273158	total:	778ms	remaining:	5.29s
151:	learn:	0.4270725	total:	783ms	remaining:	5.28s
152:	learn:	0.4270163	total:	788ms	remaining:	5.27s
153:	learn:	0.4270136	total:	793ms	remaining:	5.27s
154:	learn:	0.4270011	total:	798ms	remaining:	5.26s
155:	learn:	0.4268037	total:	803ms	remaining:	5.26s
156:	learn:	0.4263962	total:	808ms	remaining:	5.25s
157:	learn:	0.4261267	total:	812ms	remaining:	5.24s
158:	learn:	0.4260035	total:	817ms	remaining:	5.23s
159:	learn:	0.4258668	total:	822ms	remaining:	5.22s
160:	learn:	0.4256990	total:	827ms	remaining:	5.22s
161:	learn:	0.4255368	total:	832ms	remaining:	5.21s
162:	learn:	0.4250429	total:	837ms	remaining:	5.21s
163:	learn:	0.4248612	total:	843ms	remaining:	5.2s
164:	learn:	0.4246348	total:	847ms	remaining:	5.2s
165:	learn:	0.4245150	total:	852ms	remaining:	5.19s
166:	learn:	0.4240538	total:	856ms	remaining:	5.18s
167:	learn:	0.4239029	total:	861ms	remaining:	5.17s
168:	learn:	0.4238828	total:	865ms	remaining:	5.16s
169:	learn:	0.4238732	total:	870ms	remaining:	5.16s
170:	learn:	0.4237063	total:	876ms	remaining:	5.15s
171:	learn:	0.4233300	total:	880ms	remaining:	5.14s
172:	learn:	0.4232178	total:	885ms	remaining:	5.14s
173:	learn:	0.4230778	total:	890ms	remaining:	5.13s
174:	learn:	0.4230757	total:	895ms	remaining:	5.12s
175:	learn:	0.4228758	total:	900ms	remaining:	5.12s

176:	learn:	0.4228757	total:	903ms	remaining:	5.1s
177:	learn:	0.4227605	total:	907ms	remaining:	5.09s
178:	learn:	0.4227592	total:	912ms	remaining:	5.08s
179:	learn:	0.4225601	total:	916ms	remaining:	5.08s
180:	learn:	0.4225480	total:	921ms	remaining:	5.07s
181:	learn:	0.4225427	total:	926ms	remaining:	5.06s
182:	learn:	0.4224496	total:	930ms	remaining:	5.05s
183:	learn:	0.4222983	total:	935ms	remaining:	5.05s
184:	learn:	0.4222982	total:	938ms	remaining:	5.03s
185:	learn:	0.4222981	total:	941ms	remaining:	5.01s
186:	learn:	0.4222981	total:	944ms	remaining:	5s
187:	learn:	0.4220215	total:	949ms	remaining:	4.99s
188:	learn:	0.4216219	total:	955ms	remaining:	4.99s
189:	learn:	0.4215826	total:	960ms	remaining:	4.99s
190:	learn:	0.4215071	total:	965ms	remaining:	4.98s
191:	learn:	0.4214294	total:	970ms	remaining:	4.98s
192:	learn:	0.4211971	total:	975ms	remaining:	4.97s
193:	learn:	0.4210446	total:	981ms	remaining:	4.97s
194:	learn:	0.4209343	total:	986ms	remaining:	4.97s
195:	learn:	0.4209342	total:	989ms	remaining:	4.95s
196:	learn:	0.4209128	total:	995ms	remaining:	4.95s
197:	learn:	0.4209110	total:	1s	remaining:	4.95s
198:	learn:	0.4209039	total:	1.01s	remaining:	4.95s
199:	learn:	0.4207751	total:	1.01s	remaining:	4.95s
200:	learn:	0.4207750	total:	1.02s	remaining:	4.93s
201:	learn:	0.4207750	total:	1.02s	remaining:	4.92s
202:	learn:	0.4205431	total:	1.02s	remaining:	4.91s
203:	learn:	0.4204685	total:	1.03s	remaining:	4.91s
204:	learn:	0.4204082	total:	1.03s	remaining:	4.91s
205:	learn:	0.4203431	total:	1.04s	remaining:	4.91s
206:	learn:	0.4203414	total:	1.05s	remaining:	4.9s
207:	learn:	0.4199192	total:	1.05s	remaining:	4.9s
208:	learn:	0.4197690	total:	1.06s	remaining:	4.9s
209:	learn:	0.4196725	total:	1.06s	remaining:	4.89s
210:	learn:	0.4195631	total:	1.07s	remaining:	4.89s
211:	learn:	0.4195482	total:	1.07s	remaining:	4.88s
212:	learn:	0.4195351	total:	1.08s	remaining:	4.88s
213:	learn:	0.4193991	total:	1.08s	remaining:	4.87s
214:	learn:	0.4193319	total:	1.09s	remaining:	4.86s
215:	learn:	0.4192098	total:	1.09s	remaining:	4.86s
216:	learn:	0.4188843	total:	1.1s	remaining:	4.85s
217:	learn:	0.4187821	total:	1.1s	remaining:	4.84s
218:	learn:	0.4187820	total:	1.11s	remaining:	4.84s
219:	learn:	0.4186756	total:	1.11s	remaining:	4.83s
220:	learn:	0.4186645	total:	1.11s	remaining:	4.83s
221:	learn:	0.4184703	total:	1.12s	remaining:	4.82s
222:	learn:	0.4184069	total:	1.12s	remaining:	4.81s
223:	learn:	0.4184030	total:	1.13s	remaining:	4.8s
224:	learn:	0.4181655	total:	1.13s	remaining:	4.8s

225:	learn:	0.4181654	total:	1.14s	remaining:	4.78s
226:	learn:	0.4180278	total:	1.14s	remaining:	4.77s
227:	learn:	0.4180277	total:	1.14s	remaining:	4.76s
228:	learn:	0.4178775	total:	1.15s	remaining:	4.75s
229:	learn:	0.4177097	total:	1.15s	remaining:	4.75s
230:	learn:	0.4176472	total:	1.16s	remaining:	4.74s
231:	learn:	0.4173294	total:	1.16s	remaining:	4.73s
232:	learn:	0.4172969	total:	1.17s	remaining:	4.73s
233:	learn:	0.4171761	total:	1.17s	remaining:	4.73s
234:	learn:	0.4170368	total:	1.18s	remaining:	4.72s
235:	learn:	0.4169006	total:	1.18s	remaining:	4.71s
236:	learn:	0.4166760	total:	1.19s	remaining:	4.71s
237:	learn:	0.4165911	total:	1.19s	remaining:	4.7s
238:	learn:	0.4165567	total:	1.2s	remaining:	4.7s
239:	learn:	0.4164851	total:	1.2s	remaining:	4.69s
240:	learn:	0.4161931	total:	1.21s	remaining:	4.68s
241:	learn:	0.4161146	total:	1.21s	remaining:	4.68s
242:	learn:	0.4158418	total:	1.22s	remaining:	4.68s
243:	learn:	0.4154860	total:	1.22s	remaining:	4.68s
244:	learn:	0.4150711	total:	1.23s	remaining:	4.67s
245:	learn:	0.4149721	total:	1.23s	remaining:	4.67s
246:	learn:	0.4149012	total:	1.24s	remaining:	4.67s
247:	learn:	0.4147348	total:	1.25s	remaining:	4.66s
248:	learn:	0.4144908	total:	1.25s	remaining:	4.66s
249:	learn:	0.4142125	total:	1.26s	remaining:	4.66s
250:	learn:	0.4140896	total:	1.26s	remaining:	4.65s
251:	learn:	0.4139992	total:	1.27s	remaining:	4.65s
252:	learn:	0.4139117	total:	1.27s	remaining:	4.65s
253:	learn:	0.4137547	total:	1.28s	remaining:	4.64s
254:	learn:	0.4136270	total:	1.28s	remaining:	4.64s
255:	learn:	0.4135562	total:	1.29s	remaining:	4.64s
256:	learn:	0.4134512	total:	1.29s	remaining:	4.63s
257:	learn:	0.4134351	total:	1.3s	remaining:	4.63s
258:	learn:	0.4132313	total:	1.3s	remaining:	4.63s
259:	learn:	0.4129641	total:	1.31s	remaining:	4.63s
260:	learn:	0.4127829	total:	1.32s	remaining:	4.62s
261:	learn:	0.4126362	total:	1.32s	remaining:	4.62s
262:	learn:	0.4123269	total:	1.33s	remaining:	4.62s
263:	learn:	0.4121213	total:	1.33s	remaining:	4.61s
264:	learn:	0.4120253	total:	1.34s	remaining:	4.61s
265:	learn:	0.4119422	total:	1.34s	remaining:	4.61s
266:	learn:	0.4117549	total:	1.35s	remaining:	4.6s
267:	learn:	0.4116334	total:	1.35s	remaining:	4.6s
268:	learn:	0.4114683	total:	1.36s	remaining:	4.59s
269:	learn:	0.4112487	total:	1.37s	remaining:	4.59s
270:	learn:	0.4111977	total:	1.37s	remaining:	4.58s
271:	learn:	0.4110286	total:	1.38s	remaining:	4.58s
272:	learn:	0.4108970	total:	1.38s	remaining:	4.57s
273:	learn:	0.4106941	total:	1.39s	remaining:	4.57s

274:	learn:	0.4106056	total:	1.39s	remaining:	4.56s
275:	learn:	0.4104923	total:	1.4s	remaining:	4.56s
276:	learn:	0.4103330	total:	1.4s	remaining:	4.55s
277:	learn:	0.4102165	total:	1.41s	remaining:	4.54s
278:	learn:	0.4101017	total:	1.41s	remaining:	4.54s
279:	learn:	0.4099176	total:	1.42s	remaining:	4.54s
280:	learn:	0.4098461	total:	1.42s	remaining:	4.53s
281:	learn:	0.4096206	total:	1.43s	remaining:	4.52s
282:	learn:	0.4095033	total:	1.43s	remaining:	4.52s
283:	learn:	0.4093049	total:	1.43s	remaining:	4.51s
284:	learn:	0.4091245	total:	1.44s	remaining:	4.51s
285:	learn:	0.4091096	total:	1.45s	remaining:	4.5s
286:	learn:	0.4090060	total:	1.45s	remaining:	4.5s
287:	learn:	0.4089562	total:	1.46s	remaining:	4.5s
288:	learn:	0.4087933	total:	1.46s	remaining:	4.49s
289:	learn:	0.4086961	total:	1.47s	remaining:	4.48s
290:	learn:	0.4086649	total:	1.47s	remaining:	4.48s
291:	learn:	0.4085817	total:	1.48s	remaining:	4.48s
292:	learn:	0.4085022	total:	1.48s	remaining:	4.47s
293:	learn:	0.4083537	total:	1.49s	remaining:	4.46s
294:	learn:	0.4082615	total:	1.49s	remaining:	4.46s
295:	learn:	0.4081641	total:	1.5s	remaining:	4.45s
296:	learn:	0.4081379	total:	1.5s	remaining:	4.45s
297:	learn:	0.4080488	total:	1.5s	remaining:	4.44s
298:	learn:	0.4079071	total:	1.51s	remaining:	4.44s
299:	learn:	0.4077720	total:	1.51s	remaining:	4.43s
300:	learn:	0.4076916	total:	1.52s	remaining:	4.42s
301:	learn:	0.4075393	total:	1.52s	remaining:	4.42s
302:	learn:	0.4073643	total:	1.53s	remaining:	4.41s
303:	learn:	0.4072863	total:	1.53s	remaining:	4.41s
304:	learn:	0.4072378	total:	1.54s	remaining:	4.4s
305:	learn:	0.4071112	total:	1.54s	remaining:	4.4s
306:	learn:	0.4070663	total:	1.55s	remaining:	4.39s
307:	learn:	0.4069609	total:	1.55s	remaining:	4.39s
308:	learn:	0.4068481	total:	1.56s	remaining:	4.38s
309:	learn:	0.4068385	total:	1.57s	remaining:	4.38s
310:	learn:	0.4068015	total:	1.57s	remaining:	4.38s
311:	learn:	0.4066894	total:	1.58s	remaining:	4.37s
312:	learn:	0.4065847	total:	1.58s	remaining:	4.37s
313:	learn:	0.4065395	total:	1.59s	remaining:	4.36s
314:	learn:	0.4064405	total:	1.59s	remaining:	4.36s
315:	learn:	0.4064348	total:	1.6s	remaining:	4.35s
316:	learn:	0.4064099	total:	1.6s	remaining:	4.34s
317:	learn:	0.4062977	total:	1.6s	remaining:	4.34s
318:	learn:	0.4062293	total:	1.61s	remaining:	4.33s
319:	learn:	0.4061709	total:	1.61s	remaining:	4.32s
320:	learn:	0.4060859	total:	1.62s	remaining:	4.32s
321:	learn:	0.4060061	total:	1.62s	remaining:	4.31s
322:	learn:	0.4059624	total:	1.63s	remaining:	4.3s

323:	learn:	0.4058972	total:	1.63s	remaining:	4.3s
324:	learn:	0.4058794	total:	1.64s	remaining:	4.29s
325:	learn:	0.4058056	total:	1.64s	remaining:	4.29s
326:	learn:	0.4057278	total:	1.65s	remaining:	4.28s
327:	learn:	0.4057260	total:	1.65s	remaining:	4.28s
328:	learn:	0.4056330	total:	1.66s	remaining:	4.27s
329:	learn:	0.4056216	total:	1.66s	remaining:	4.27s
330:	learn:	0.4055374	total:	1.67s	remaining:	4.26s
331:	learn:	0.4054537	total:	1.67s	remaining:	4.25s
332:	learn:	0.4054299	total:	1.68s	remaining:	4.25s
333:	learn:	0.4053491	total:	1.68s	remaining:	4.24s
334:	learn:	0.4052642	total:	1.69s	remaining:	4.24s
335:	learn:	0.4052411	total:	1.69s	remaining:	4.23s
336:	learn:	0.4051957	total:	1.7s	remaining:	4.22s
337:	learn:	0.4051187	total:	1.7s	remaining:	4.22s
338:	learn:	0.4050610	total:	1.7s	remaining:	4.21s
339:	learn:	0.4050582	total:	1.71s	remaining:	4.21s
340:	learn:	0.4049869	total:	1.71s	remaining:	4.2s
341:	learn:	0.4049731	total:	1.72s	remaining:	4.2s
342:	learn:	0.4048922	total:	1.72s	remaining:	4.19s
343:	learn:	0.4048371	total:	1.73s	remaining:	4.18s
344:	learn:	0.4048095	total:	1.73s	remaining:	4.18s
345:	learn:	0.4047904	total:	1.74s	remaining:	4.17s
346:	learn:	0.4047171	total:	1.74s	remaining:	4.17s
347:	learn:	0.4046264	total:	1.75s	remaining:	4.16s
348:	learn:	0.4045530	total:	1.75s	remaining:	4.16s
349:	learn:	0.4044961	total:	1.76s	remaining:	4.15s
350:	learn:	0.4044806	total:	1.76s	remaining:	4.15s
351:	learn:	0.4044692	total:	1.77s	remaining:	4.14s
352:	learn:	0.4044327	total:	1.77s	remaining:	4.13s
353:	learn:	0.4043892	total:	1.77s	remaining:	4.13s
354:	learn:	0.4042804	total:	1.78s	remaining:	4.12s
355:	learn:	0.4042783	total:	1.78s	remaining:	4.12s
356:	learn:	0.4042216	total:	1.79s	remaining:	4.11s
357:	learn:	0.4041760	total:	1.79s	remaining:	4.11s
358:	learn:	0.4041696	total:	1.8s	remaining:	4.1s
359:	learn:	0.4040927	total:	1.8s	remaining:	4.09s
360:	learn:	0.4040605	total:	1.81s	remaining:	4.09s
361:	learn:	0.4040238	total:	1.81s	remaining:	4.08s
362:	learn:	0.4039937	total:	1.82s	remaining:	4.08s
363:	learn:	0.4039497	total:	1.82s	remaining:	4.07s
364:	learn:	0.4039123	total:	1.83s	remaining:	4.07s
365:	learn:	0.4038934	total:	1.83s	remaining:	4.06s
366:	learn:	0.4038889	total:	1.84s	remaining:	4.06s
367:	learn:	0.4038666	total:	1.84s	remaining:	4.05s
368:	learn:	0.4038403	total:	1.85s	remaining:	4.05s
369:	learn:	0.4037519	total:	1.85s	remaining:	4.04s
370:	learn:	0.4036868	total:	1.86s	remaining:	4.04s
371:	learn:	0.4036480	total:	1.86s	remaining:	4.03s

372:	learn:	0.4036356	total:	1.87s	remaining:	4.03s
373:	learn:	0.4035780	total:	1.87s	remaining:	4.02s
374:	learn:	0.4035523	total:	1.88s	remaining:	4.02s
375:	learn:	0.4035326	total:	1.88s	remaining:	4.01s
376:	learn:	0.4035112	total:	1.89s	remaining:	4.01s
377:	learn:	0.4034851	total:	1.89s	remaining:	4s
378:	learn:	0.4034627	total:	1.9s	remaining:	4s
379:	learn:	0.4034022	total:	1.91s	remaining:	4s
380:	learn:	0.4033387	total:	1.91s	remaining:	3.99s
381:	learn:	0.4032940	total:	1.92s	remaining:	3.99s
382:	learn:	0.4032541	total:	1.92s	remaining:	3.98s
383:	learn:	0.4032407	total:	1.93s	remaining:	3.98s
384:	learn:	0.4032311	total:	1.93s	remaining:	3.97s
385:	learn:	0.4032186	total:	1.94s	remaining:	3.97s
386:	learn:	0.4031906	total:	1.94s	remaining:	3.97s
387:	learn:	0.4031892	total:	1.95s	remaining:	3.96s
388:	learn:	0.4031760	total:	1.95s	remaining:	3.96s
389:	learn:	0.4030859	total:	1.96s	remaining:	3.95s
390:	learn:	0.4030568	total:	1.97s	remaining:	3.95s
391:	learn:	0.4030392	total:	1.97s	remaining:	3.94s
392:	learn:	0.4030301	total:	1.97s	remaining:	3.94s
393:	learn:	0.4029592	total:	1.98s	remaining:	3.93s
394:	learn:	0.4029432	total:	1.98s	remaining:	3.93s
395:	learn:	0.4029158	total:	1.99s	remaining:	3.92s
396:	learn:	0.4029091	total:	1.99s	remaining:	3.92s
397:	learn:	0.4029077	total:	2s	remaining:	3.91s
398:	learn:	0.4028855	total:	2s	remaining:	3.91s
399:	learn:	0.4028649	total:	2.01s	remaining:	3.9s
400:	learn:	0.4028292	total:	2.01s	remaining:	3.9s
401:	learn:	0.4027888	total:	2.02s	remaining:	3.89s
402:	learn:	0.4027640	total:	2.02s	remaining:	3.88s
403:	learn:	0.4027615	total:	2.03s	remaining:	3.88s
404:	learn:	0.4027477	total:	2.03s	remaining:	3.87s
405:	learn:	0.4027414	total:	2.04s	remaining:	3.87s
406:	learn:	0.4027309	total:	2.04s	remaining:	3.86s
407:	learn:	0.4027142	total:	2.05s	remaining:	3.86s
408:	learn:	0.4027014	total:	2.05s	remaining:	3.85s
409:	learn:	0.4026754	total:	2.06s	remaining:	3.85s
410:	learn:	0.4025960	total:	2.06s	remaining:	3.84s
411:	learn:	0.4025623	total:	2.07s	remaining:	3.84s
412:	learn:	0.4025568	total:	2.07s	remaining:	3.83s
413:	learn:	0.4025415	total:	2.08s	remaining:	3.83s
414:	learn:	0.4025310	total:	2.08s	remaining:	3.83s
415:	learn:	0.4025287	total:	2.09s	remaining:	3.82s
416:	learn:	0.4025025	total:	2.09s	remaining:	3.81s
417:	learn:	0.4025014	total:	2.1s	remaining:	3.81s
418:	learn:	0.4024627	total:	2.1s	remaining:	3.81s
419:	learn:	0.4024627	total:	2.11s	remaining:	3.8s
420:	learn:	0.4024280	total:	2.11s	remaining:	3.8s

421:	learn:	0.4024072	total:	2.12s	remaining:	3.79s
422:	learn:	0.4023288	total:	2.13s	remaining:	3.79s
423:	learn:	0.4023238	total:	2.13s	remaining:	3.78s
424:	learn:	0.4023042	total:	2.14s	remaining:	3.78s
425:	learn:	0.4022076	total:	2.14s	remaining:	3.78s
426:	learn:	0.4021502	total:	2.15s	remaining:	3.77s
427:	learn:	0.4021320	total:	2.15s	remaining:	3.77s
428:	learn:	0.4021266	total:	2.16s	remaining:	3.76s
429:	learn:	0.4020790	total:	2.16s	remaining:	3.76s
430:	learn:	0.4020714	total:	2.17s	remaining:	3.75s
431:	learn:	0.4020646	total:	2.17s	remaining:	3.75s
432:	learn:	0.4020556	total:	2.18s	remaining:	3.75s
433:	learn:	0.4020479	total:	2.19s	remaining:	3.74s
434:	learn:	0.4020219	total:	2.19s	remaining:	3.74s
435:	learn:	0.4020199	total:	2.2s	remaining:	3.73s
436:	learn:	0.4020182	total:	2.2s	remaining:	3.73s
437:	learn:	0.4019630	total:	2.21s	remaining:	3.72s
438:	learn:	0.4019576	total:	2.21s	remaining:	3.72s
439:	learn:	0.4019256	total:	2.22s	remaining:	3.72s
440:	learn:	0.4019034	total:	2.22s	remaining:	3.71s
441:	learn:	0.4018810	total:	2.23s	remaining:	3.71s
442:	learn:	0.4018659	total:	2.23s	remaining:	3.7s
443:	learn:	0.4018333	total:	2.24s	remaining:	3.7s
444:	learn:	0.4018279	total:	2.24s	remaining:	3.69s
445:	learn:	0.4018249	total:	2.25s	remaining:	3.69s
446:	learn:	0.4017914	total:	2.26s	remaining:	3.68s
447:	learn:	0.4016573	total:	2.26s	remaining:	3.68s
448:	learn:	0.4016439	total:	2.27s	remaining:	3.68s
449:	learn:	0.4016267	total:	2.27s	remaining:	3.67s
450:	learn:	0.4015923	total:	2.28s	remaining:	3.67s
451:	learn:	0.4015855	total:	2.29s	remaining:	3.67s
452:	learn:	0.4015278	total:	2.29s	remaining:	3.66s
453:	learn:	0.4014716	total:	2.3s	remaining:	3.66s
454:	learn:	0.4014556	total:	2.3s	remaining:	3.65s
455:	learn:	0.4014479	total:	2.31s	remaining:	3.65s
456:	learn:	0.4014419	total:	2.31s	remaining:	3.65s
457:	learn:	0.4013581	total:	2.32s	remaining:	3.64s
458:	learn:	0.4013406	total:	2.33s	remaining:	3.64s
459:	learn:	0.4013320	total:	2.33s	remaining:	3.63s
460:	learn:	0.4013096	total:	2.34s	remaining:	3.63s
461:	learn:	0.4012566	total:	2.34s	remaining:	3.62s
462:	learn:	0.4011813	total:	2.35s	remaining:	3.62s
463:	learn:	0.4011331	total:	2.35s	remaining:	3.62s
464:	learn:	0.4011320	total:	2.36s	remaining:	3.61s
465:	learn:	0.4011268	total:	2.37s	remaining:	3.61s
466:	learn:	0.4010817	total:	2.37s	remaining:	3.6s
467:	learn:	0.4010669	total:	2.38s	remaining:	3.6s
468:	learn:	0.4010308	total:	2.38s	remaining:	3.6s
469:	learn:	0.4010005	total:	2.39s	remaining:	3.59s

470:	learn: 0.4009496	total: 2.39s	remaining: 3.58s
471:	learn: 0.4009456	total: 2.4s	remaining: 3.58s
472:	learn: 0.4009367	total: 2.4s	remaining: 3.58s
473:	learn: 0.4009254	total: 2.41s	remaining: 3.57s
474:	learn: 0.4009102	total: 2.41s	remaining: 3.57s
475:	learn: 0.4008882	total: 2.42s	remaining: 3.56s
476:	learn: 0.4008836	total: 2.42s	remaining: 3.56s
477:	learn: 0.4008533	total: 2.43s	remaining: 3.55s
478:	learn: 0.4008533	total: 2.43s	remaining: 3.55s
479:	learn: 0.4008337	total: 2.44s	remaining: 3.54s
480:	learn: 0.4007884	total: 2.44s	remaining: 3.54s
481:	learn: 0.4007691	total: 2.45s	remaining: 3.53s
482:	learn: 0.4007616	total: 2.45s	remaining: 3.52s
483:	learn: 0.4007256	total: 2.46s	remaining: 3.52s
484:	learn: 0.4007152	total: 2.46s	remaining: 3.51s
485:	learn: 0.4006887	total: 2.47s	remaining: 3.51s
486:	learn: 0.4006620	total: 2.47s	remaining: 3.5s
487:	learn: 0.4006611	total: 2.48s	remaining: 3.5s
488:	learn: 0.4006531	total: 2.48s	remaining: 3.49s
489:	learn: 0.4006458	total: 2.49s	remaining: 3.49s
490:	learn: 0.4006438	total: 2.49s	remaining: 3.48s
491:	learn: 0.4005941	total: 2.5s	remaining: 3.48s
492:	learn: 0.4005809	total: 2.5s	remaining: 3.47s
493:	learn: 0.4005240	total: 2.51s	remaining: 3.47s
494:	learn: 0.4005161	total: 2.51s	remaining: 3.46s
495:	learn: 0.4005095	total: 2.52s	remaining: 3.46s
496:	learn: 0.4005043	total: 2.52s	remaining: 3.45s
497:	learn: 0.4004920	total: 2.53s	remaining: 3.45s
498:	learn: 0.4004913	total: 2.53s	remaining: 3.44s
499:	learn: 0.4004789	total: 2.54s	remaining: 3.44s
500:	learn: 0.4004668	total: 2.55s	remaining: 3.44s
501:	learn: 0.4004543	total: 2.55s	remaining: 3.43s
502:	learn: 0.4004543	total: 2.56s	remaining: 3.42s
503:	learn: 0.4004523	total: 2.56s	remaining: 3.42s
504:	learn: 0.4004499	total: 2.57s	remaining: 3.42s
505:	learn: 0.4004180	total: 2.57s	remaining: 3.41s
506:	learn: 0.4004050	total: 2.58s	remaining: 3.41s
507:	learn: 0.4004037	total: 2.58s	remaining: 3.4s
508:	learn: 0.4003662	total: 2.59s	remaining: 3.4s
509:	learn: 0.4003227	total: 2.6s	remaining: 3.4s
510:	learn: 0.4003112	total: 2.6s	remaining: 3.39s
511:	learn: 0.4003103	total: 2.61s	remaining: 3.39s
512:	learn: 0.4002469	total: 2.61s	remaining: 3.38s
513:	learn: 0.4002316	total: 2.62s	remaining: 3.38s
514:	learn: 0.4002030	total: 2.62s	remaining: 3.37s
515:	learn: 0.4001995	total: 2.63s	remaining: 3.37s
516:	learn: 0.4001698	total: 2.63s	remaining: 3.36s
517:	learn: 0.4001162	total: 2.64s	remaining: 3.36s
518:	learn: 0.4001025	total: 2.64s	remaining: 3.35s
519:	learn: 0.4000629	total: 2.65s	remaining: 3.35s

520:	learn: 0.4000584	total: 2.65s	remaining: 3.34s
521:	learn: 0.4000484	total: 2.66s	remaining: 3.34s
522:	learn: 0.4000434	total: 2.67s	remaining: 3.33s
523:	learn: 0.4000188	total: 2.67s	remaining: 3.33s
524:	learn: 0.3999898	total: 2.67s	remaining: 3.32s
525:	learn: 0.3999799	total: 2.68s	remaining: 3.32s
526:	learn: 0.3999732	total: 2.68s	remaining: 3.31s
527:	learn: 0.3999668	total: 2.69s	remaining: 3.31s
528:	learn: 0.3999668	total: 2.69s	remaining: 3.3s
529:	learn: 0.3999599	total: 2.7s	remaining: 3.3s
530:	learn: 0.3998968	total: 2.7s	remaining: 3.29s
531:	learn: 0.3998750	total: 2.71s	remaining: 3.29s
532:	learn: 0.3998662	total: 2.71s	remaining: 3.28s
533:	learn: 0.3998295	total: 2.72s	remaining: 3.27s
534:	learn: 0.3998176	total: 2.72s	remaining: 3.27s
535:	learn: 0.3997937	total: 2.73s	remaining: 3.26s
536:	learn: 0.3997937	total: 2.73s	remaining: 3.26s
537:	learn: 0.3997933	total: 2.74s	remaining: 3.25s
538:	learn: 0.3997883	total: 2.74s	remaining: 3.25s
539:	learn: 0.3997870	total: 2.75s	remaining: 3.24s
540:	learn: 0.3997711	total: 2.75s	remaining: 3.24s
541:	learn: 0.3997619	total: 2.76s	remaining: 3.23s
542:	learn: 0.3997433	total: 2.76s	remaining: 3.23s
543:	learn: 0.3997145	total: 2.77s	remaining: 3.22s
544:	learn: 0.3996982	total: 2.77s	remaining: 3.22s
545:	learn: 0.3996840	total: 2.78s	remaining: 3.21s
546:	learn: 0.3996629	total: 2.78s	remaining: 3.21s
547:	learn: 0.3996205	total: 2.79s	remaining: 3.2s
548:	learn: 0.3995833	total: 2.79s	remaining: 3.19s
549:	learn: 0.3995778	total: 2.8s	remaining: 3.19s
550:	learn: 0.3995694	total: 2.8s	remaining: 3.18s
551:	learn: 0.3995640	total: 2.81s	remaining: 3.18s
552:	learn: 0.3995602	total: 2.81s	remaining: 3.17s
553:	learn: 0.3995413	total: 2.82s	remaining: 3.17s
554:	learn: 0.3995308	total: 2.82s	remaining: 3.16s
555:	learn: 0.3995141	total: 2.83s	remaining: 3.16s
556:	learn: 0.3995118	total: 2.83s	remaining: 3.15s
557:	learn: 0.3994847	total: 2.83s	remaining: 3.15s
558:	learn: 0.3994846	total: 2.84s	remaining: 3.14s
559:	learn: 0.3994703	total: 2.85s	remaining: 3.13s
560:	learn: 0.3994693	total: 2.85s	remaining: 3.13s
561:	learn: 0.3994674	total: 2.86s	remaining: 3.13s
562:	learn: 0.3994651	total: 2.86s	remaining: 3.12s
563:	learn: 0.3994546	total: 2.87s	remaining: 3.12s
564:	learn: 0.3994447	total: 2.87s	remaining: 3.11s
565:	learn: 0.3994347	total: 2.88s	remaining: 3.1s
566:	learn: 0.3994308	total: 2.88s	remaining: 3.1s
567:	learn: 0.3993891	total: 2.89s	remaining: 3.09s
568:	learn: 0.3993876	total: 2.89s	remaining: 3.09s

569:	learn:	0.3993753	total:	2.9s	remaining:	3.08s
570:	learn:	0.3993613	total:	2.9s	remaining:	3.08s
571:	learn:	0.3993244	total:	2.91s	remaining:	3.07s
572:	learn:	0.3993143	total:	2.91s	remaining:	3.07s
573:	learn:	0.3993139	total:	2.92s	remaining:	3.06s
574:	learn:	0.3993064	total:	2.92s	remaining:	3.06s
575:	learn:	0.3993035	total:	2.93s	remaining:	3.06s
576:	learn:	0.3992928	total:	2.93s	remaining:	3.05s
577:	learn:	0.3992850	total:	2.94s	remaining:	3.04s
578:	learn:	0.3992423	total:	2.94s	remaining:	3.04s
579:	learn:	0.3992324	total:	2.95s	remaining:	3.03s
580:	learn:	0.3991917	total:	2.95s	remaining:	3.03s
581:	learn:	0.3991910	total:	2.96s	remaining:	3.02s
582:	learn:	0.3991627	total:	2.96s	remaining:	3.02s
583:	learn:	0.3991488	total:	2.97s	remaining:	3.01s
584:	learn:	0.3991246	total:	2.97s	remaining:	3.01s
585:	learn:	0.3991234	total:	2.98s	remaining:	3s
586:	learn:	0.3991196	total:	2.98s	remaining:	3s
587:	learn:	0.3991173	total:	2.99s	remaining:	2.99s
588:	learn:	0.3991108	total:	2.99s	remaining:	2.99s
589:	learn:	0.3990992	total:	3s	remaining:	2.98s
590:	learn:	0.3990781	total:	3s	remaining:	2.98s
591:	learn:	0.3990725	total:	3s	remaining:	2.97s
592:	learn:	0.3990649	total:	3.01s	remaining:	2.96s
593:	learn:	0.3990209	total:	3.02s	remaining:	2.96s
594:	learn:	0.3990169	total:	3.02s	remaining:	2.95s
595:	learn:	0.3990120	total:	3.02s	remaining:	2.95s
596:	learn:	0.3990068	total:	3.03s	remaining:	2.94s
597:	learn:	0.3990006	total:	3.03s	remaining:	2.94s
598:	learn:	0.3989527	total:	3.04s	remaining:	2.93s
599:	learn:	0.3989137	total:	3.04s	remaining:	2.93s
600:	learn:	0.3988868	total:	3.05s	remaining:	2.92s
601:	learn:	0.3988564	total:	3.05s	remaining:	2.92s
602:	learn:	0.3988510	total:	3.06s	remaining:	2.91s
603:	learn:	0.3988449	total:	3.06s	remaining:	2.9s
604:	learn:	0.3988145	total:	3.07s	remaining:	2.9s
605:	learn:	0.3988081	total:	3.07s	remaining:	2.9s
606:	learn:	0.3987917	total:	3.08s	remaining:	2.89s
607:	learn:	0.3987865	total:	3.08s	remaining:	2.88s
608:	learn:	0.3987844	total:	3.09s	remaining:	2.88s
609:	learn:	0.3987743	total:	3.09s	remaining:	2.87s
610:	learn:	0.3987263	total:	3.1s	remaining:	2.87s
611:	learn:	0.3987233	total:	3.1s	remaining:	2.86s
612:	learn:	0.3987186	total:	3.11s	remaining:	2.86s
613:	learn:	0.3987095	total:	3.11s	remaining:	2.85s
614:	learn:	0.3987026	total:	3.12s	remaining:	2.85s
615:	learn:	0.3987022	total:	3.12s	remaining:	2.84s
616:	learn:	0.3987019	total:	3.13s	remaining:	2.84s
617:	learn:	0.3986710	total:	3.13s	remaining:	2.83s

618:	learn:	0.3986695	total:	3.14s	remaining:	2.83s
619:	learn:	0.3986440	total:	3.14s	remaining:	2.82s
620:	learn:	0.3986416	total:	3.15s	remaining:	2.82s
621:	learn:	0.3986376	total:	3.15s	remaining:	2.81s
622:	learn:	0.3986285	total:	3.16s	remaining:	2.81s
623:	learn:	0.3986283	total:	3.16s	remaining:	2.8s
624:	learn:	0.3986213	total:	3.17s	remaining:	2.8s
625:	learn:	0.3985980	total:	3.17s	remaining:	2.79s
626:	learn:	0.3985691	total:	3.17s	remaining:	2.79s
627:	learn:	0.3985608	total:	3.18s	remaining:	2.78s
628:	learn:	0.3985276	total:	3.19s	remaining:	2.77s
629:	learn:	0.3984964	total:	3.19s	remaining:	2.77s
630:	learn:	0.3984909	total:	3.19s	remaining:	2.76s
631:	learn:	0.3984674	total:	3.2s	remaining:	2.76s
632:	learn:	0.3984599	total:	3.2s	remaining:	2.75s
633:	learn:	0.3984331	total:	3.21s	remaining:	2.75s
634:	learn:	0.3984274	total:	3.21s	remaining:	2.74s
635:	learn:	0.3984274	total:	3.22s	remaining:	2.74s
636:	learn:	0.3984129	total:	3.22s	remaining:	2.73s
637:	learn:	0.3983662	total:	3.23s	remaining:	2.73s
638:	learn:	0.3983543	total:	3.23s	remaining:	2.72s
639:	learn:	0.3983324	total:	3.24s	remaining:	2.72s
640:	learn:	0.3982780	total:	3.24s	remaining:	2.71s
641:	learn:	0.3982730	total:	3.25s	remaining:	2.71s
642:	learn:	0.3982525	total:	3.25s	remaining:	2.7s
643:	learn:	0.3982449	total:	3.26s	remaining:	2.7s
644:	learn:	0.3982136	total:	3.26s	remaining:	2.69s
645:	learn:	0.3982132	total:	3.27s	remaining:	2.69s
646:	learn:	0.3982117	total:	3.27s	remaining:	2.68s
647:	learn:	0.3981964	total:	3.28s	remaining:	2.67s
648:	learn:	0.3981910	total:	3.28s	remaining:	2.67s
649:	learn:	0.3981833	total:	3.29s	remaining:	2.67s
650:	learn:	0.3981819	total:	3.29s	remaining:	2.66s
651:	learn:	0.3981801	total:	3.3s	remaining:	2.65s
652:	learn:	0.3981750	total:	3.3s	remaining:	2.65s
653:	learn:	0.3981602	total:	3.31s	remaining:	2.64s
654:	learn:	0.3981251	total:	3.31s	remaining:	2.64s
655:	learn:	0.3981234	total:	3.32s	remaining:	2.63s
656:	learn:	0.3980889	total:	3.32s	remaining:	2.63s
657:	learn:	0.3980677	total:	3.33s	remaining:	2.62s
658:	learn:	0.3980586	total:	3.33s	remaining:	2.62s
659:	learn:	0.3980238	total:	3.34s	remaining:	2.61s
660:	learn:	0.3980160	total:	3.34s	remaining:	2.61s
661:	learn:	0.3979184	total:	3.35s	remaining:	2.6s
662:	learn:	0.3979182	total:	3.35s	remaining:	2.6s
663:	learn:	0.3979106	total:	3.35s	remaining:	2.59s
664:	learn:	0.3979084	total:	3.36s	remaining:	2.59s
665:	learn:	0.3978997	total:	3.37s	remaining:	2.58s
666:	learn:	0.3978596	total:	3.37s	remaining:	2.58s

667:	learn:	0.3978257	total:	3.37s	remaining:	2.57s
668:	learn:	0.3977894	total:	3.38s	remaining:	2.57s
669:	learn:	0.3977795	total:	3.38s	remaining:	2.56s
670:	learn:	0.3977754	total:	3.39s	remaining:	2.56s
671:	learn:	0.3977668	total:	3.39s	remaining:	2.55s
672:	learn:	0.3977666	total:	3.4s	remaining:	2.54s
673:	learn:	0.3977510	total:	3.4s	remaining:	2.54s
674:	learn:	0.3977443	total:	3.41s	remaining:	2.53s
675:	learn:	0.3977265	total:	3.41s	remaining:	2.53s
676:	learn:	0.3977184	total:	3.42s	remaining:	2.52s
677:	learn:	0.3976875	total:	3.42s	remaining:	2.52s
678:	learn:	0.3976743	total:	3.43s	remaining:	2.51s
679:	learn:	0.3976499	total:	3.43s	remaining:	2.51s
680:	learn:	0.3976443	total:	3.44s	remaining:	2.5s
681:	learn:	0.3976216	total:	3.44s	remaining:	2.5s
682:	learn:	0.3975675	total:	3.44s	remaining:	2.49s
683:	learn:	0.3975627	total:	3.45s	remaining:	2.49s
684:	learn:	0.3975452	total:	3.45s	remaining:	2.48s
685:	learn:	0.3975427	total:	3.46s	remaining:	2.48s
686:	learn:	0.3975414	total:	3.46s	remaining:	2.47s
687:	learn:	0.3975306	total:	3.47s	remaining:	2.46s
688:	learn:	0.3975040	total:	3.47s	remaining:	2.46s
689:	learn:	0.3975029	total:	3.48s	remaining:	2.45s
690:	learn:	0.3974853	total:	3.48s	remaining:	2.45s
691:	learn:	0.3974851	total:	3.49s	remaining:	2.44s
692:	learn:	0.3974762	total:	3.49s	remaining:	2.44s
693:	learn:	0.3974697	total:	3.5s	remaining:	2.43s
694:	learn:	0.3974578	total:	3.5s	remaining:	2.43s
695:	learn:	0.3974196	total:	3.51s	remaining:	2.42s
696:	learn:	0.3973770	total:	3.51s	remaining:	2.42s
697:	learn:	0.3973698	total:	3.52s	remaining:	2.41s
698:	learn:	0.3973682	total:	3.52s	remaining:	2.41s
699:	learn:	0.3973371	total:	3.53s	remaining:	2.4s
700:	learn:	0.3973331	total:	3.53s	remaining:	2.4s
701:	learn:	0.3973270	total:	3.54s	remaining:	2.39s
702:	learn:	0.3973237	total:	3.54s	remaining:	2.39s
703:	learn:	0.3973150	total:	3.55s	remaining:	2.38s
704:	learn:	0.3972712	total:	3.55s	remaining:	2.38s
705:	learn:	0.3972706	total:	3.56s	remaining:	2.37s
706:	learn:	0.3972671	total:	3.56s	remaining:	2.37s
707:	learn:	0.3972661	total:	3.57s	remaining:	2.36s
708:	learn:	0.3972659	total:	3.57s	remaining:	2.36s
709:	learn:	0.3972578	total:	3.58s	remaining:	2.35s
710:	learn:	0.3972453	total:	3.58s	remaining:	2.35s
711:	learn:	0.3971999	total:	3.59s	remaining:	2.34s
712:	learn:	0.3971775	total:	3.59s	remaining:	2.34s
713:	learn:	0.3971699	total:	3.6s	remaining:	2.33s
714:	learn:	0.3971676	total:	3.6s	remaining:	2.33s
715:	learn:	0.3971635	total:	3.61s	remaining:	2.32s

716:	learn:	0.3971628	total:	3.61s	remaining:	2.32s
717:	learn:	0.3971628	total:	3.62s	remaining:	2.31s
718:	learn:	0.3971588	total:	3.62s	remaining:	2.31s
719:	learn:	0.3971407	total:	3.63s	remaining:	2.3s
720:	learn:	0.3971385	total:	3.63s	remaining:	2.3s
721:	learn:	0.3970891	total:	3.63s	remaining:	2.29s
722:	learn:	0.3970881	total:	3.64s	remaining:	2.29s
723:	learn:	0.3970880	total:	3.65s	remaining:	2.28s
724:	learn:	0.3970679	total:	3.65s	remaining:	2.27s
725:	learn:	0.3970664	total:	3.65s	remaining:	2.27s
726:	learn:	0.3970597	total:	3.66s	remaining:	2.27s
727:	learn:	0.3970494	total:	3.67s	remaining:	2.26s
728:	learn:	0.3970253	total:	3.67s	remaining:	2.25s
729:	learn:	0.3970243	total:	3.67s	remaining:	2.25s
730:	learn:	0.3970198	total:	3.68s	remaining:	2.25s
731:	learn:	0.3970178	total:	3.68s	remaining:	2.24s
732:	learn:	0.3970026	total:	3.69s	remaining:	2.23s
733:	learn:	0.3969913	total:	3.69s	remaining:	2.23s
734:	learn:	0.3969799	total:	3.7s	remaining:	2.22s
735:	learn:	0.3969798	total:	3.7s	remaining:	2.22s
736:	learn:	0.3969767	total:	3.71s	remaining:	2.21s
737:	learn:	0.3969393	total:	3.71s	remaining:	2.21s
738:	learn:	0.3969335	total:	3.72s	remaining:	2.2s
739:	learn:	0.3969196	total:	3.72s	remaining:	2.2s
740:	learn:	0.3969088	total:	3.73s	remaining:	2.19s
741:	learn:	0.3969059	total:	3.73s	remaining:	2.19s
742:	learn:	0.3969026	total:	3.74s	remaining:	2.18s
743:	learn:	0.3968796	total:	3.74s	remaining:	2.18s
744:	learn:	0.3968760	total:	3.75s	remaining:	2.17s
745:	learn:	0.3968755	total:	3.75s	remaining:	2.17s
746:	learn:	0.3968728	total:	3.76s	remaining:	2.16s
747:	learn:	0.3968673	total:	3.76s	remaining:	2.16s
748:	learn:	0.3968632	total:	3.77s	remaining:	2.15s
749:	learn:	0.3968622	total:	3.77s	remaining:	2.15s
750:	learn:	0.3968575	total:	3.78s	remaining:	2.14s
751:	learn:	0.3968463	total:	3.78s	remaining:	2.14s
752:	learn:	0.3968255	total:	3.79s	remaining:	2.13s
753:	learn:	0.3968092	total:	3.79s	remaining:	2.13s
754:	learn:	0.3968044	total:	3.79s	remaining:	2.12s
755:	learn:	0.3967824	total:	3.8s	remaining:	2.12s
756:	learn:	0.3967612	total:	3.8s	remaining:	2.11s
757:	learn:	0.3967551	total:	3.81s	remaining:	2.11s
758:	learn:	0.3967485	total:	3.81s	remaining:	2.1s
759:	learn:	0.3967433	total:	3.82s	remaining:	2.09s
760:	learn:	0.3967243	total:	3.82s	remaining:	2.09s
761:	learn:	0.3967168	total:	3.83s	remaining:	2.08s
762:	learn:	0.3967154	total:	3.83s	remaining:	2.08s
763:	learn:	0.3967125	total:	3.84s	remaining:	2.07s
764:	learn:	0.3967115	total:	3.84s	remaining:	2.07s

765:	learn:	0.3966829	total:	3.85s	remaining:	2.06s
766:	learn:	0.3966805	total:	3.85s	remaining:	2.06s
767:	learn:	0.3966667	total:	3.86s	remaining:	2.05s
768:	learn:	0.3966602	total:	3.86s	remaining:	2.05s
769:	learn:	0.3966501	total:	3.87s	remaining:	2.04s
770:	learn:	0.3966485	total:	3.87s	remaining:	2.04s
771:	learn:	0.3966329	total:	3.88s	remaining:	2.03s
772:	learn:	0.3966162	total:	3.88s	remaining:	2.03s
773:	learn:	0.3966018	total:	3.88s	remaining:	2.02s
774:	learn:	0.3965728	total:	3.89s	remaining:	2.02s
775:	learn:	0.3965632	total:	3.9s	remaining:	2.01s
776:	learn:	0.3965422	total:	3.9s	remaining:	2.01s
777:	learn:	0.3965300	total:	3.9s	remaining:	2s
778:	learn:	0.3965300	total:	3.91s	remaining:	2s
779:	learn:	0.3965190	total:	3.91s	remaining:	1.99s
780:	learn:	0.3964944	total:	3.92s	remaining:	1.99s
781:	learn:	0.3964847	total:	3.92s	remaining:	1.98s
782:	learn:	0.3964479	total:	3.93s	remaining:	1.98s
783:	learn:	0.3963684	total:	3.93s	remaining:	1.97s
784:	learn:	0.3963656	total:	3.94s	remaining:	1.97s
785:	learn:	0.3963270	total:	3.94s	remaining:	1.96s
786:	learn:	0.3963269	total:	3.95s	remaining:	1.96s
787:	learn:	0.3963215	total:	3.95s	remaining:	1.95s
788:	learn:	0.3963172	total:	3.96s	remaining:	1.95s
789:	learn:	0.3963093	total:	3.96s	remaining:	1.94s
790:	learn:	0.3963087	total:	3.97s	remaining:	1.94s
791:	learn:	0.3962995	total:	3.98s	remaining:	1.93s
792:	learn:	0.3962981	total:	3.98s	remaining:	1.93s
793:	learn:	0.3962603	total:	3.99s	remaining:	1.92s
794:	learn:	0.3962562	total:	3.99s	remaining:	1.92s
795:	learn:	0.3962467	total:	4s	remaining:	1.91s
796:	learn:	0.3962121	total:	4s	remaining:	1.91s
797:	learn:	0.3961832	total:	4s	remaining:	1.9s
798:	learn:	0.3961821	total:	4.01s	remaining:	1.9s
799:	learn:	0.3961745	total:	4.01s	remaining:	1.89s
800:	learn:	0.3961615	total:	4.02s	remaining:	1.89s
801:	learn:	0.3961451	total:	4.02s	remaining:	1.88s
802:	learn:	0.3961306	total:	4.03s	remaining:	1.88s
803:	learn:	0.3961282	total:	4.03s	remaining:	1.87s
804:	learn:	0.3961208	total:	4.04s	remaining:	1.87s
805:	learn:	0.3961167	total:	4.04s	remaining:	1.86s
806:	learn:	0.3961138	total:	4.05s	remaining:	1.86s
807:	learn:	0.3960920	total:	4.05s	remaining:	1.85s
808:	learn:	0.3960838	total:	4.06s	remaining:	1.85s
809:	learn:	0.3960815	total:	4.06s	remaining:	1.84s
810:	learn:	0.3960778	total:	4.07s	remaining:	1.84s
811:	learn:	0.3960664	total:	4.07s	remaining:	1.83s
812:	learn:	0.3960442	total:	4.08s	remaining:	1.83s
813:	learn:	0.3960120	total:	4.08s	remaining:	1.82s

814:	learn:	0.3959825	total:	4.09s	remaining:	1.82s
815:	learn:	0.3959788	total:	4.09s	remaining:	1.81s
816:	learn:	0.3959644	total:	4.1s	remaining:	1.8s
817:	learn:	0.3958899	total:	4.1s	remaining:	1.8s
818:	learn:	0.3958792	total:	4.11s	remaining:	1.79s
819:	learn:	0.3958535	total:	4.11s	remaining:	1.79s
820:	learn:	0.3958485	total:	4.12s	remaining:	1.78s
821:	learn:	0.3958237	total:	4.12s	remaining:	1.78s
822:	learn:	0.3958032	total:	4.13s	remaining:	1.77s
823:	learn:	0.3957831	total:	4.13s	remaining:	1.77s
824:	learn:	0.3957802	total:	4.13s	remaining:	1.76s
825:	learn:	0.3957711	total:	4.14s	remaining:	1.76s
826:	learn:	0.3957658	total:	4.14s	remaining:	1.75s
827:	learn:	0.3957581	total:	4.15s	remaining:	1.75s
828:	learn:	0.3957466	total:	4.15s	remaining:	1.74s
829:	learn:	0.3957206	total:	4.16s	remaining:	1.74s
830:	learn:	0.3957023	total:	4.16s	remaining:	1.73s
831:	learn:	0.3956931	total:	4.17s	remaining:	1.73s
832:	learn:	0.3956640	total:	4.17s	remaining:	1.72s
833:	learn:	0.3956618	total:	4.18s	remaining:	1.72s
834:	learn:	0.3956422	total:	4.18s	remaining:	1.71s
835:	learn:	0.3956422	total:	4.19s	remaining:	1.71s
836:	learn:	0.3956352	total:	4.19s	remaining:	1.7s
837:	learn:	0.3956092	total:	4.2s	remaining:	1.7s
838:	learn:	0.3956089	total:	4.2s	remaining:	1.69s
839:	learn:	0.3956055	total:	4.21s	remaining:	1.69s
840:	learn:	0.3955989	total:	4.21s	remaining:	1.68s
841:	learn:	0.3955675	total:	4.22s	remaining:	1.68s
842:	learn:	0.3955625	total:	4.22s	remaining:	1.67s
843:	learn:	0.3955427	total:	4.23s	remaining:	1.67s
844:	learn:	0.3955329	total:	4.23s	remaining:	1.66s
845:	learn:	0.3955260	total:	4.24s	remaining:	1.66s
846:	learn:	0.3955190	total:	4.24s	remaining:	1.65s
847:	learn:	0.3955149	total:	4.25s	remaining:	1.65s
848:	learn:	0.3955013	total:	4.25s	remaining:	1.64s
849:	learn:	0.3954985	total:	4.26s	remaining:	1.64s
850:	learn:	0.3954809	total:	4.26s	remaining:	1.63s
851:	learn:	0.3954733	total:	4.27s	remaining:	1.63s
852:	learn:	0.3954582	total:	4.27s	remaining:	1.62s
853:	learn:	0.3954346	total:	4.28s	remaining:	1.62s
854:	learn:	0.3954213	total:	4.28s	remaining:	1.61s
855:	learn:	0.3954101	total:	4.29s	remaining:	1.61s
856:	learn:	0.3953791	total:	4.29s	remaining:	1.6s
857:	learn:	0.3953760	total:	4.29s	remaining:	1.6s
858:	learn:	0.3953698	total:	4.3s	remaining:	1.59s
859:	learn:	0.3953566	total:	4.3s	remaining:	1.59s
860:	learn:	0.3953566	total:	4.31s	remaining:	1.58s
861:	learn:	0.3953534	total:	4.31s	remaining:	1.58s
862:	learn:	0.3953291	total:	4.32s	remaining:	1.57s

863:	learn:	0.3953279	total:	4.32s	remaining:	1.57s
864:	learn:	0.3953138	total:	4.33s	remaining:	1.56s
865:	learn:	0.3953029	total:	4.33s	remaining:	1.56s
866:	learn:	0.3952975	total:	4.34s	remaining:	1.55s
867:	learn:	0.3952773	total:	4.34s	remaining:	1.54s
868:	learn:	0.3952753	total:	4.35s	remaining:	1.54s
869:	learn:	0.3952666	total:	4.35s	remaining:	1.53s
870:	learn:	0.3952663	total:	4.36s	remaining:	1.53s
871:	learn:	0.3952562	total:	4.36s	remaining:	1.52s
872:	learn:	0.3952384	total:	4.37s	remaining:	1.52s
873:	learn:	0.3952294	total:	4.37s	remaining:	1.51s
874:	learn:	0.3952268	total:	4.38s	remaining:	1.51s
875:	learn:	0.3952259	total:	4.38s	remaining:	1.5s
876:	learn:	0.3952070	total:	4.39s	remaining:	1.5s
877:	learn:	0.3952031	total:	4.39s	remaining:	1.5s
878:	learn:	0.3952020	total:	4.4s	remaining:	1.49s
879:	learn:	0.3952018	total:	4.4s	remaining:	1.49s
880:	learn:	0.3951977	total:	4.41s	remaining:	1.48s
881:	learn:	0.3951936	total:	4.41s	remaining:	1.48s
882:	learn:	0.3951882	total:	4.42s	remaining:	1.47s
883:	learn:	0.3951854	total:	4.42s	remaining:	1.47s
884:	learn:	0.3951772	total:	4.43s	remaining:	1.46s
885:	learn:	0.3951481	total:	4.43s	remaining:	1.46s
886:	learn:	0.3951434	total:	4.44s	remaining:	1.45s
887:	learn:	0.3951391	total:	4.45s	remaining:	1.45s
888:	learn:	0.3951379	total:	4.45s	remaining:	1.44s
889:	learn:	0.3951244	total:	4.46s	remaining:	1.44s
890:	learn:	0.3950975	total:	4.46s	remaining:	1.43s
891:	learn:	0.3950963	total:	4.47s	remaining:	1.43s
892:	learn:	0.3950957	total:	4.47s	remaining:	1.42s
893:	learn:	0.3950922	total:	4.48s	remaining:	1.42s
894:	learn:	0.3950917	total:	4.49s	remaining:	1.41s
895:	learn:	0.3950848	total:	4.49s	remaining:	1.41s
896:	learn:	0.3950490	total:	4.5s	remaining:	1.4s
897:	learn:	0.3950328	total:	4.5s	remaining:	1.4s
898:	learn:	0.3950229	total:	4.51s	remaining:	1.39s
899:	learn:	0.3950185	total:	4.51s	remaining:	1.39s
900:	learn:	0.3950124	total:	4.52s	remaining:	1.38s
901:	learn:	0.3950081	total:	4.52s	remaining:	1.38s
902:	learn:	0.3950081	total:	4.53s	remaining:	1.37s
903:	learn:	0.3950026	total:	4.54s	remaining:	1.37s
904:	learn:	0.3949990	total:	4.54s	remaining:	1.36s
905:	learn:	0.3949925	total:	4.54s	remaining:	1.36s
906:	learn:	0.3949908	total:	4.55s	remaining:	1.35s
907:	learn:	0.3949821	total:	4.56s	remaining:	1.35s
908:	learn:	0.3949820	total:	4.56s	remaining:	1.34s
909:	learn:	0.3949734	total:	4.57s	remaining:	1.34s
910:	learn:	0.3949611	total:	4.57s	remaining:	1.33s
911:	learn:	0.3949415	total:	4.58s	remaining:	1.33s

912:	learn:	0.3949409	total:	4.58s	remaining:	1.32s
913:	learn:	0.3949313	total:	4.59s	remaining:	1.32s
914:	learn:	0.3949306	total:	4.6s	remaining:	1.32s
915:	learn:	0.3949259	total:	4.6s	remaining:	1.31s
916:	learn:	0.3949222	total:	4.61s	remaining:	1.31s
917:	learn:	0.3949143	total:	4.61s	remaining:	1.3s
918:	learn:	0.3949142	total:	4.62s	remaining:	1.3s
919:	learn:	0.3948994	total:	4.62s	remaining:	1.29s
920:	learn:	0.3948966	total:	4.63s	remaining:	1.29s
921:	learn:	0.3948738	total:	4.63s	remaining:	1.28s
922:	learn:	0.3948353	total:	4.64s	remaining:	1.28s
923:	learn:	0.3948085	total:	4.64s	remaining:	1.27s
924:	learn:	0.3947980	total:	4.65s	remaining:	1.27s
925:	learn:	0.3947960	total:	4.66s	remaining:	1.26s
926:	learn:	0.3947823	total:	4.66s	remaining:	1.26s
927:	learn:	0.3947756	total:	4.67s	remaining:	1.25s
928:	learn:	0.3947680	total:	4.67s	remaining:	1.25s
929:	learn:	0.3947373	total:	4.67s	remaining:	1.24s
930:	learn:	0.3947323	total:	4.68s	remaining:	1.24s
931:	learn:	0.3947170	total:	4.69s	remaining:	1.23s
932:	learn:	0.3946862	total:	4.69s	remaining:	1.23s
933:	learn:	0.3946528	total:	4.7s	remaining:	1.22s
934:	learn:	0.3946420	total:	4.7s	remaining:	1.22s
935:	learn:	0.3946348	total:	4.71s	remaining:	1.21s
936:	learn:	0.3946276	total:	4.71s	remaining:	1.21s
937:	learn:	0.3946155	total:	4.72s	remaining:	1.2s
938:	learn:	0.3945991	total:	4.72s	remaining:	1.2s
939:	learn:	0.3945744	total:	4.73s	remaining:	1.19s
940:	learn:	0.3945703	total:	4.73s	remaining:	1.19s
941:	learn:	0.3945462	total:	4.74s	remaining:	1.18s
942:	learn:	0.3945457	total:	4.74s	remaining:	1.18s
943:	learn:	0.3945417	total:	4.75s	remaining:	1.17s
944:	learn:	0.3945357	total:	4.75s	remaining:	1.17s
945:	learn:	0.3945318	total:	4.76s	remaining:	1.16s
946:	learn:	0.3945102	total:	4.76s	remaining:	1.16s
947:	learn:	0.3944973	total:	4.77s	remaining:	1.15s
948:	learn:	0.3944944	total:	4.77s	remaining:	1.15s
949:	learn:	0.3944940	total:	4.78s	remaining:	1.14s
950:	learn:	0.3944828	total:	4.78s	remaining:	1.14s
951:	learn:	0.3944827	total:	4.79s	remaining:	1.13s
952:	learn:	0.3944789	total:	4.79s	remaining:	1.13s
953:	learn:	0.3944412	total:	4.8s	remaining:	1.12s
954:	learn:	0.3944397	total:	4.8s	remaining:	1.12s
955:	learn:	0.3944055	total:	4.81s	remaining:	1.11s
956:	learn:	0.3944028	total:	4.81s	remaining:	1.11s
957:	learn:	0.3943845	total:	4.82s	remaining:	1.1s
958:	learn:	0.3943784	total:	4.82s	remaining:	1.1s
959:	learn:	0.3943472	total:	4.83s	remaining:	1.09s
960:	learn:	0.3943412	total:	4.83s	remaining:	1.09s

961:	learn:	0.3943366	total:	4.84s	remaining:	1.08s
962:	learn:	0.3943191	total:	4.84s	remaining:	1.08s
963:	learn:	0.3942945	total:	4.85s	remaining:	1.07s
964:	learn:	0.3942812	total:	4.86s	remaining:	1.07s
965:	learn:	0.3942705	total:	4.86s	remaining:	1.06s
966:	learn:	0.3942697	total:	4.87s	remaining:	1.06s
967:	learn:	0.3942411	total:	4.87s	remaining:	1.05s
968:	learn:	0.3942224	total:	4.88s	remaining:	1.05s
969:	learn:	0.3942025	total:	4.88s	remaining:	1.04s
970:	learn:	0.3942025	total:	4.89s	remaining:	1.04s
971:	learn:	0.3941995	total:	4.89s	remaining:	1.03s
972:	learn:	0.3941797	total:	4.9s	remaining:	1.03s
973:	learn:	0.3941084	total:	4.9s	remaining:	1.02s
974:	learn:	0.3941047	total:	4.91s	remaining:	1.02s
975:	learn:	0.3940819	total:	4.91s	remaining:	1.01s
976:	learn:	0.3940816	total:	4.92s	remaining:	1.01s
977:	learn:	0.3940546	total:	4.92s	remaining:	1s
978:	learn:	0.3940495	total:	4.93s	remaining:	997ms
979:	learn:	0.3940488	total:	4.93s	remaining:	992ms
980:	learn:	0.3940458	total:	4.94s	remaining:	987ms
981:	learn:	0.3940370	total:	4.95s	remaining:	982ms
982:	learn:	0.3940349	total:	4.95s	remaining:	977ms
983:	learn:	0.3940349	total:	4.95s	remaining:	972ms
984:	learn:	0.3940228	total:	4.96s	remaining:	967ms
985:	learn:	0.3940137	total:	4.96s	remaining:	962ms
986:	learn:	0.3940039	total:	4.97s	remaining:	957ms
987:	learn:	0.3939781	total:	4.98s	remaining:	952ms
988:	learn:	0.3939676	total:	4.98s	remaining:	947ms
989:	learn:	0.3939664	total:	5s	remaining:	944ms
990:	learn:	0.3939636	total:	5s	remaining:	939ms
991:	learn:	0.3938639	total:	5.01s	remaining:	935ms
992:	learn:	0.3938114	total:	5.02s	remaining:	930ms
993:	learn:	0.3938103	total:	5.03s	remaining:	925ms
994:	learn:	0.3938058	total:	5.03s	remaining:	920ms
995:	learn:	0.3938053	total:	5.04s	remaining:	915ms
996:	learn:	0.3937922	total:	5.04s	remaining:	910ms
997:	learn:	0.3937892	total:	5.05s	remaining:	905ms
998:	learn:	0.3937880	total:	5.05s	remaining:	900ms
999:	learn:	0.3937561	total:	5.06s	remaining:	895ms
1000:	learn:	0.3937516	total:	5.06s	remaining:	890ms
1001:	learn:	0.3937318	total:	5.07s	remaining:	885ms
1002:	learn:	0.3937297	total:	5.07s	remaining:	880ms
1003:	learn:	0.3937286	total:	5.08s	remaining:	875ms
1004:	learn:	0.3937282	total:	5.08s	remaining:	870ms
1005:	learn:	0.3937255	total:	5.09s	remaining:	865ms
1006:	learn:	0.3937123	total:	5.09s	remaining:	860ms
1007:	learn:	0.3937046	total:	5.11s	remaining:	857ms
1008:	learn:	0.3937000	total:	5.12s	remaining:	852ms
1009:	learn:	0.3936831	total:	5.13s	remaining:	848ms

1010: learn: 0.3936564 total: 5.14s	remaining: 844ms
1011: learn: 0.3936498 total: 5.15s	remaining: 839ms
1012: learn: 0.3936037 total: 5.15s	remaining: 834ms
1013: learn: 0.3936033 total: 5.16s	remaining: 830ms
1014: learn: 0.3935970 total: 5.17s	remaining: 825ms
1015: learn: 0.3935938 total: 5.18s	remaining: 820ms
1016: learn: 0.3935895 total: 5.18s	remaining: 815ms
1017: learn: 0.3935860 total: 5.19s	remaining: 811ms
1018: learn: 0.3935747 total: 5.2s	remaining: 806ms
1019: learn: 0.3935536 total: 5.2s	remaining: 801ms
1020: learn: 0.3935473 total: 5.21s	remaining: 796ms
1021: learn: 0.3935461 total: 5.21s	remaining: 791ms
1022: learn: 0.3935334 total: 5.22s	remaining: 786ms
1023: learn: 0.3935169 total: 5.23s	remaining: 781ms
1024: learn: 0.3935124 total: 5.23s	remaining: 776ms
1025: learn: 0.3935015 total: 5.24s	remaining: 771ms
1026: learn: 0.3934998 total: 5.24s	remaining: 766ms
1027: learn: 0.3934991 total: 5.25s	remaining: 761ms
1028: learn: 0.3934646 total: 5.25s	remaining: 756ms
1029: learn: 0.3934629 total: 5.26s	remaining: 751ms
1030: learn: 0.3934521 total: 5.27s	remaining: 746ms
1031: learn: 0.3934487 total: 5.27s	remaining: 741ms
1032: learn: 0.3934328 total: 5.28s	remaining: 736ms
1033: learn: 0.3934315 total: 5.28s	remaining: 731ms
1034: learn: 0.3934294 total: 5.29s	remaining: 726ms
1035: learn: 0.3934263 total: 5.3s	remaining: 721ms
1036: learn: 0.3934016 total: 5.31s	remaining: 717ms
1037: learn: 0.3933839 total: 5.32s	remaining: 712ms
1038: learn: 0.3933733 total: 5.33s	remaining: 708ms
1039: learn: 0.3933356 total: 5.34s	remaining: 703ms
1040: learn: 0.3933270 total: 5.34s	remaining: 698ms
1041: learn: 0.3933128 total: 5.35s	remaining: 693ms
1042: learn: 0.3932991 total: 5.36s	remaining: 689ms
1043: learn: 0.3932748 total: 5.37s	remaining: 684ms
1044: learn: 0.3932707 total: 5.38s	remaining: 680ms
1045: learn: 0.3932609 total: 5.39s	remaining: 675ms
1046: learn: 0.3932357 total: 5.39s	remaining: 670ms
1047: learn: 0.3932338 total: 5.4s	remaining: 665ms
1048: learn: 0.3932321 total: 5.41s	remaining: 660ms
1049: learn: 0.3932248 total: 5.41s	remaining: 655ms
1050: learn: 0.3932169 total: 5.42s	remaining: 650ms
1051: learn: 0.3932167 total: 5.42s	remaining: 645ms
1052: learn: 0.3932159 total: 5.43s	remaining: 639ms
1053: learn: 0.3931936 total: 5.44s	remaining: 634ms
1054: learn: 0.3931752 total: 5.44s	remaining: 629ms
1055: learn: 0.3931744 total: 5.45s	remaining: 624ms
1056: learn: 0.3931643 total: 5.45s	remaining: 619ms
1057: learn: 0.3931605 total: 5.46s	remaining: 614ms
1058: learn: 0.3931415 total: 5.47s	remaining: 609ms
1059: learn: 0.3931372 total: 5.47s	remaining: 604ms

1060: learn: 0.3931161 total: 5.48s	remaining: 599ms
1061: learn: 0.3931157 total: 5.48s	remaining: 594ms
1062: learn: 0.3931064 total: 5.49s	remaining: 589ms
1063: learn: 0.3930978 total: 5.49s	remaining: 583ms
1064: learn: 0.3930928 total: 5.5s	remaining: 578ms
1065: learn: 0.3930907 total: 5.5s	remaining: 573ms
1066: learn: 0.3930829 total: 5.51s	remaining: 568ms
1067: learn: 0.3930800 total: 5.51s	remaining: 563ms
1068: learn: 0.3930772 total: 5.52s	remaining: 558ms
1069: learn: 0.3930748 total: 5.52s	remaining: 552ms
1070: learn: 0.3930739 total: 5.53s	remaining: 547ms
1071: learn: 0.3930649 total: 5.53s	remaining: 542ms
1072: learn: 0.3930649 total: 5.54s	remaining: 537ms
1073: learn: 0.3930393 total: 5.54s	remaining: 532ms
1074: learn: 0.3930085 total: 5.55s	remaining: 526ms
1075: learn: 0.3930067 total: 5.55s	remaining: 521ms
1076: learn: 0.3929995 total: 5.56s	remaining: 516ms
1077: learn: 0.3929913 total: 5.56s	remaining: 511ms
1078: learn: 0.3929730 total: 5.57s	remaining: 506ms
1079: learn: 0.3929576 total: 5.57s	remaining: 500ms
1080: learn: 0.3929558 total: 5.58s	remaining: 495ms
1081: learn: 0.3929525 total: 5.58s	remaining: 490ms
1082: learn: 0.3929449 total: 5.59s	remaining: 485ms
1083: learn: 0.3929426 total: 5.59s	remaining: 480ms
1084: learn: 0.3929391 total: 5.6s	remaining: 475ms
1085: learn: 0.3929292 total: 5.6s	remaining: 469ms
1086: learn: 0.3929087 total: 5.61s	remaining: 464ms
1087: learn: 0.3929047 total: 5.62s	remaining: 459ms
1088: learn: 0.3929035 total: 5.62s	remaining: 454ms
1089: learn: 0.3929029 total: 5.63s	remaining: 449ms
1090: learn: 0.3929022 total: 5.63s	remaining: 444ms
1091: learn: 0.3928914 total: 5.64s	remaining: 439ms
1092: learn: 0.3928847 total: 5.65s	remaining: 434ms
1093: learn: 0.3928714 total: 5.65s	remaining: 429ms
1094: learn: 0.3928529 total: 5.66s	remaining: 424ms
1095: learn: 0.3928510 total: 5.66s	remaining: 419ms
1096: learn: 0.3928444 total: 5.67s	remaining: 413ms
1097: learn: 0.3928311 total: 5.67s	remaining: 408ms
1098: learn: 0.3928199 total: 5.68s	remaining: 403ms
1099: learn: 0.3927952 total: 5.68s	remaining: 398ms
1100: learn: 0.3927886 total: 5.69s	remaining: 393ms
1101: learn: 0.3927837 total: 5.69s	remaining: 387ms
1102: learn: 0.3927704 total: 5.7s	remaining: 382ms
1103: learn: 0.3927678 total: 5.7s	remaining: 377ms
1104: learn: 0.3927672 total: 5.71s	remaining: 372ms
1105: learn: 0.3927631 total: 5.71s	remaining: 367ms
1106: learn: 0.3927506 total: 5.71s	remaining: 361ms
1107: learn: 0.3927337 total: 5.72s	remaining: 356ms
1108: learn: 0.3927330 total: 5.72s	remaining: 351ms

1109: learn: 0.3927189 total: 5.73s	remaining: 346ms
1110: learn: 0.3926873 total: 5.73s	remaining: 341ms
1111: learn: 0.3926801 total: 5.74s	remaining: 335ms
1112: learn: 0.3926394 total: 5.74s	remaining: 330ms
1113: learn: 0.3926387 total: 5.75s	remaining: 325ms
1114: learn: 0.3926359 total: 5.75s	remaining: 320ms
1115: learn: 0.3926325 total: 5.76s	remaining: 315ms
1116: learn: 0.3926262 total: 5.76s	remaining: 310ms
1117: learn: 0.3925999 total: 5.77s	remaining: 304ms
1118: learn: 0.3925928 total: 5.77s	remaining: 299ms
1119: learn: 0.3925787 total: 5.78s	remaining: 294ms
1120: learn: 0.3925547 total: 5.78s	remaining: 289ms
1121: learn: 0.3925535 total: 5.79s	remaining: 284ms
1122: learn: 0.3924896 total: 5.79s	remaining: 279ms
1123: learn: 0.3924802 total: 5.8s	remaining: 273ms
1124: learn: 0.3924716 total: 5.8s	remaining: 268ms
1125: learn: 0.3924706 total: 5.81s	remaining: 263ms
1126: learn: 0.3924654 total: 5.81s	remaining: 258ms
1127: learn: 0.3924516 total: 5.82s	remaining: 253ms
1128: learn: 0.3924475 total: 5.82s	remaining: 248ms
1129: learn: 0.3924380 total: 5.83s	remaining: 242ms
1130: learn: 0.3924282 total: 5.83s	remaining: 237ms
1131: learn: 0.3924228 total: 5.84s	remaining: 232ms
1132: learn: 0.3924206 total: 5.84s	remaining: 227ms
1133: learn: 0.3924151 total: 5.85s	remaining: 222ms
1134: learn: 0.3923983 total: 5.85s	remaining: 217ms
1135: learn: 0.3923828 total: 5.86s	remaining: 211ms
1136: learn: 0.3923828 total: 5.86s	remaining: 206ms
1137: learn: 0.3923763 total: 5.87s	remaining: 201ms
1138: learn: 0.3923735 total: 5.87s	remaining: 196ms
1139: learn: 0.3923723 total: 5.88s	remaining: 191ms
1140: learn: 0.3923530 total: 5.88s	remaining: 186ms
1141: learn: 0.3923503 total: 5.88s	remaining: 180ms
1142: learn: 0.3923503 total: 5.89s	remaining: 175ms
1143: learn: 0.3923434 total: 5.89s	remaining: 170ms
1144: learn: 0.3923405 total: 5.9s	remaining: 165ms
1145: learn: 0.3923389 total: 5.9s	remaining: 160ms
1146: learn: 0.3923372 total: 5.91s	remaining: 155ms
1147: learn: 0.3923352 total: 5.91s	remaining: 149ms
1148: learn: 0.3923297 total: 5.92s	remaining: 144ms
1149: learn: 0.3923187 total: 5.92s	remaining: 139ms
1150: learn: 0.3922929 total: 5.93s	remaining: 134ms
1151: learn: 0.3922908 total: 5.93s	remaining: 129ms
1152: learn: 0.3922707 total: 5.94s	remaining: 124ms
1153: learn: 0.3922655 total: 5.94s	remaining: 118ms
1154: learn: 0.3922530 total: 5.95s	remaining: 113ms
1155: learn: 0.3922336 total: 5.95s	remaining: 108ms
1156: learn: 0.3922330 total: 5.96s	remaining: 103ms
1157: learn: 0.3922215 total: 5.96s	remaining: 97.9ms

```
1158: learn: 0.3921325 total: 5.97s    remaining: 92.7ms
1159: learn: 0.3921301 total: 5.97s    remaining: 87.5ms
1160: learn: 0.3921286 total: 5.98s    remaining: 82.4ms
1161: learn: 0.3921136 total: 5.98s    remaining: 77.2ms
1162: learn: 0.3920872 total: 5.99s    remaining: 72.1ms
1163: learn: 0.3920504 total: 5.99s    remaining: 66.9ms
1164: learn: 0.3920323 total: 6s      remaining: 61.8ms
1165: learn: 0.3920126 total: 6s      remaining: 56.6ms
1166: learn: 0.3920086 total: 6s      remaining: 51.5ms
1167: learn: 0.3919999 total: 6.01s    remaining: 46.3ms
1168: learn: 0.3919711 total: 6.02s    remaining: 41.2ms
1169: learn: 0.3919675 total: 6.02s    remaining: 36ms
1170: learn: 0.3919668 total: 6.03s    remaining: 30.9ms
1171: learn: 0.3919004 total: 6.03s    remaining: 25.7ms
1172: learn: 0.3919004 total: 6.03s    remaining: 20.6ms
1173: learn: 0.3918893 total: 6.04s    remaining: 15.4ms
1174: learn: 0.3918882 total: 6.04s    remaining: 10.3ms
1175: learn: 0.3918807 total: 6.05s    remaining: 5.14ms
1176: learn: 0.3918782 total: 6.05s    remaining: 0us
```

```
<catboost.core.CatBoostClassifier at 0x274c24c5e20>
```

```
# Результаты
```

```
y_pred_train_cb_optuna = cb_optuna.predict_proba(train_pool_full)[: ,  
1]
```

```
y_pred_test_cb_optuna = cb_optuna.predict_proba(test_pool)[: , 1]
```

```
roc_auc_cb_optuna = roc_auc_score(y_train, y_pred_train_cb_optuna)
```

```
print(f'Train ROC-AUC: {roc_auc_cb_optuna:.4f}')
```

```
# Train ROC-AUC: 0.8587
```

```
Train ROC-AUC: 0.8587
```

```
submission['Churn'] = y_pred_test_cb_optuna
```

```
submission.to_csv(root + 'my_submission_cb_optuna.csv', index=False)
```

ROC-AUC вашего решения равен 0.8537932650357107.

Стекинг

Будем стакать модели разной природы.

```
# Подготовка данных
```

```
best_features = cb_features
```

```

num_features = [col for col in num_cols if col in best_features]
cat_features = [col for col in cat_cols if col in best_features]

X_train = train_generated[num_features + cat_features].copy()
y_train = train_generated['Churn'].copy()
X_test = test_generated[num_features + cat_features].copy()

X_train_stack, X_val_stack, y_train_stack, y_val_stack =
train_test_split(X_train, y_train,

test_size=0.2,

random_state=RANDOM_STATE,

stratify=y_train)

```

Обучим еще несколько моделей

KNN

```

def objective_knn(trial):
    params = {
        'n_neighbors': trial.suggest_int('n_neighbors', 3, 51,
step=2),
        'weights': trial.suggest_categorical('weights',
['uniform', 'distance']),
        'metric': trial.suggest_categorical('metric',
['euclidean', 'manhattan', 'minkowski']),
        'n_jobs': -1
    }

    if params['metric'] == 'minkowski':
        params['p'] = trial.suggest_int('p', 1, 2)

    # Разделяем данные
    X_train_knn, X_val_knn, y_train_knn, y_val_knn =
train_test_split(X_train_stack,

y_train_stack,

test_size=0.25,

random_state=RANDOM_STATE,

stratify=y_train_stack

)

    preprocessor = ColumnTransformer([
        ('num', StandardScaler(), num_features),

```

```

        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
    ])

    X_train_processed = preprocessor.fit_transform(X_train_knn)
    X_val_processed = preprocessor.transform(X_val_knn)

    model = KNeighborsClassifier(**params)
    model.fit(X_train_processed, y_train_knn)

    y_pred = model.predict_proba(X_val_processed)[: , 1]

    return roc_auc_score(y_val_knn, y_pred)

# study_knn = optuna.create_study(direction='maximize')
# study_knn.optimize(objective_knn, n_trials=100,
# show_progress_bar=True)

# best_params_knn = study_knn.best_params
# best_params_knn
# # {'n_neighbors': 37, 'weights': 'uniform', 'metric': 'minkowski',
# 'p': 1}

preprocessor_knn = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])

X_train_knn = preprocessor_knn.fit_transform(X_train_stack)
y_train_knn = y_train_stack
X_test_knn = preprocessor_knn.transform(X_test)

best_params_knn = {'n_neighbors': 37, 'weights': 'uniform', 'metric':
'minkowski', 'p': 1}

knn_optuna = KNeighborsClassifier(**best_params_knn, n_jobs=-1)
knn_optuna.fit(X_train_knn, y_train_knn)

KNeighborsClassifier(n_jobs=-1, n_neighbors=37, p=1)

# Результаты

y_pred_train_knn_optuna = knn_optuna.predict_proba(X_train_knn)[: , 1]
y_pred_test_knn_optuna = knn_optuna.predict_proba(X_test_knn)[: , 1]

roc_auc_knn_optuna = roc_auc_score(y_train_knn,
y_pred_train_knn_optuna)

print(f'Train ROC-AUC: {roc_auc_knn_optuna:.4f}')

# Train ROC-AUC: 0.8510

Train ROC-AUC: 0.8510

```

```
# submission['Churn'] = y_pred_test_knn_optuna
# submission.to_csv(root + 'my_submission_knn_optuna.csv',
index=False)
```

ROC-AUC вашего решения равен 0.8385660731021556.

Random Forest

```
def objective_rf(trial):
    params = {
        'n_estimators':      trial.suggest_int('n_estimators', 100,
500, step=50),
        'max_depth':         trial.suggest_int('max_depth', 3, 15),
        'min_samples_split': trial.suggest_int('min_samples_split',
2, 20),
        'min_samples_leaf':  trial.suggest_int('min_samples_leaf',
1, 10),
        'max_features':
trial.suggest_categorical('max_features', ['sqrt', 'log2', None]),
        'class_weight':
trial.suggest_categorical('class_weight', ['balanced',
'balanced_subsample', None]),
        'random_state':      RANDOM_STATE,
        'n_jobs':            -1
    }

    X_train_rf, X_val_rf, y_train_rf, y_val_rf =
train_test_split(X_train_stack,
y_train_stack,
test_size=0.25,
random_state=RANDOM_STATE,
stratify=y_train_stack

                    )

    preprocessor = ColumnTransformer([
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
    ])

    X_train_processed = preprocessor.fit_transform(X_train_rf)
    X_val_processed = preprocessor.transform(X_val_rf)

    model = RandomForestClassifier(**params)
    model.fit(X_train_processed, y_train_rf)

    y_pred = model.predict_proba(X_val_processed)[: , 1]
```

```

    return roc_auc_score(y_val_rf, y_pred)

# study_rf = optuna.create_study(direction='maximize')
# study_rf.optimize(objective_rf, n_trials=100,
# show_progress_bar=True)

# best_params_rf = study_rf.best_params
# best_params_rf

# # {'n_estimators': 400,
# #   'max_depth': 8,
# #   'min_samples_split': 3,
# #   'min_samples_leaf': 9,
# #   'max_features': 'log2',
# #   'class_weight': None}

preprocessor_rf = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])

X_train_rf = preprocessor_rf.fit_transform(X_train_stack)
y_train_rf = y_train_stack
X_test_rf = preprocessor_rf.transform(X_test)

best_params_rf = {'n_estimators': 400,
                  'max_depth': 8,
                  'min_samples_split': 3,
                  'min_samples_leaf': 9,
                  'max_features': 'log2',
                  'class_weight': None}

rf_optuna = RandomForestClassifier(**best_params_rf)
rf_optuna.fit(X_train_rf, y_train_rf)

RandomForestClassifier(max_depth=8, max_features='log2',
min_samples_leaf=9,
                    min_samples_split=3, n_estimators=400)

# Результаты

y_pred_train_rf_optuna = rf_optuna.predict_proba(X_train_rf)[: , 1]
y_pred_test_rf_optuna = rf_optuna.predict_proba(X_test_rf)[: , 1]

roc_auc_rf_optuna = roc_auc_score(y_train_rf, y_pred_train_rf_optuna)

print(f'Train ROC-AUC: {roc_auc_rf_optuna:.4f}')

# Train ROC-AUC: 0.8890

Train ROC-AUC: 0.8881

```

```
# submission['Churn'] = y_pred_test_rf_optuna
# submission.to_csv(root + 'my_submission_rf_optuna.csv', index=False)
```

ROC-AUC вашего решения равен 0.8489100927511877.

LightGBM

```
X_train_enc = X_train_stack.copy()
X_val_enc = X_val_stack.copy()
X_test_enc = X_test.copy()

# Предобработка категориальных признаков для lgb
for col in cat_features:
    le = LabelEncoder()
    X_train_enc[col] = le.fit_transform(X_train_enc[col].astype(str))
    X_val_enc[col] = le.transform(X_val_enc[col].astype(str))
    X_test_enc[col] = le.transform(X_test_enc[col].astype(str))

def objective_lgb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100,
600, step=50),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate',
0.01, 0.1, log=True),
        'num_leaves': trial.suggest_int('num_leaves', 10,
80),
        'subsample': trial.suggest_float('subsample', 0.6,
0.9),
        'colsample_bytree':
trial.suggest_float('colsample_bytree', 0.6, 0.9),
        'reg_alpha': trial.suggest_float('reg_alpha', 0,
3),
        'reg_lambda': trial.suggest_float('reg_lambda', 0,
3),
        'min_child_samples': trial.suggest_int('min_child_samples',
10, 50),
        'random_state': RANDOM_STATE,
        'verbose': -1,
        'n_jobs': -1
    }

    model = LGBMClassifier(**params)
    model.fit(X_train_enc, y_train_stack)

    y_pred = model.predict_proba(X_val_enc)[ :, 1]

    return roc_auc_score(y_val_stack, y_pred)
```

```

# study_lgb = optuna.create_study(direction='maximize')
# study_lgb.optimize(objective_lgb, n_trials=1000,
# show_progress_bar=True)

# best_params_lgb = study_lgb.best_params
# best_params_lgb

# # {'n_estimators': 200,
# #   'max_depth': 3,
# #   'learning_rate': 0.03824588822076221,
# #   'num_leaves': 32,
# #   'subsample': 0.6467389927045808,
# #   'colsample_bytree': 0.83606154972336,
# #   'reg_alpha': 2.9495885705020424,
# #   'reg_lambda': 2.4889422819779723,
# #   'min_child_samples': 38}

X_train_lgb = X_train.copy()
X_test_lgb = X_test.copy()

for col in cat_features:
    le = LabelEncoder()
    X_train_lgb[col] = le.fit_transform(X_train_lgb[col].astype(str))
    X_test_lgb[col] = le.transform(X_test_lgb[col].astype(str))

best_params_lgb = {'n_estimators': 200,
                    'max_depth': 3,
                    'learning_rate': 0.03824588822076221,
                    'num_leaves': 32,
                    'subsample': 0.6467389927045808,
                    'colsample_bytree': 0.83606154972336,
                    'reg_alpha': 2.9495885705020424,
                    'reg_lambda': 2.4889422819779723,
                    'min_child_samples': 38}

lgb_optuna = LGBMClassifier(**best_params_lgb,
                             random_state=RANDOM_STATE, verbose=-1, n_jobs=-1)

lgb_optuna.fit(X_train_lgb, y_train)

LGBMClassifier(colsample_bytree=0.83606154972336,
                learning_rate=0.03824588822076221, max_depth=3,
                min_child_samples=38, n_estimators=200, n_jobs=-1,
                num_leaves=32,
                random_state=42, reg_alpha=2.9495885705020424,
                reg_lambda=2.4889422819779723,
                subsample=0.6467389927045808,
                verbose=-1)

# Результаты

```

```

y_pred_train_lgb_optuna = lgb_optuna.predict_proba(X_train_lgb)[: , 1]
y_pred_test_lgb_optuna = lgb_optuna.predict_proba(X_test_lgb)[: , 1]

roc_auc_rf_optuna = roc_auc_score(y_train_rf, y_pred_train_rf_optuna)

print(f'Train ROC-AUC: {roc_auc_rf_optuna:.4f}')

# Train ROC-AUC: 0.8889

Train ROC-AUC: 0.8881

# submission['Churn'] = y_pred_test_lgb_optuna
# submission.to_csv(root + 'my_submission_lgb_optuna.csv',
index=False)

```

ROC-AUC вашего решения равен 0.8505946417606567.

Комбинируем разные модели с лучшими результатами

```

# Подготовка данных

num_cols = [
    'ClientPeriod',
    'AverageMonthlySpend',
    'HasMultipleServices',
    'NotAlone',
    'TotalSpent',
    'MonthlySpending',
    'ClientPeriodGroup',
    'IsAloneSenior',
    'IsSeniorCitizen',
]

cat_cols = [
    'PaymentMethod',
    'IsBillingPaperless',
    'HasContractPhone',
    'HasInternetService',
    'Sex',
    'HasPartner',
    'HasChild',
    'HasPhoneService',
    'HasMultiplePhoneNumbers',
    'HasOnlineSecurityService',
    'HasOnlineBackup',
    'HasDeviceProtection',
    'HasTechSupportAccess',
    'HasOnlineTV',
    'HasMovieSubscription',
]

```

```

best_features = [
    'HasContractPhone',
    'ClientPeriod',
    'AverageMonthlySpend',
    'TotalSpent',
    'MonthlySpending',
    'HasOnlineSecurityService',
    'HasMultiplePhoneNumbers',
    'PaymentMethod',
    'HasInternetService',
    'ClientPeriodGroup',
    'HasOnlineBackup',
    'HasTechSupportAccess',
    'IsBillingPaperless',
    'HasMovieSubscription',
    'Sex',
    'HasMultipleServices',
    'HasDeviceProtection',
    'HasOnlineTV',
    'IsSeniorCitizen',
    'HasChild'
]

num_features = [col for col in num_cols if col in best_features]
cat_features = [col for col in cat_cols if col in best_features]

X_train = train_generated[num_features + cat_features].copy()
y_train = train_generated['Churn'].copy()
X_test = test_generated[num_features + cat_features].copy()

# Собираем пайплайны моделей, прописываем лучшие параметры из прошлых шагов

from sklearn.preprocessing import OneHotEncoder

# lr
preprocessor_lr = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_features)
    ])

best_params_lr = {'C': 16.774054266041745, 'solver': 'lbfgs',
    'max_iter': 843}

lr_model = LogisticRegression(**best_params_lr,
    random_state=RANDOM_STATE,
    n_jobs=-1)

```

```

pipeline_lr = Pipeline([
    ('preprocessor', preprocessor_lr),
    ('classifier', lr_model)
])

# cb
preprocessor_cb = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_features)
    ])

best_params_cb = {
    'iterations': 1177,
    'learning_rate': 0.04352833222664741,
    'depth': 2,
    'l2_leaf_reg': 8.712627216223945,
    'random_strength': 18.956025845929155,
    'bagging_temperature': 1.1644875612616532,
    'min_data_in_leaf': 40,
    'subsample': 0.5472929227565183,
    'colsample_bylevel': 0.4608651413955162,
    'max_bin': 16
}

cb_model = CatBoostClassifier(**best_params_cb,
                              random_seed=RANDOM_STATE,
                              verbose=False,
                              eval_metric='AUC')

pipeline_cb = Pipeline([
    ('preprocessor', preprocessor_cb),
    ('classifier', cb_model)
])

# knn
preprocessor_knn = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_features)
    ])

best_params_knn = {'n_neighbors': 37, 'weights': 'uniform', 'metric':
'minkowski', 'p': 1}

```

```

knn_model = KNeighborsClassifier(**best_params_knn, n_jobs=-1)

pipeline_knn = Pipeline([
    ('preprocessor', preprocessor_knn),
    ('classifier', knn_model)
])

# rf
preprocessor_rf = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_features)
    ])

best_params_rf = {
    'n_estimators': 400,
    'max_depth': 8,
    'min_samples_split': 3,
    'min_samples_leaf': 9,
    'max_features': 'log2',
    'class_weight': None
}

rf_model = RandomForestClassifier(**best_params_rf,
random_state=RANDOM_STATE, n_jobs=-1)

pipeline_rf = Pipeline([
    ('preprocessor', preprocessor_rf),
    ('classifier', rf_model)
])

# lgb
preprocessor_lgb = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', OneHotEncoder(handle_unknown='ignore',
sparse_output=False), cat_features)
    ])

best_params_lgb = {
    'n_estimators': 200,
    'max_depth': 3,
    'learning_rate': 0.03824588822076221,
    'num_leaves': 32,
    'subsample': 0.6467389927045808,
    'colsample_bytree': 0.83606154972336,
    'reg_alpha': 2.9495885705020424,

```

```

        'reg_lambda': 2.4889422819779723,
        'min_child_samples': 38
    }

lgb_model = LGBMClassifier(**best_params_lgb,
random_state=RANDOM_STATE, verbose=-1, n_jobs=-1)

pipeline_lgb = Pipeline([
    ('preprocessor', preprocessor_lgb),
    ('classifier', lgb_model)
])

final_estimator = CatBoostClassifier(iterations=500,
                                     learning_rate=0.03,
                                     depth=3,
                                     random_seed=RANDOM_STATE,
                                     verbose=False,
                                     eval_metric='AUC')

estimators = [
    ('lr', pipeline_lr),
    ('cb', pipeline_cb),
    ('knn', pipeline_knn),
    ('rf', pipeline_rf),
    # ('lgb', pipeline_lgb), # c lgb xyже
]

stacking_model = StackingClassifier(estimators=estimators,
                                   final_estimator=final_estimator,
                                   cv=5,
                                   stack_method='predict_proba',
                                   n_jobs=-1)

stacking_model.fit(X_train, y_train)

StackingClassifier(cv=5,
                  estimators=[('lr',
                              Pipeline(steps=[('preprocessor',
ColumnTransformer(transformers=[('num',
StandardScaler(),
['ClientPeriod',
'AverageMonthlySpend',
'HasMultipleServices',
'TotalSpent',

```

```

'MonthlySpending',
'ClientPeriodGroup',
'IsSeniorCitizen']]),
('cat',
OneHotEncoder(handle_unknown='ignore',
sparse_output=False),
['PaymentMethod',
'IsBil...
'HasDeviceProtection',
'HasTechSupportAccess',
'HasOnlineTV',
'HasMovieSubscription']]))),
('classifier',
RandomForestClassifier(max_depth=8,
max_features='log2',
min_samples_leaf=9,
min_samples_split=3,
n_estimators=400,
n_jobs=-1,
random_state=42)))]],
final_estimator=<catboost.core.CatBoostClassifier
object at 0x00000274B050D730>,
n_jobs=-1, stack_method='predict_proba')

# Результаты
y_pred_train_stacking = stacking_model.predict_proba(X_train)[:, 1]
y_pred_test_stacking = stacking_model.predict_proba(X_test)[:, 1]
roc_auc_stacking = roc_auc_score(y_train, y_pred_train_stacking)

```

```
print(f'Train ROC-AUC: {roc_auc_stacking:.4f}')
# Train ROC-AUC: 0.8686
Train ROC-AUC: 0.8686
# submission['Churn'] = y_pred_test_stacking
# submission.to_csv(root + 'my_submission_stacking.csv', index=False)
```

ROC-AUC вашего решения равен 0.8485861099440908. Хуже чем просто CatBoost с подбором через optuna (0.8537932650357107), но зато стекинг реализовал 😊

Блендинг

```
# Просто по интуиции расставим веса вручную ☺

pred_blending = 0.2 * y_pred_test_lr_optuna + \
                0.5 * y_pred_test_cb_optuna + \
                0.05 * y_pred_test_knn_optuna + \
                0.1 * y_pred_test_rf_optuna + \
                0.15 * y_pred_test_lgb_optuna

submission['Churn'] = pred_blending
submission.to_csv(root + 'my_submission_blending.csv', index=False)
```

ROC-AUC вашего решения равен 0.8530192612222474.

Итоги

Модели, которые неплохо себя показали:

Модель	ROC-AUC
LogisticRegression с optuna	0.8470340626312898
CatBoostClassifier base	0.8514413599198526
CatBoostClassifier с optuna	0.8537932650357107
RandomForestClassifier с optuna	0.8489100927511877
KNeighborsClassifier с optuna	0.8385660731021556
LGBMClassifier с optuna	0.8505946417606567
Stacking	0.8485861099440908
Blending	0.8530192612222474

Лучшая модель **CatBoostClassifier с optuna** с результатом **0.8537932650357107**.

```
# ROC-кривая

y_pred_train = cb_optuna.predict_proba(X_train)[: , 1]
y_pred_test = cb_optuna.predict_proba(X_test)[: , 1]

# Вычисляем ROC-кривую
fpr, tpr, thresholds = roc_curve(y_train, y_pred_train)
roc_auc = auc(fpr, tpr)

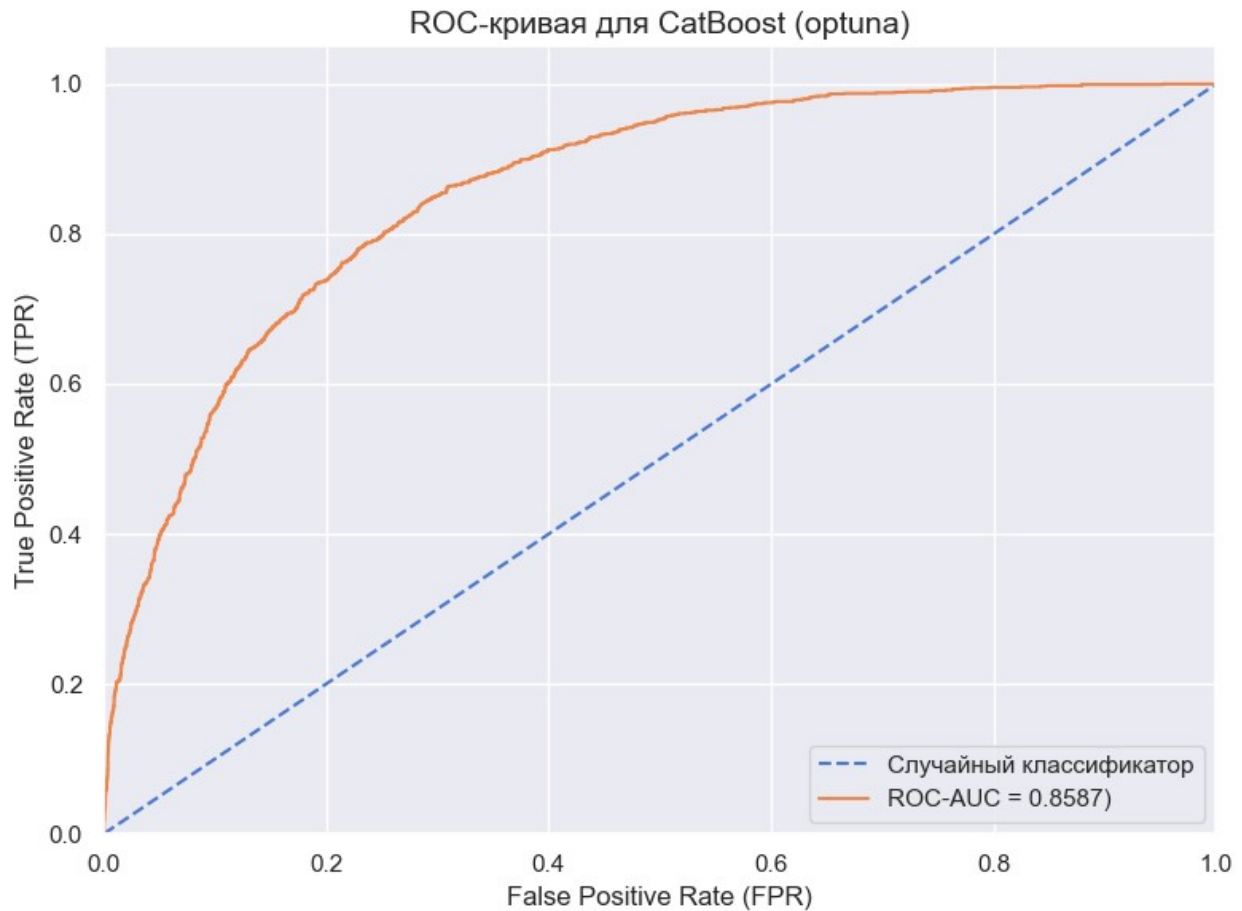
# Строим график
plt.figure(figsize=(8, 6))
plt.plot([0, 1], [0, 1], linestyle='--', label='Случайный
классификатор')
plt.plot(fpr, tpr, label=f'ROC-AUC = {roc_auc:.4f}')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])

plt.xlabel('False Positive Rate (FPR)', fontsize=12)
plt.ylabel('True Positive Rate (TPR)', fontsize=12)

plt.title('ROC-кривая для CatBoost (optuna)', fontsize=14)
plt.legend(loc='lower right')

plt.tight_layout()

plt.show();
```



Улучшить результат можно следующими способами:

- Сгенерировать новые признаки;
- Выделить из существующих признаков значения-триггеры;
- Подобрать другой набор признаков для обучения;
- Попробовать PolynomialFeatures
- Попробовать взять логарифм для некоторых признаков
- Индивидуально подойти к кодировке категориальных признаков (разные энкодеры);
- Для генерации признаков использовать нейронные сети;
- При блендинге осуществить подбор коэффициентов α ($\alpha * \text{pred_model}$).